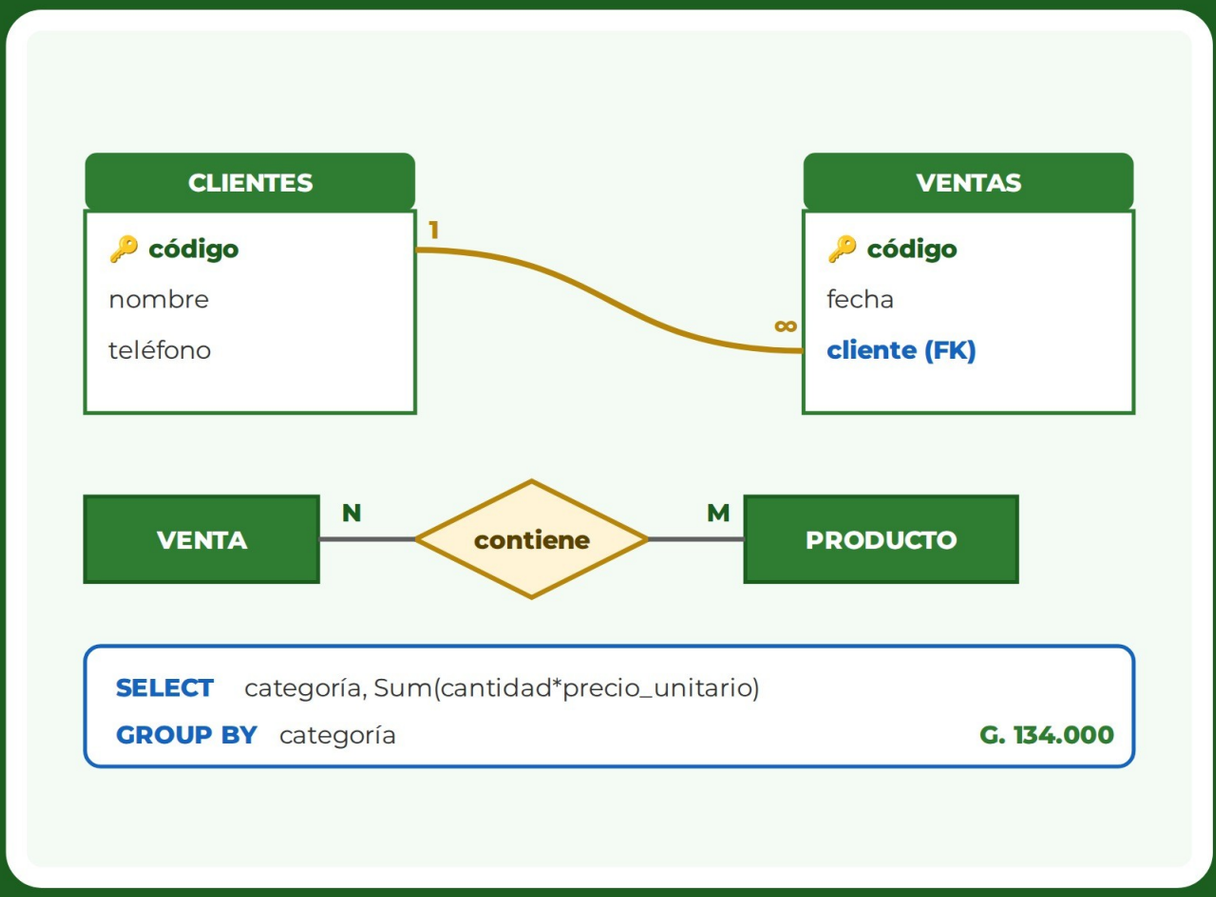
BACHILLERATO TÉCNICO EN SERVICIOS · ESPECIALIDAD INFORMÁTICA

Lenguajes y bases de datos

ALGORÍTMICA

3.er Curso



Libro del estudiante con prácticas de laboratorio

Enfoque «Aprender Haciendo» · PSeInt y Microsoft Access

Caso integrador: Copetín Karumbé

Programa de estudio del Ministerio de Educación y Ciencias · Diseño Curricular de Informática

República del Paraguay · 2026

Presentación

Este libro cierra la serie de Algorítmica del Bachillerato Técnico en Servicios, Especialidad Informática, conforme al programa oficial del Ministerio de Educación y Ciencias. Reúne en un solo volumen las 21 clases del año y sus 21 prácticas: ya no hace falta un cuaderno aparte.

Vas a recorrer dos grandes temas. Primero, los lenguajes de programación: qué son, cómo se traducen, qué paradigmas existen y cómo se elige uno. Después, las bases de datos: qué resuelven, cómo se diseñan con el modelo Entidad-Relación, cómo se normalizan y cómo se crean y consultan en Microsoft Access.

Cada clase abre con una ficha de capacidad e indicadores, desarrolla el tema con ejemplos resueltos, tablas y figuras, y cierra con actividades para ejercitar, resolver y decidir. Enseguida viene la Práctica con el mismo número: se hace en PSeInt, en papel o en Access, con datos nuevos, pasos guiados y puntos de control para que verifiques tu trabajo.

El caso que nos acompaña es el Copetín Karumbé, un emprendimiento gastronómico paraguayo. En las prácticas aparecen además una biblioteca escolar, un club de barrio, los talleres del colegio, una cantina y una librería, porque diseñar bases de datos es reconocer los mismos patrones en negocios distintos. El año termina con el Proyecto Final Integrador para la Feria de Informática.

Capacidad transversal del programa MEC: Reconoce la importancia de la práctica Conductual en el desarrollo de las actividades. Esta capacidad se desarrolla al cumplir procedimientos, compartir roles, proteger datos, verificar resultados, documentar cambios y participar responsablemente en las prácticas y el proyecto.

Cómo está organizado este libro

Pieza	Qué encontrás
Clase N	Ficha con la capacidad del programa oficial e indicadores · desarrollo con ejemplos resueltos · recuadros Concepto clave, Ejemplo, Errores frecuentes y, cuando corresponde, En Paraguay o Aplicación profesional · actividades de aplicación.
Práctica N	Empieza en página propia y se puede fotocopiar suelta: competencia, lo que necesitás saber, actividades con objetivo, modelo, pasos y punto de control, y un desafío final. Algunas suman una transferencia con revisión entre pares.
Punto de control	Te dice cómo comprobar tu trabajo, sin darte la respuesta: si no se cumple, revisá tus pasos antes de seguir.
Evaluaciones	Al cierre de cada unidad (banda ámbar) y de cada etapa (banda azul), en páginas propias.

Índice

Prueba diagnóstica — ¿Qué sabés ya? **4**

UNIDAD 1 — Lenguajes de programación **5**

Clase 1 — ¿Qué es un lenguaje de programación? 5

Práctica 1 — El cobro de un pedido para llevar, en órdenes precisas y en PSeInt 8

Clase 2 — Niveles de lenguaje: de la máquina al alto nivel 9

Práctica 2 — Traductores y errores en acción 14

Clase 3 — Los cuatro paradigmas de programación 15

Práctica 3 — Paradigmas en la biblioteca y un acumulador en PSeInt 18

Clase 4 — Elegir el lenguaje adecuado 19

Práctica 4 — Elegir la tecnología para la biblioteca escolar 22

Evaluación de la Unidad 1 23

UNIDAD 2 — Bases de datos y modelo relacional **24**

Clase 5 — ¿Qué es una base de datos? 24

Práctica 5 — Del cuaderno de préstamos a datos estructurados 28

Clase 6 — Tipos y aplicaciones de bases de datos 30

Práctica 6 — El catálogo de la biblioteca como tabla relacional 34

Clase 7 — El SGBD y los problemas que resuelve 36

Práctica 7 — La cantina del colegio: problemas que resuelve un SGBD 39

Clase 8 — Tablas, campos y registros. Clave principal y foránea 40

Práctica 8 — Claves de la biblioteca 44

Clase 9 — Integridad referencial 46

Práctica 9 — La integridad referencial pone orden en la biblioteca 50

Clase 10 — La base por dentro: abstracción, usuarios y lenguajes 52

Práctica 10 — Lenguajes, usuarios y vistas de la biblioteca 55

Evaluación de la Unidad 2 57

Evaluación integradora de la 1.ª etapa 58

UNIDAD 3 — Diseño: Entidad-Relación y normalización **59**

Clase 11 — Modelos de datos: el plano antes de construir 59

Práctica 11 — El mini-mundo del club deportivo 62

Clase 12 — Entidades y atributos 63

Práctica 12 — Las entidades del club: socios y disciplinas 67

Clase 13 — Relaciones y cardinalidad 68

Práctica 13 — Cardinalidades del club, leídas en los datos 72

Clase 14 — El diagrama Entidad-Relación 73

Práctica 14 — El DER de los talleres del colegio 77

Clase 15 — Del diagrama a las tablas 78

Práctica 15 — Del DER de los talleres a las tablas 82

Clase 16 — Por qué normalizar: anomalías y dependencia funcional 83

Práctica 16 — Anomalías y dependencias en la planilla de talleres 87

Clase 17 — Formas normales: 1FN, 2FN y 3FN 88

Práctica 17 — Normalizar los pedidos de la librería Arasa'i 91

Evaluación de la Unidad 3 92

UNIDAD 4 — La base en acción: Access y consultas **93**

Clase 18 — Crear la base en Access: tablas y relaciones 93

Práctica 18 — La base de la segunda semana, creada en Access 97

Clase 19 — Filtros y consultas de selección 99

Práctica 19 — Filtros y consultas sobre la segunda semana 103

Clase 20 — Consultas paramétricas y con criterios 104

Práctica 20 — Consultas que preguntan, sobre la segunda semana 107

Clase 21 — Totales, campos calculados y referencias cruzadas 108

Práctica 21 — Totales, cálculos y cruces de la segunda semana 112

Evaluación de la Unidad 4 113

Evaluación integradora de la 2.ª etapa 114

Proyecto Final Integrador — La solución digital completa **115**

Prueba diagnóstica — ¿Qué sabés ya?

Nombre y Apellido: _____ Curso: _____ Fecha: ____ / ____ / ____

Respondé con lo que sabés hasta hoy. Esta prueba no lleva nota: sirve para que tu docente conozca tu punto de partida.

1. Escribí, con tus palabras, qué es un programa de computadora.

2. Nombrá un lenguaje de programación que hayas escuchado o usado.

3. Escribí, paso a paso, las órdenes para calcular el vuelto de una compra de G. 7.000 pagada con G. 10.000.

4. ¿Dónde creés que guarda un negocio la lista de sus productos y precios? Nombrá un ejemplo.

5. Marcá V o F: «Una computadora entiende las órdenes aunque estén incompletas o sean ambiguas».

6. ¿Usaste alguna vez una planilla (Excel) o un programa de gestión? Contá brevemente para qué.

UNIDAD 1 — Lenguajes de programación

En esta unidad vas a conocer qué es un lenguaje de programación, cómo se traduce a lo que ejecuta la máquina, qué estilos de programación existen y con qué criterios se elige el lenguaje para un problema real.

Clase 1 — ¿Qué es un lenguaje de programación?

Capacidad	Reconoce los diferentes tipos de lenguajes de programación y sus campos de aplicación.
Tema	El lenguaje de programación: definición, elementos y campos de aplicación.
Indicadores de logro	• Define lenguaje de programación con sus propias palabras. • Distingue entre lenguaje natural y lenguaje de programación. • Menciona al menos tres campos de aplicación del software.

De la idea a las instrucciones

Una computadora no entiende castellano ni guaraní: solo ejecuta instrucciones muy precisas. Un lenguaje de programación es un conjunto de reglas y símbolos que nos permite escribir esas instrucciones de una forma que la máquina pueda transformar en acciones concretas.

Programar es, entonces, expresar la solución de un problema como una secuencia de órdenes sin ambigüedad. Donde una persona entiende "cobrale al cliente", la computadora necesita el detalle: sumar los precios, aplicar el descuento, mostrar el total.

Lenguaje natural y lenguaje de programación

El lenguaje natural (el que hablamos) es flexible y tolera la ambigüedad: la misma frase puede significar cosas distintas según el contexto. Un lenguaje de programación es lo contrario: cada instrucción tiene un único significado. Por eso decimos que es un lenguaje formal, con una sintaxis (cómo se escribe) y una semántica (qué significa) bien definidas.

Ejemplo — una orden ambigua y una orden precisa

«Serviles algo dulce» es una orden natural: cada persona la interpreta distinto. «Entregá 2 chipas y cobrá G. 6.000» es una orden precisa: no deja lugar a dudas. Programar se parece a escribir órdenes del segundo tipo.

Para qué sirve: campos de aplicación

El software está en todas partes: sistemas de gestión para comercios, aplicaciones móviles, páginas web, videojuegos, control de máquinas, análisis de datos. Cada campo suele preferir ciertos lenguajes, porque no todos resuelven igual de bien los mismos problemas.

Campo de aplicación	Ejemplo cotidiano	Lenguajes frecuentes
Gestión / escritorio	Sistema de ventas de un comercio	Java, C#, Python
Aplicaciones web	Tienda en línea, página institucional	JavaScript, PHP, Python
Aplicaciones móviles	App de delivery	Kotlin, Swift
Datos y automatización	Reportes, cálculos masivos	Python, R

Concepto clave — el software es una herramienta, no un fin

El objetivo no es «saber un lenguaje», sino resolver un problema real. El Copetín Karumbé no necesita «programar por programar»: necesita un sistema que registre sus ventas y le diga cuánto recaudó. El lenguaje es el medio.

Empresas y emprendimientos desarrollan software para comercio, servicios, educación y administración. Saber programar abre oportunidades laborales y de emprendimiento, siempre según las tecnologías y necesidades de cada organización.

Sintaxis y semántica: las dos caras de un lenguaje

Todo lenguaje de programación tiene dos dimensiones. La sintaxis es el conjunto de reglas de escritura: qué palabras existen, en qué orden van, qué símbolos se usan. La semántica es el significado: qué hace realmente cada instrucción cuando se ejecuta.

Un programa puede estar perfectamente escrito (sintaxis correcta) y aun así hacer algo distinto de lo que se buscaba (error de semántica). El traductor detecta los errores de sintaxis; los de semántica solo se descubren probando el programa y comparando el resultado con el esperado.

Concepto clave — escribir bien no es lo mismo que resolver bien

Una orden puede estar bien escrita y calcular mal. Por eso todo programa se prueba con casos cuyo resultado ya se conoce de antemano: si el programa devuelve otra cosa, hay un error de semántica que corregir.

Del problema al programa: el ciclo completo

Programar no empieza en el teclado: empieza en el problema. El camino completo tiene cuatro estaciones — problema, algoritmo, programa y resultado — y se recorre tantas veces como haga falta, porque casi ningún programa queda bien en el primer intento. La figura muestra el ciclo y su vuelta atrás.



Figura 1.1 — El ciclo del desarrollo: del problema al resultado, con vuelta al algoritmo cuando algo falla.

Etapa	Pregunta que responde	Producto
Problema	¿Qué se necesita resolver?	Enunciado claro
Algoritmo	¿Con qué pasos se resuelve?	Secuencia ordenada de pasos
Programa	¿Cómo se escribe en un lenguaje?	Código fuente
Resultado	¿Funciona como se esperaba?	Salida verificable

Ejemplo resuelto — la primera orden del Copetín

El Copetín Karumbé necesita cobrar la venta de Rosa: 2 chipas y 1 gaseosa. Resolvamos el ciclo completo, paso a paso, con los precios de la tabla de productos (chipa G. 3.000, gaseosa G. 8.000).

1. Problema: calcular el total a cobrar por 2 chipas y 1 gaseosa.
2. Algoritmo: (a) tomar el precio de la chipa; (b) multiplicarlo por 2; (c) tomar el precio de la gaseosa; (d) multiplicarlo por 1; (e) sumar ambos subtotales; (f) mostrar el total.
3. Programa (pseudocódigo): subtotal1 ← 3000 × 2; subtotal2 ← 8000 × 1; total ← subtotal1 + subtotal2; Escribir total.
4. Ejecución: subtotal1 = G. 6.000; subtotal2 = G. 8.000; total = G. 14.000.
5. Verificación: la cuenta a mano da lo mismo — G. 6.000 + G. 8.000 = G. 14.000. El resultado coincide: el ciclo cierra.

Ejemplo — el mismo algoritmo, cualquier venta

El valor del algoritmo es que sirve para todas las ventas, no solo para una. Cambiando cantidades y precios, los mismos seis pasos cobran 3 empanadas y 2 gaseosas: 3 × 6.000 + 2 × 8.000 = G. 34.000. Escribir el algoritmo una vez ahorra pensarlo mil veces.

Los tres actores: programador, traductor y máquina

En todo programa intervienen tres actores con papeles bien repartidos. El programador piensa el algoritmo y lo escribe en un lenguaje. El traductor (que conocerás a fondo en la próxima clase) convierte lo escrito en instrucciones de máquina. Y la máquina ejecuta esas instrucciones al pie de la letra, sin interpretar ni adivinar: hace exactamente lo que se le ordenó, que no siempre es lo que se quiso ordenar.

Esa obediencia literal explica la exigencia de precisión de la primera sección: la máquina no completa lo que falta ni corrige lo ambiguo. Todo malentendido entre lo querido y lo escrito termina en un resultado equivocado.

Los errores también se clasifican

No todos los errores de un programa son iguales, y distinguirlos ahorra horas de búsqueda. El error de sintaxis rompe las reglas de escritura y el traductor lo detecta de inmediato. El error de ejecución aparece recién al correr el programa, cuando una operación se vuelve imposible (dividir entre cero, por ejemplo). Y el error de lógica es el más sigiloso: el programa corre sin quejarse, pero calcula otra cosa.

Tipo de error	Cuándo aparece	Quién lo detecta
De sintaxis	Al traducir	El traductor, con mensaje y línea
De ejecución	Al correr el programa	El sistema, deteniendo el programa
De lógica	Nunca se anuncia	La persona, comparando el resultado con el esperado

Ejemplo resuelto — tres errores en la caja del Copetín

El programa de cobro del Copetín falló tres veces en la semana. Diagnostiquemos cada caso.

1. Lunes: el traductor rechaza el programa y señala la línea 4, donde faltaba cerrar un paréntesis. Error de sintaxis: se corrige la escritura y listo.

2. Miércoles: al calcular el promedio por venta de un día sin ventas, el programa se detiene: intentó dividir la recaudación entre cero ventas. Error de ejecución: hay que prever el caso «sin ventas» antes de dividir.

3. Viernes: el programa cobra una venta de 2 chipas y 1 gaseosa y muestra G. 11.000 en lugar de G. 14.000: alguien escribió suma donde iba multiplicación (3.000 + 2 en vez de 3.000 × 2). Error de lógica: corrió perfecto y calculó mal.

4. Moraleja: el error más caro es el de lógica, porque nadie lo anuncia. La defensa es la de siempre: verificar contra un resultado calculado a mano.

> **Concepto clave — el mensaje de error es un aliado**
> El mensaje de error no es un reto: es información de primera mano sobre qué falló y dónde. Leerlo completo — en lugar de cerrarlo con fastidio — es el hábito que separa a quien resuelve en minutos de quien tantea durante horas.

Actividades de aplicación

Ejercitá:

1. Escribí con tus palabras qué es un lenguaje de programación.

2. Indicá dos diferencias entre el lenguaje natural y un lenguaje de programación.

3. Nombrá tres campos de aplicación del software y un ejemplo de cada uno.

4. Clasificá estas tareas según su campo: (a) app de pedidos, (b) sistema de ventas de un local, (c) página web del colegio.

Resolvé (caso Copetín Karumbé):

5. El Copetín Karumbé quiere un sistema que registre sus ventas. Indicá a qué campo de aplicación pertenece y proponé un lenguaje razonable.

6. Escribí, como órdenes precisas y numeradas, los pasos para cobrar 1 milanesa (G. 15.000) y 2 cocidos (G. 4.000 c/u). Verificá el total con una cuenta a mano.

7. El programa de cobro muestra G. 19.000 para la venta de 1 milanesa y 2 cocidos. ¿Qué tipo de error es y cuál pudo ser la causa?

Pensá y decidí (justificá tu respuesta):

8. «Cualquier lenguaje sirve igual para cualquier problema.»

9. «Si un programa no muestra ningún mensaje de error, calcula bien.»

Desafío:

10. Elegí una tarea de tu vida diaria (preparar un tereré, por ejemplo) y escribila como una secuencia de instrucciones precisas, numeradas, que otra persona pueda seguir sin preguntarte nada.

Práctica 1 — El cobro de un pedido para llevar, en órdenes precisas y en PSeInt

Nombre y Apellido: _____ Curso: _____ Fecha: ____ / ____ / ____

Competencia de la práctica
Traducir un procedimiento de cobro a instrucciones sin ambigüedad y ejecutarlo en PSeInt, verificando el resultado con un cálculo hecho a mano.
Entorno de trabajo: PSeInt.

Lo que necesitás saber

Una computadora ejecuta exactamente lo que se le ordena. Cada paso debe decir qué dato usa, qué operación hace y dónde guarda el resultado.

En PSeInt, Leer guarda en una variable lo que escribe el usuario, <- asigna un valor (en el pseudocódigo en papel lo escribimos con la flecha ←) y Escribir muestra un resultado. Si … Entonces … SiNo … FinSi elige entre dos caminos.

Abrí PSeInt y, en Configurar → Opciones del Lenguaje (perfiles), seleccioná el perfil Flexible: es el perfil de referencia de este libro.

Actividad 1 — Órdenes precisas en papel

Objetivo: *Escribir el algoritmo para cobrar un pedido para llevar: 3 mbeju (G. 5.000 c/u) y 2 cocidos (G. 4.000 c/u), con un costo de envío fijo de G. 5.000. El cliente paga con G. 50.000.*

Modelo — Estructura que debe tener tu algoritmo (sin los números)

1) Leer las cantidades y el pago.
2) Calcular el subtotal de cada producto.
3) Sumar los subtotales y el envío.
4) Calcular el vuelto.
5) Mostrar el total y el vuelto.

Pasos:

1. Numerá cada paso; ninguno puede decir «calcular» sin decir cómo.
2. Reemplazá cada precio y cada cantidad por su valor.
3. Hacé la cuenta a mano y anotá el total y el vuelto al costado.

Punto de control: El vuelto más el total deben dar exactamente lo pagado (G. 50.000). Si un compañero sigue tus pasos sin preguntarte nada, llega a los mismos números.

Actividad 2 — El mismo algoritmo, ejecutado en PSeInt

Objetivo: *Escribir el algoritmo en PSeInt, ejecutarlo y comprobar que coincide con la cuenta a mano.*

Modelo — Esqueleto para completar (las líneas «…» las escribís vos)

```
Algoritmo CobroParaLlevar
    Definir cantMbeju, cantCocido, pago, total, vuelto Como Entero
    Escribir "Cantidad de mbeju:"
    Leer cantMbeju
    …
    total <- …
    Si pago >= total Entonces
        vuelto <- pago - total
        Escribir "Total: ", total, "  Vuelto: ", vuelto
    SiNo
        Escribir "El pago no alcanza"
    FinSi
FinAlgoritmo
```

Pasos:

1. Copiá el esqueleto en PSeInt y completá las líneas que faltan (leer cantCocido y pago; calcular total con el envío).
2. Ejecutalo con el pedido del papel (3, 2 y 50.000).
3. Ejecutalo de nuevo con 1 mbeju, 1 cocido y un pago de G. 10.000.
4. Guardá el archivo como CobroParaLlevar.psc.

Punto de control: Con el primer caso, la pantalla muestra los mismos números que tu papel. Con el segundo caso, el programa no debe mostrar un vuelto negativo.

Desafío final (para quienes terminan antes)

Agregá un descuento de G. 2.000 cuando el total (con envío) supere G. 25.000. Probá que el descuento aparezca en el primer caso y no en el segundo.

Clase 2 — Niveles de lenguaje: de la máquina al alto nivel

Capacidad	Reconoce los diferentes tipos de lenguajes de programación y sus campos de aplicación.
Tema	Lenguaje de máquina, ensamblador y alto nivel. Compilador e intérprete.
Indicadores de logro	• Ordena los niveles de lenguaje del más cercano a la máquina al más cercano a la persona. • Explica la función de un traductor (compilador o intérprete). • Distingue compilador de intérprete con un ejemplo.

Tres niveles de cercanía a la máquina

No todos los lenguajes están a la misma «distancia» del hardware. El lenguaje de máquina es el único que la computadora ejecuta directamente: ceros y unos. El lenguaje ensamblador reemplaza esos números por abreviaturas (ADD, MOV), un poco más legibles. Los lenguajes de alto nivel (Python, Java) se parecen más al idioma humano y a la notación matemática.

Cuanto más alto es el nivel, más fácil es para la persona y más lejos está de la máquina. Por eso el alto nivel necesita un paso extra: alguien que traduzca ese código a lenguaje de máquina.

La figura resume la escalera completa: la misma orden desciende, traducción mediante, desde el alto nivel que escribe la persona hasta los unos y ceros que la computadora ejecuta.



Figura 2.1 — Los tres niveles de lenguaje y el camino de traducción entre ellos.

Nivel	Cómo se ve	Quién lo entiende directo
Máquina (bajo nivel)	10110000 01100001	La computadora
Ensamblador	MOV AL, 61h	La computadora después de ensamblarlo a código máquina
Alto nivel	total = 2 * 3000	La persona; requiere una cadena de traducción/ejecución

El traductor: compilador o intérprete

Para ejecutar un programa de alto nivel hace falta una implementación que traduzca o procese ese código. En un modelo introductorio podemos distinguir compilación previa e interpretación durante la ejecución, pero las implementaciones reales pueden generar código máquina, bytecode o representaciones intermedias y combinar compilación, máquina virtual, interpretación y JIT.

El esquema muestra tres recorridos habituales: compilación nativa, ejecución mediante código intermedio/máquina virtual y ejecución mediante un intérprete o runtime. Son modelos de implementación, no propiedades absolutas del lenguaje.

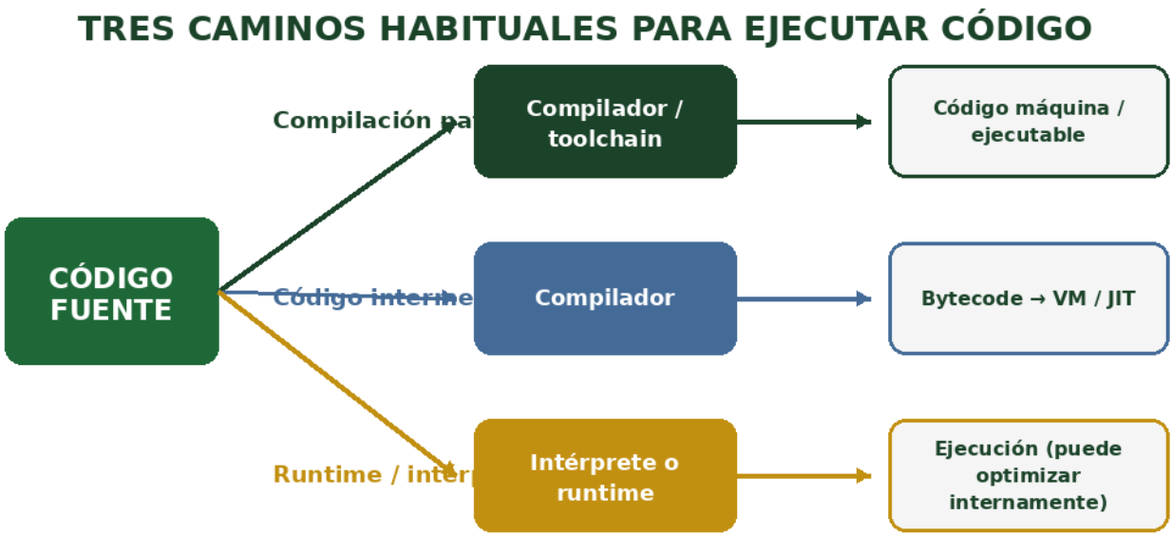


Figura 2.2 — Tres recorridos habituales del código fuente: compilación nativa, código intermedio/VM y ejecución mediante intérprete o runtime.

Ejemplo — compilar vs. interpretar, con una receta

Compilar es como traducir un libro de cocina entero al guaraní y entregar el libro terminado. Interpretar es como tener a un traductor al lado que te va diciendo cada paso en el momento de cocinar. El primero tarda al inicio pero después es rápido; el segundo arranca enseguida pero traduce cada vez.

Ventajas y desventajas

La estrategia de implementación influye en rendimiento, portabilidad, distribución y ciclo de pruebas, pero no existe una regla universal: el resultado depende del compilador o runtime, las optimizaciones y el programa concreto. Muchos entornos modernos combinan varias estrategias.

Concepto clave — el código necesita una implementación

Un programa de alto nivel no se ejecuta por sí mismo: necesita una implementación que lo traduzca o procese hasta instrucciones que la máquina pueda ejecutar. Esa implementación puede compilar, interpretar, usar una máquina virtual, aplicar JIT o combinar estrategias.

Ejemplo cotidiano — PSeInt en el aula

PSeInt, que usás para practicar algoritmos, trabaja como un intérprete: analiza el pseudocódigo, lo ejecuta en el momento (incluso paso a paso) y te muestra el resultado al instante, sin generar un ejecutable aparte. Por eso es ideal para aprender y corregir rápido.

El proceso de traducción por dentro

En una cadena típica de compilación nativa —como la que se usa con frecuencia al compilar C— el código fuente pasa por varias etapas, puede producir código objeto y luego enlazarse para obtener un ejecutable. Otros compiladores generan bytecode u otra representación intermedia, por lo que esta figura debe leerse como un ejemplo de cadena nativa, no como el único camino posible.

La figura muestra una cadena nativa simplificada y un desvío importante: si la herramienta detecta errores que impiden compilar, informa el problema y el código fuente debe corregirse antes de continuar.



Figura 2.3 — La cadena de compilación: del código fuente al ejecutable, con el desvío de los errores de sintaxis.

Concepto clave — portabilidad

Un mismo código fuente puede compilarse para computadoras distintas: lo que cambia es el ejecutable que se genera para cada una. Por eso se dice que el código fuente es portable y el ejecutable no: el fuente viaja entre máquinas, el ejecutable pertenece a una familia de máquinas.

Lenguajes híbridos: compilar y después interpretar

Muchos entornos actuales combinan estrategias: pueden compilar el fuente a un código intermedio y ejecutarlo en una máquina virtual, interpretar partes del programa o compilar en tiempo de ejecución (JIT). La implementación concreta determina el recorrido; por eso conviene hablar del entorno y no etiquetar rígidamente al lenguaje.

Estrategia	Cómo trabaja (modelo didáctico)	Ventaja posible
Compilación nativa	Genera código máquina antes de la ejecución	Buen rendimiento y distribución como binario
Interpretación/runtime	Procesa el programa durante la ejecución	Interactividad y ciclo rápido de pruebas
Código intermedio / VM	Compila a una representación intermedia que ejecuta una VM; puede incluir JIT	Portabilidad y optimización en ejecución

Ejemplo resuelto — el mismo cálculo en tres niveles

Veamos cómo se vería la orden «calcular el total de la venta V1» (2 chipas y 1 gaseosa) en cada nivel de lenguaje. No hace falta memorizar las formas: lo importante es notar cómo crece la cercanía con la persona a medida que sube el nivel.

1. Lenguaje de máquina: una tira de unos y ceros como 10110000 01100001… — ilegible para una persona, directa para el procesador.
2. Lenguaje ensamblador: instrucciones abreviadas como MOV, MUL y ADD que mueven y suman valores en la memoria — legibles con esfuerzo, una por cada instrucción de máquina.
3. Lenguaje de alto nivel (pseudocódigo): total ← 2 × 3000 + 1 × 8000 — una sola línea que cualquier estudiante entiende y que el traductor convierte en decenas de instrucciones de máquina.
4. En los tres niveles el resultado es el mismo: G. 14.000. Lo que cambia es quién carga con el esfuerzo de la traducción: la persona (abajo) o el traductor (arriba).

Aplicación profesional — el alto nivel es la norma

En el desarrollo profesional, la mayor parte de las aplicaciones se escribe en lenguajes de alto nivel, porque priorizan productividad, legibilidad y mantenimiento. Los lenguajes de bajo nivel quedan para necesidades específicas, como sistemas embebidos o tareas cercanas al hardware.

Código abierto y código cerrado

Que el código fuente esté disponible no basta para llamarlo «código abierto». Un programa propietario puede distribuirse sin su fuente o incluso mostrarlo con restricciones. Se considera software de código abierto cuando su licencia concede derechos para usar, estudiar, modificar y redistribuir el código bajo las condiciones establecidas. Por eso conviene distinguir «fuente disponible» de «licencia de código abierto».

Ambos modelos conviven en la industria: hay negocios sólidos construidos sobre cada uno. Para quien estudia programación, el código abierto tiene un valor extra: es una biblioteca gigante de programas reales para leer y aprender.

Ejemplo — la receta y el plato

El código cerrado es como comprar el plato hecho: se disfruta, pero la receta queda en secreto. El abierto entrega el plato Y la receta: se puede repetir, ajustar la sal o enseñarla. Ninguno es «mejor» en abstracto: depende de qué negocio y qué necesidad haya detrás.

Compilador o intérprete: la decisión práctica

La elección entre compilar e interpretar rara vez la hace el programador aislado: viene con el lenguaje elegido y con el destino del programa. Aun así, entender las consecuencias prácticas de cada camino permite anticipar cómo se trabajará todos los días.

Aspecto	Compilación previa (ejemplo)	Entorno interpretado/interactivo (ejemplo)
Rendimiento	Puede beneficiarse de optimización previa	Depende del runtime; puede incluir optimización/JIT
Ciclo de prueba	Suele incluir un paso de compilación	Puede ofrecer respuesta inmediata en REPL o consola
Distribución	Puede entregarse un binario o paquete	Suele requerir runtime; también puede empaquetarse
Errores	Muchos se detectan antes de ejecutar	Otros pueden aparecer al alcanzar una ruta durante la ejecución
Idea clave	Es una estrategia de implementación	No define por sí sola la velocidad ni al lenguaje

Ejemplo resuelto — el ciclo de trabajo en cada camino

Simulemos una mañana de desarrollo del sistema del Copetín bajo cada modalidad, para sentir la diferencia en el cuerpo.

1. Ejemplo de flujo con compilación previa: se escribe el código → se compila → se corrigen los errores informados → se vuelve a compilar → se ejecuta y se prueba con la venta V1 → G. 14.000.
2. Ejemplo en un entorno interactivo: se prueba una expresión del subtotal (6.000: correcto), luego otra (8.000: correcto) y finalmente la suma (14.000: correcto). Esa respuesta rápida es propia de muchos REPL y entornos interactivos, aunque no todos los intérpretes trabajan literalmente línea por línea.

Balance: un entorno interactivo puede facilitar aprendizaje y prototipado, mientras una compilación optimizada puede favorecer ciertos escenarios de rendimiento. Son tendencias que deben comprobarse en la implementación y la carga de trabajo reales.

Concepto clave — traducir y ejecutar tienen costos

La traducción, optimización y ejecución consumen recursos en distintos momentos. No existe una regla simple de «pagar todo antes» o «pagar todo durante»: una implementación puede repartir el trabajo entre compilación previa, carga, runtime y JIT. Para decidir, se mide el programa real.

De paseo por el escalón intermedio

Vale la pena espiar una vez el aspecto del nivel ensamblador, aunque nunca lo escribas. Cobrar 2 chipas, que en alto nivel es una línea (total ← 2 × 3000), en ensamblador se vuelve una pequeña coreografía de órdenes elementales, cada una con su abreviatura: cargar un valor en un registro de la CPU, multiplicarlo, guardar el resultado en memoria.

1. CARGAR R1, 3000 — el precio de la chipa entra al registro R1 de la CPU.
2. MULT R1, 2 — el registro se multiplica por la cantidad: R1 vale ahora 6000.
3. GUARDAR total, R1 — el resultado viaja del registro a la memoria, a la variable total.
4. Tres órdenes elementales para lo que el alto nivel dice en una. Esa es la distancia que los traductores recorren por vos, millones de veces por segundo.

Concepto clave — más cerca de la máquina, más pasos

Cuanto más bajo el nivel, más se fragmenta cada idea en micropasos: lo que el pensamiento humano junta («cobrar dos chipas»), la máquina lo ejecuta como cargar, operar y guardar. Los lenguajes de alto nivel existen precisamente para devolverle a la persona el derecho de pensar en ideas y no en micropasos.

Actividades de aplicación

Ejercitá:

1. Ordená de más cercano a la máquina a más cercano a la persona: alto nivel, máquina, ensamblador.
2. Explicá en una oración qué hace un traductor.
3. Indicá una diferencia entre compilador e intérprete.
4. Asociá el modelo más típico: (a) realiza una traducción previa y puede generar un ejecutable; (b) ejecuta mediante un intérprete o runtime sin requerir necesariamente un ejecutable nativo independiente. Opciones: compilación previa / interpretación-runtime.
5. Mencioná una ventaja posible de la compilación previa y una de trabajar con un entorno interpretado o interactivo.

Resolvé (caso Copetín Karumbé):

6. El programador del Copetín escribió «subtotal = 3 * 6000» en alto nivel. Explicá por qué la CPU no puede ejecutar esa línea tal cual y qué hace falta.
7. Si el sistema del Copetín se va a usar todos los días y el rendimiento es un criterio importante, ¿qué estrategia de ejecución o implementación priorizarías? Fundamentá sin asumir que el nombre del lenguaje determina por sí solo el recorrido.

Pensá y decidí (justificá tu respuesta):

8. «El lenguaje de máquina necesita un traductor para ejecutarse.»
9. «Un intérprete traduce todo el programa antes de correrlo.»

Desafío:

10. Buscá dos lenguajes reales (por ejemplo C y Python) y averiguá cómo se ejecuta una implementación habitual de cada uno: compilación nativa, bytecode/VM, intérprete/runtime o una combinación. Anotá tu fuente.

Práctica 2 — Traductores y errores en acción

Nombre y Apellido: _____ Curso: _____ Fecha: ____ / ____ / ____

Competencia de la práctica

Reconocer niveles de lenguaje y observar en PSeInt cuándo y cómo aparecen los errores de sintaxis, de ejecución y de lógica.

Entorno de trabajo: PSeInt.

Lo que necesitás saber

Máquina → ensamblador → alto nivel: cuanto más alto el nivel, más necesita una herramienta que lo traduzca o lo procese.

PSeInt interpreta el pseudocódigo: lo analiza, avisa los errores de escritura y lo ejecuta en el momento, incluso paso a paso.

Tené a mano el archivo CobroParaLlevar.psc de la Práctica 1.

Actividad 1 — ¿En qué nivel está cada fragmento?

Objetivo: *Clasificar cinco fragmentos nuevos según su nivel e indicar qué herramienta necesita cada uno para ejecutarse.*

Modelo — Fragmentos

(a) 0100 1000 0110 1001

(b) ADD R2, R3

(c) print(precio * 2)

(d) Escribir "Total: ", total

(e) MOV R1, 5000

Pasos:

1. Armá una tabla con tres columnas: fragmento, nivel y herramienta que necesita.
2. Ubicá primero el fragmento que la CPU ejecuta sin ayuda.
3. Para los de alto nivel, anotá si los conocés de algún lenguaje.

Punto de control: Ningún fragmento quedó sin nivel y cada herramienta anotada corresponde al nivel que le asignaste.

Actividad 2 — Tres errores provocados a propósito

Objetivo: *Provocar en PSeInt un error de cada tipo y registrar cuándo aparece y quién lo detecta.*

Modelo — Tabla para completar

Tipo de error | Qué cambiaste | Cuándo apareció | Quién lo detectó

Pasos:

1. Sintaxis: abrí CobroParaLlevar.psc, borrá la línea FinSi y pulsá Ejecutar. Copiá el mensaje que muestra PSeInt y volvé a escribir FinSi.
2. Lógica: cambiá el cálculo para que sume el precio del mbeju en vez de multiplicarlo (cantMbeju + 5000). Ejecutá con 3, 2 y 50.000 y compará con tu papel.
3. Ejecución: agregá al final las líneas Definir ventasDelDia Como Entero, ventasDelDia <- 0 y Escribir total / ventasDelDia. Ejecutá y anotá qué pasa. Al intentar dividir por cero, el problema aparece durante la ejecución. Registrá el mensaje exacto mostrado por la versión de PSeInt instalada.
4. Usá el comando Ejecutar Paso a Paso (F5) en la versión correcta y observá cómo PSeInt avanza línea por línea.

Punto de control: Tu tabla tiene un error de cada tipo. En la fila del error de lógica, la columna «quién lo detectó» no puede decir «PSeInt».

Desafío final (para quienes terminan antes)

Si tu versión de PSeInt ofrece exportar el algoritmo a otros lenguajes (Archivo → Exportar), exportalo a C o a Python y comparalo con tu pseudocódigo: ¿qué líneas se parecen y cuáles cambiaron?

Clase 3 — Los cuatro paradigmas de programación

Capacidad	Establece diferencias entre los distintos lenguajes de programación existentes en el mercado.
Tema	Paradigmas imperativo, funcional, lógico y orientado a objetos: propiedades.
Indicadores de logro	• Describe la idea central de cada paradigma. • Asocia un lenguaje de ejemplo a cada paradigma. • Distingue el enfoque imperativo del orientado a objetos.

Qué es un paradigma

Un paradigma de programación es una forma de pensar y organizar la solución de un problema. No es un lenguaje: es un estilo. Un mismo problema puede resolverse con distintos paradigmas, y muchos lenguajes permiten más de uno.

Los cuatro paradigmas clásicos

El paradigma imperativo describe paso a paso lo que la computadora debe hacer: primero esto, después aquello (es el estilo de PSeInt). El funcional arma la solución combinando funciones, evitando cambiar datos. El lógico declara hechos y reglas, y deja que el programa deduzca las respuestas. El orientado a objetos organiza el programa en «objetos» que reúnen datos y las acciones que operan sobre ellos.

Paradigma	Idea central	Propiedad clave	Lenguaje ejemplo
Imperativo	Ordenar paso a paso	Secuencia de instrucciones	C, pseudocódigo de PSeInt
Funcional	Combinar funciones	Evita cambiar datos	Haskell
Lógico	Declarar hechos y reglas	El programa deduce	Prolog
Orientado a objetos	Objetos con datos y acciones	Encapsula y reutiliza	Java, Python

Ejemplo — el mismo problema, dos estilos

Para calcular el total de una venta: en estilo imperativo escribís «recorré cada producto, multiplicá cantidad por precio y andá sumando». En estilo orientado a objetos definís un objeto Venta que sabe calcular su propio total. Distinta manera de pensar, mismo resultado.

Por qué existen tantos

Cada paradigma brilla en cierto tipo de problema. El orientado a objetos ordena muy bien los sistemas grandes; el funcional ayuda cuando importa la confiabilidad; el lógico resuelve problemas de deducción. Conocerlos te da más herramientas para elegir.

Concepto clave — paradigma no es lo mismo que lenguaje

El paradigma es el estilo de pensar la solución; el lenguaje es la herramienta concreta. Python, por ejemplo, admite el estilo imperativo y el orientado a objetos.

El paradigma estructurado en detalle: secuencia, decisión y repetición

El paradigma estructurado sostiene que cualquier algoritmo puede construirse con solo tres estructuras de control: la secuencia (pasos uno tras otro), la decisión (elegir un camino según una condición) y la repetición (volver a ejecutar mientras se cumpla una condición). La figura muestra las tres.

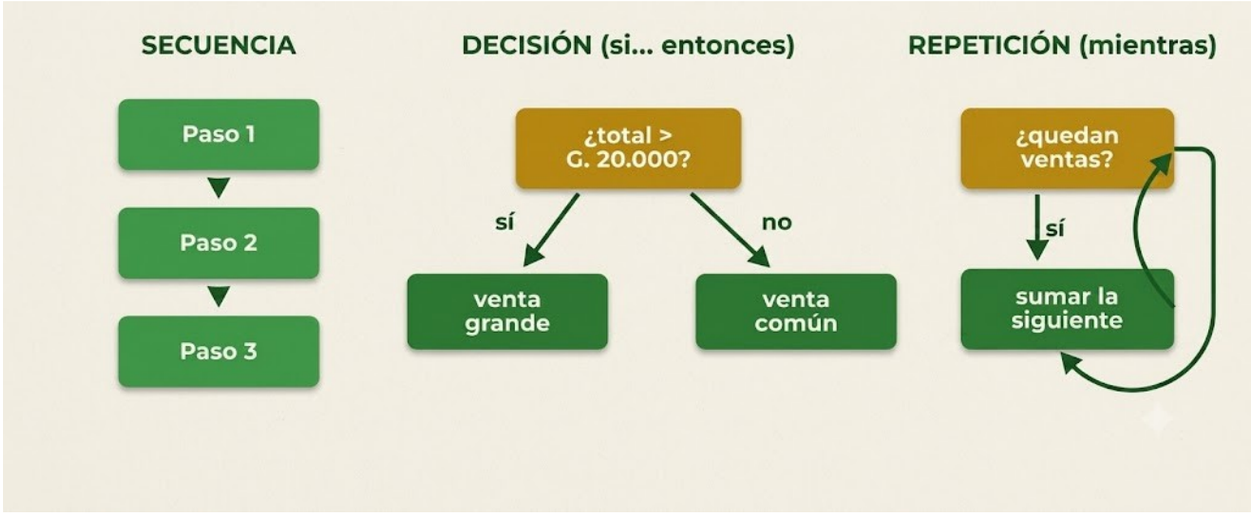


Figura 3.1 — Las tres estructuras del paradigma estructurado: secuencia, decisión y repetición.

Con estas tres piezas se arma cualquier programa, igual que con pocas piezas se arma cualquier construcción. En el Copetín: cobrar una venta es una secuencia; clasificarla como grande o común es una decisión; sumar las cinco ventas del día es una repetición.

El paradigma orientado a objetos en detalle: el producto como objeto

En la orientación a objetos, el programa se organiza en objetos que juntan datos y comportamiento. Un objeto Producto del Copetín tendría como datos su código, nombre, categoría y precio, y como comportamiento las acciones que sabe hacer: informar su precio o calcular un subtotal para una cantidad dada.

> **Ejemplo — el objeto Producto en acción**
> El objeto «empanada» guarda su precio (G. 6.000) y sabe responder a la orden calcularSubtotal(3): devuelve G. 18.000. El que usa el objeto no necesita saber cómo calcula por dentro; solo le pide el resultado. Esa separación entre el qué y el cómo es la gran idea del paradigma.

Ejemplo resuelto — la recaudación en dos paradigmas

Calculemos la recaudación total del Copetín (las cinco ventas: G. 14.000, 19.000, 34.000, 17.000 y 50.000) con dos enfoques distintos, para sentir la diferencia de estilo.

1. Enfoque imperativo/estructurado: total ← 0; repetir para cada venta: total ← total + importe de la venta; al terminar, mostrar total. La suma avanza paso a paso: 14.000 → 33.000 → 67.000 → 84.000 → 134.000.
2. Enfoque declarativo: se escribe QUÉ se quiere — «la suma de los importes de todas las ventas» — y el sistema decide cómo recorrerlas y sumarlas. Es el estilo de los lenguajes de consulta de bases de datos que vas a usar en la Unidad 4.
3. Verificación: ambos caminos llegan al mismo total, G. 134.000. El paradigma cambia la forma de pensar y escribir, no el resultado.

> **Concepto clave — los paradigmas se combinan**
> Los lenguajes modernos son multiparadigma: permiten escribir partes estructuradas, partes orientadas a objetos y partes declarativas en el mismo programa. Conocer los paradigmas no es elegir un bando: es tener más herramientas para cada parte del problema.

El paradigma funcional en una mirada

Dentro de la familia declarativa vive el paradigma funcional: el programa se arma combinando funciones puras, que reciben valores y devuelven resultados sin modificar nada afuera. Sumar la recaudación, en estilo funcional, es aplicar la función «sumar» a la lista de importes — sin variables que cambian ni pasos que se pisan.

Su gran promesa es la previsibilidad: una función pura, con las mismas entradas, devuelve SIEMPRE lo mismo. Esa cualidad lo volvió popular para procesar grandes volúmenes de datos, donde miles de operaciones corren en paralelo y cualquier «pisada» entre pasos sería catastrófica.

Eventos: el paradigma de las interfaces

Los programas con ventanas y botones siguen otro libreto: la programación dirigida por eventos. El programa no ejecuta una secuencia de principio a fin: espera. Cada acción del usuario — un clic, una tecla, abrir un formulario — dispara un evento, y el programa responde ejecutando el fragmento de código asociado. Cuando trabajes con formularios de Access en la Unidad 4, vas a estar parado exactamente en este paradigma.

Paradigma	Idea central	Dónde lo vas a cruzar
Estructurado	Secuencia, decisión y repetición	Los algoritmos de este libro
Orientado a objetos	Datos y comportamiento juntos	Los lenguajes de uso general
Lógico	Hechos y reglas para deducir	Sistemas expertos
Funcional	Funciones puras que se combinan	Procesamiento de datos
Dirigido por eventos	Responder a acciones del usuario	Formularios y botones de Access

Ejemplo resuelto — reconocer el paradigma en cinco situaciones

1. Situación 1: «total ← 0; para cada venta, total ← total + importe» — pasos y repetición: paradigma estructurado.

2. Situación 2: «al hacer clic en el botón Cobrar, calcular el total» — respuesta a una acción: dirigido por eventos.

3. Situación 3: «la suma de los importes de todas las ventas» pedida a la base de datos — se declara el qué: declarativo.

4. Situación 4: «el objeto Venta sabe calcular su propio total» — datos y comportamiento juntos: orientado a objetos.

5. Situación 5: «si un cliente compró las tres categorías, entonces es cliente integral» aplicado como regla — hechos y reglas: lógico. (¿Quién lo cumple en el Copetín? Solo Rosa: compró Panificados, Salados y Bebidas.)

Ejemplo — Rosa, la clienta integral

La regla de la situación 5 se verifica con la tabla cruzada de la Unidad 4: Rosa registra compras en las tres categorías (G. 6.000 + 18.000 + 24.000); Luis y Miguel, en dos; Ana, en una. Una regla lógica bien escrita se comprueba sola contra los datos.

Actividades de aplicación

Ejercitá:

1. Definí «paradigma de programación» en una oración.
2. Escribí la idea central de cada uno de los cuatro paradigmas.
3. Asociá cada lenguaje a su paradigma típico: Prolog, Java, C, Haskell.
4. Indicá qué paradigma describe la solución «paso a paso».

Resolvé (caso Copetín Karumbé):

5. El sistema del Copetín tendrá objetos como Producto, Venta y Cliente, cada uno con sus datos y sus acciones. ¿A qué paradigma corresponde ese diseño?
6. Reescribí en estilo imperativo, con un acumulador, el cálculo del total de un pedido de 2 empanadas (G. 6.000 c/u) y 3 cocidos (G. 4.000 c/u).
7. Indicá el paradigma de cada situación: (a) «al pulsar el botón Imprimir ticket, se imprime el comprobante»; (b) «si un producto se vendió más de cinco veces en la semana, entonces es producto estrella»; (c) «la suma de los importes de las ventas del día», pedida a la base de datos.

Pensá y decidí (justificá tu respuesta):

8. «Un lenguaje solo puede pertenecer a un paradigma.»
9. «El paradigma orientado a objetos organiza el programa en objetos que reúnen datos y acciones.»

Desafío:

10. Elegí un paradigma distinto del imperativo y explicá con tus palabras en qué tipo de problema conviene usarlo.

Práctica 3 — Paradigmas en la biblioteca y un acumulador en PSeInt

Nombre y Apellido: _____ Curso: _____ Fecha: ____ / ____ / ____

Competencia de la práctica

Reconocer paradigmas en fragmentos nuevos y programar en estilo estructurado un recorrido con acumulador.

Entorno de trabajo: PSeInt.

Lo que necesitás saber

Imperativo/estructurado: pasos, decisiones y repeticiones. Orientado a objetos: objetos con datos y acciones. Lógico: hechos y reglas. Funcional: funciones que se combinan. Eventos: el programa responde a acciones del usuario. Declarativo: se pide el qué, no el cómo.

Un acumulador es una variable que empieza en 0 y en cada vuelta suma un valor nuevo.

En PSeInt (perfil Flexible), la estructura Para i <- 1 Hasta 6 Hacer … FinPara repite seis veces lo que está adentro.

Actividad 1 — ¿Qué paradigma es?

Objetivo: *Clasificar seis fragmentos del sistema de una biblioteca escolar.*

Modelo — Fragmentos

(a) Para cada libro de la lista: si está prestado, sumar 1 al contador.

(b) libro.prestar(socio)

(c) socio_habilitado(X) si no_tiene_deudas(X).

(d) Al hacer clic en «Buscar», mostrar los resultados.

(e) suma(map(precio, compras))

(f) Contar los préstamos de marzo, pedido a la base de datos.

Pasos:

1. Leé cada fragmento y subrayá la palabra que delata el paradigma (para cada, objeto con punto, si…, al hacer clic, función aplicada a una lista, pedido a la base).
2. Escribí el paradigma y una frase que lo justifique.

Punto de control: Clasificaste los seis fragmentos, justificaste cada elección con una evidencia del enunciado y no asignaste dos categorías por simple intuición.

Actividad 2 — La recaudación de una semana, con acumulador

Objetivo: *Programar en PSeInt la suma de los seis importes de la semana del 16 al 18 de marzo y su promedio.*

Modelo — Importes de la semana (en guaraníes)

15.000 · 21.000 · 27.000 · 24.000 · 18.000 · 39.000

Pasos:

1. Definí total, importe e i como Entero y promedio como Real; inicializá total en 0.
2. Usá Para i <- 1 Hasta 6 Hacer: leé el importe y sumalo a total.
3. Después del ciclo, calculá promedio <- total / 6 y mostrá los dos valores.
4. Antes de ejecutar, completá en papel una traza con el valor de total al terminar cada vuelta.

Punto de control: El último valor de tu traza coincide con el total que muestra PSeInt, y el promedio multiplicado por 6 da ese mismo total.

Desafío final (para quienes terminan antes)

Agregá un contador que diga cuántas ventas superaron G. 20.000.

Clase 4 — Elegir el lenguaje adecuado

Capacidad	Reconoce la importancia de los diferentes tipos de lenguajes de programación existentes.
Tema	Criterios para seleccionar un lenguaje de programación.
Indicadores de logro	• Enuncia al menos tres criterios para elegir un lenguaje. • Justifica una elección de lenguaje para un caso dado. • Reconoce que no existe un lenguaje «mejor» para todo.

No hay un lenguaje «mejor»

La pregunta correcta no es «¿cuál es el mejor lenguaje?», sino «¿cuál es el más adecuado para este problema y este equipo?». La respuesta depende de varios factores, no de modas.

Criterios de selección

Al elegir un lenguaje se consideran, entre otros: el tipo de aplicación (web, escritorio, móvil), el rendimiento necesario, la facilidad de aprendizaje, la comunidad y documentación disponibles, y el conocimiento del equipo que va a desarrollar y mantener el sistema.

Criterio	Pregunta que responde
Tipo de aplicación	¿Es web, escritorio o móvil?
Rendimiento	¿Necesita ser muy rápido?
Facilidad	¿Qué tan rápido lo aprende el equipo?
Comunidad y soporte	¿Hay ayuda, librerías y ejemplos?
Equipo disponible	¿Qué ya saben los programadores?

Ejemplo — elegir el lenguaje del Copetín

Para el sistema del Copetín conviene un lenguaje con buena base de datos, fácil de mantener y con comunidad grande. Python encaja bien: es fácil de aprender, tiene mucho soporte y se conecta cómodamente a bases de datos. No es «el mejor del mundo», es el más adecuado para este caso.

Decidir con argumentos

Una buena decisión se explica: «elijo X porque el sistema es de gestión, el equipo ya conoce X y hay mucha documentación». Elegir por moda o por costumbre, sin argumentos, suele salir caro después.

Concepto clave — el contexto manda

El lenguaje adecuado es el que mejor equilibra el problema, el equipo y el mantenimiento. La misma tarea puede pedir lenguajes distintos en contextos distintos.

Los lenguajes más usados y para qué se usan

Conocer el «mapa» de los lenguajes ayuda a decidir con fundamento. Cada uno domina un territorio: la tabla resume los más difundidos y su uso típico en el mercado laboral.

Lenguaje	Territorio típico	Rasgo distintivo
Python	Análisis de datos, automatización, educación	Sintaxis simple, curva de aprendizaje suave
JavaScript	Páginas y aplicaciones web	Corre en el navegador de cualquier equipo
Java	Sistemas de empresas, aplicaciones Android	Compila a código intermedio portable
C#	Aplicaciones de escritorio Windows y videojuegos	Integración con el ecosistema Microsoft
SQL	Consultas a bases de datos	Declarativo: se pide el qué, no el cómo

Concepto clave — el lenguaje acompaña al problema

La tabla no es un ranking: es un mapa. La pregunta correcta nunca es «¿cuál es el mejor lenguaje?», sino «¿cuál es el mejor lenguaje PARA ESTE problema, con ESTE equipo y ESTE presupuesto?».

El costo total de una decisión

Elegir lenguaje es una decisión económica además de técnica. Un sistema no termina cuando se entrega: hay que mantenerlo por años. Por eso pesan criterios que no se ven en el código: cuánta gente sabe el lenguaje (para conseguir programadores), cuánta documentación y comunidad tiene (para resolver problemas) y qué herramientas gratuitas o pagas exige.

Una forma ordenada de decidir es la matriz de decisión ponderada: se eligen los criterios, se les asigna un peso según su importancia y se puntúa cada opción. La figura muestra el resultado final de una matriz: el puntaje ordena las opciones con argumentos.

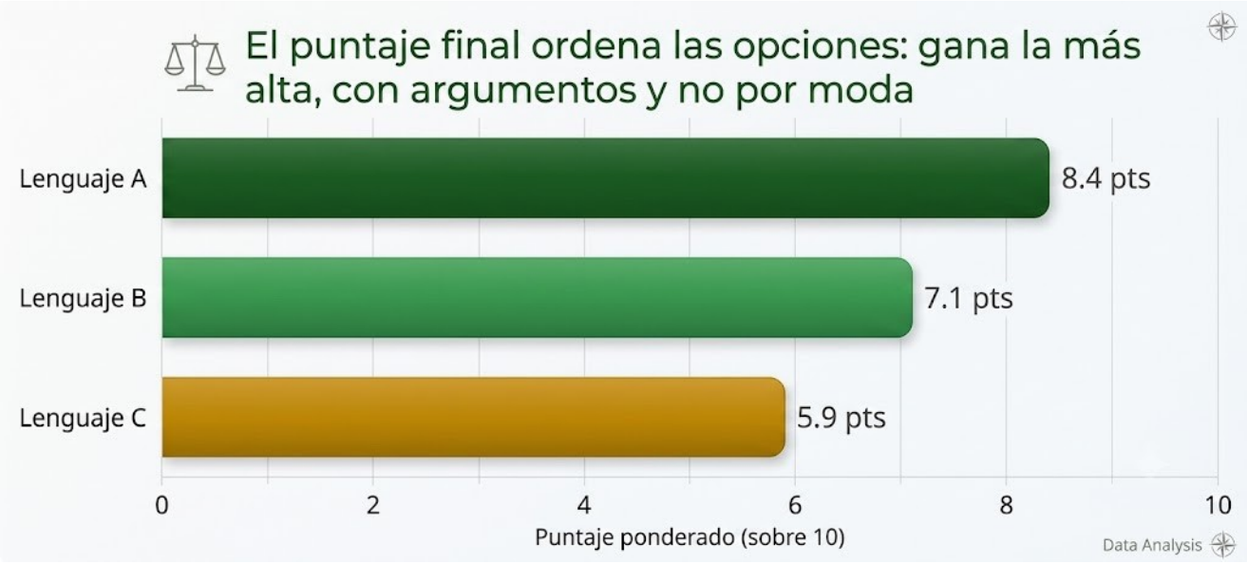


Figura 4.1 — El resultado de una matriz de decisión ponderada: cada lenguaje recibe un puntaje y gana el más alto.

Ejemplo resuelto — la matriz de decisión del Copetín

El Copetín Karumbé debe elegir el lenguaje para su sistema de gestión de ventas. Armemos la matriz paso a paso con tres criterios y sus pesos.

1. Definir criterios y pesos (deben sumar 1): facilidad de aprendizaje 0,4 — el equipo es nuevo; disponibilidad de programadores 0,4 — habrá que mantener el sistema; velocidad de ejecución 0,2 — las ventas del copetín no exigen gran potencia.
2. Puntuar cada opción de 1 a 10 en cada criterio, con honestidad y datos.
3. Multiplicar cada puntaje por el peso de su criterio y sumar. Opción A: 9×0,4 + 9×0,4 + 6×0,2 = 8,4. Opción B: 7×0,4 + 8×0,4 + 5×0,2 = 7,0. Opción C: 5×0,4 + 6×0,4 + 7,5×0,2 = 5,9.
4. Decidir y documentar: gana la opción A con 8,4 puntos. Lo importante no es solo el ganador: es que la decisión queda escrita y cualquiera puede revisarla o discutirla con argumentos.

Aplicación profesional — decidir con la realidad del equipo

Al elegir una tecnología también importa la disponibilidad de personas capacitadas para mantenerla. Un lenguaje con poca documentación, comunidad o especialistas disponibles puede aumentar el costo de mantenimiento aunque técnicamente sea adecuado.

Propósito general y propósito específico

Hay lenguajes nacidos para cualquier problema — los de propósito general, como los que dominan la tabla de la sección anterior — y lenguajes nacidos para UN territorio, los de propósito específico. El ejemplo mayor es el lenguaje de consultas de bases de datos: no sirve para escribir un videojuego, pero en su territorio (preguntar a los datos) es imbatible y universal.

En un sistema real conviven ambos: el de propósito general arma las pantallas y la lógica; el específico conversa con la base. El sistema del Copetín no será la excepción: lo que hagas en Access con consultas es, precisamente, un lenguaje de propósito específico en acción.

El ecosistema pesa tanto como el lenguaje

Un lenguaje no llega solo: trae su ecosistema. Las librerías son piezas ya programadas y probadas que se reutilizan (calcular, graficar, conectarse a una base) — programar sin librerías sería fabricar cada clavo antes de clavar. Los entornos de desarrollo asisten al escribir: marcan errores, autocompletan, ayudan a probar. Y la comunidad responde preguntas, documenta y mantiene todo lo anterior vivo.

Componente del ecosistema	Qué aporta	Pregunta para evaluarlo
Librerías	Soluciones listas y probadas	¿Existe librería para lo que mi problema necesita?
Entorno de desarrollo	Velocidad y menos errores al escribir	¿Hay entornos gratuitos y maduros?
Documentación	Aprendizaje y consulta	¿Está completa y actualizada?
Comunidad	Respuestas y continuidad	¿Se consiguen respuestas y programadores?

Ejemplo resuelto — el chequeo final antes de decidir

La matriz de decisión ordenó las opciones; antes de firmar, el equipo del Copetín pasa la opción ganadora por un chequeo de ecosistema.

1. ¿Hay librería para conectarse a la base de datos elegida? Sí — descartarlo hubiera sido motivo de reevaluación inmediata.
2. ¿Existe entorno de desarrollo gratuito? Sí, con marcado de errores y depuración incluidos.
3. ¿La documentación oficial está viva? Sí: versión estable reciente y manual completo.
4. ¿Se consiguen programadores del lenguaje en el mercado local? Sí — coherente con el peso 0,4 que la matriz le dio a ese criterio.
5. Veredicto: la opción A confirma su puntaje de 8,4. Decisión tomada, documentada y defendible.

Concepto clave — decidir es documentar

Una decisión técnica sin registro se vuelve a discutir cada seis meses. La matriz, el chequeo y sus conclusiones se guardan por escrito: cuando alguien pregunte «¿y por qué este lenguaje?», la respuesta tendrá fecha, criterios y números.

Actividades de aplicación

Ejercitá:

1. Enumerá tres criterios para elegir un lenguaje de programación.
2. Explicá por qué no existe un lenguaje «mejor» para todo.
3. Completá: «Elijo un lenguaje según el _____, el _____ y el _____».
4. Dado un juego para celular, ¿qué criterio pesa más: tipo de aplicación o color del logo? Justificá.

Resolvé (caso Copetín Karumbé):

5. Una ferretería de barrio quiere una tienda en línea donde sus clientes vean el stock desde el celular. Elegí un tipo de lenguaje razonable y justificá con dos criterios.
6. Un cliente quiere una app móvil. Indicá qué criterio descarta usar un lenguaje pensado solo para escritorio.
7. Recalculá la matriz de decisión del Copetín si el dueño cambia los pesos a facilidad 0,2; disponibilidad de programadores 0,3 y velocidad 0,5, con los mismos puntajes. ¿Cambia el ganador?

Pensá y decidí (justificá tu respuesta):

8. «Siempre conviene el lenguaje más nuevo o de moda.»
9. «El conocimiento del equipo es un criterio válido para elegir el lenguaje.»

Desafío:

10. Elegí una app que uses seguido y proponé, con argumentos, qué criterios habrán pesado al elegir su lenguaje de desarrollo.

Práctica 4 — Elegir la tecnología para la biblioteca escolar

Nombre y Apellido: _____ Curso: _____ Fecha: ____ / ____ / ____

Competencia de la práctica
Construir y documentar una matriz de decisión ponderada para elegir la tecnología de un sistema real, y analizar cuánto depende la decisión de los pesos elegidos.
Entorno de trabajo: papel y lápiz.

Lo que necesitás saber

La matriz ponderada multiplica el puntaje de cada opción por el peso de cada criterio y suma. Los pesos suman 1.

Una decisión técnica se documenta: criterios, pesos, puntajes, ganador y riesgos.

Caso: la biblioteca del colegio quiere que los estudiantes reserven libros desde el celular y que la bibliotecaria registre préstamos desde la computadora.

Actividad 1 — La matriz de decisión

Objetivo: *Calcular el puntaje de tres opciones con los criterios y los pesos que fijó la dirección.*

Modelo — Datos de la dirección
Criterios y pesos: funciona en el navegador del celular 0,5 · hay programadores disponibles 0,3 · costo de herramientas (más alto = más barato) 0,2
Opción X — sistema web (JavaScript + lenguaje de servidor): 9 · 8 · 9
Opción Y — programa de escritorio en C#: 3 · 8 · 7
Opción Z — app Android nativa en Kotlin: 7 · 6 · 8

Pasos:

1. Verificá que los pesos sumen 1.
2. Armá la tabla: opción, tres productos (puntaje × peso) y total.
3. Ordená las opciones de mayor a menor.

Punto de control: Cada total está entre 1 y 10 y coincide con el que obtiene un compañero que calcula por separado.

Actividad 2 — ¿Y si cambian las prioridades?

Objetivo: *Recalcular la matriz con otros pesos y comprobar si la decisión se sostiene.*

Modelo — Nuevos pesos (la cooperadora prioriza el costo)
navegador del celular 0,2 · programadores 0,3 · costo 0,5 (mismos puntajes)

Pasos:

1. Recalculá los tres totales.
2. Compará el orden con el de la Actividad 1.
3. Escribí una conclusión de dos oraciones: ¿la decisión es robusta o depende mucho de los pesos?

Punto de control: Tus dos tablas tienen las mismas opciones y los mismos puntajes; solo cambiaron los pesos.

Actividad 3 — Chequeo de ecosistema y ficha de decisión

Objetivo: *Pasar la opción ganadora por el chequeo de ecosistema y dejar la decisión documentada.*

Modelo — Ficha de decisión (completala)
Fecha · Problema · Criterios y pesos · Opciones y puntajes · Decisión · Chequeo de ecosistema (librerías, entorno, documentación, comunidad) · Riesgos

Pasos:

1. Respondé las cuatro preguntas del chequeo de ecosistema para la opción ganadora.
2. Anotá un riesgo de la decisión y cómo lo mitigarías.
3. Completá la ficha con letra clara: la va a leer otra persona.

Punto de control: La ficha tiene sus siete campos completos y el riesgo anotado no repite un criterio de la matriz.

Desafío final (para quienes terminan antes)

Proponé un criterio que la dirección olvidó (por ejemplo, la protección de los datos personales de los estudiantes), asignale un peso y explicá si cambiaría la decisión.

Evaluación de la Unidad 1

Lenguajes de programación

Nombre y Apellido: _____ Curso: _____ Fecha: ____ / ____ / ____

Parte A — Diagnóstico rápido (marcá la opción correcta):

1. Un lenguaje de programación se caracteriza por ser:
   a) preciso y sin ambigüedad
   b) ambiguo y flexible
   c) solo oral

2. El traductor que, en el modelo de compilación previa, procesa todo el programa y genera un ejecutable es el:
   a) intérprete
   b) ensamblador
   c) compilador

3. Organizar el programa en objetos que reúnen datos y acciones corresponde al paradigma:
   a) lógico
   b) funcional
   c) orientado a objetos

Parte B — Resolución y aplicación:

4. Ordená los niveles de lenguaje del más cercano a la máquina al más cercano a la persona.

5. Explicá con tus palabras la diferencia entre compilador e intérprete.

6. Asociá cada situación con su paradigma: (a) «si un socio no tiene deudas, entonces está habilitado»; (b) «venta.calcularTotal()»; (c) «al pulsar Guardar, se registra el pedido»; (d) «para cada venta: total ← total + importe».

7. Una escuela quiere una aplicación para que las familias consulten las notas desde el celular. Elegí un tipo de lenguaje o tecnología y justificá con dos criterios.

8. Escribí, como órdenes precisas y numeradas, el cálculo del total de una venta de 2 mbeju (G. 5.000 c/u) y 3 cocidos (G. 4.000 c/u).

UNIDAD 2 — Bases de datos y modelo relacional

En esta unidad vas a descubrir por qué los negocios pasan del cuaderno a la base de datos: qué es un SGBD, cómo se organizan las tablas con sus claves, qué protege la integridad referencial y cómo se mira una base por dentro.

Clase 5 — ¿Qué es una base de datos?

Capacidad	Reconoce los conceptos requeridos en la programación de base de datos utilizando un gerenciador de base de datos.
Tema	Base de datos: concepto y características. Del archivo suelto a la información organizada.
Indicadores de logro	• Define base de datos con sus propias palabras. • Menciona tres características de una base de datos bien organizada. • Distingue un conjunto de datos sueltos de una base de datos.

Del cuaderno de ventas a la base de datos

El Copetín Karumbé anota sus ventas en un cuaderno. Sirve, pero repetir el nombre y el precio del producto en cada renglón hace crecer los errores: un precio mal copiado, un nombre distinto para el mismo producto. Una base de datos guarda esa información una sola vez, organizada y lista para consultarse.

Una base de datos es una colección de datos organizados y relacionados entre sí, guardados de forma que se puedan consultar y actualizar con facilidad y sin repetir información innecesaria.

Características de una base de datos

Una base de datos bien hecha organiza los datos, los relaciona, evita la repetición innecesaria (redundancia), mantiene la coherencia y permite el acceso rápido a la información. No es solo «guardar»: es guardar con orden.

Característica	Qué significa
Organización	Los datos siguen una estructura (tablas, campos)
Relación	Los datos se vinculan entre sí (venta ↔ cliente)
No redundancia	Cada dato se guarda una sola vez
Coherencia	La información no se contradice
Accesibilidad	Se consulta y actualiza con facilidad

Ejemplo — el mismo dato repetido trae problemas

Si en el cuaderno del Copetín la chipa aparece a veces como «Chipa» a G. 3.000 y otras como «chipá» a G. 3.500, ya no se sabe cuál es el precio verdadero. Una base de datos guarda la chipa y su precio UNA vez: se corrige en un solo lugar.

Concepto clave — base de datos

Colección de datos organizados y relacionados, guardados para consultarlos y actualizarlos con facilidad y sin repetición innecesaria.

En Paraguay — bases de datos que usás sin darte cuenta

El padrón electoral, el sistema Marangatú de la DNIT y el registro de tu colegio son bases de datos. Cada vez que consultás tu boletín en línea, una base de datos responde.

Dato, información y conocimiento

No es lo mismo un dato que información. El dato es el valor crudo y suelto: «14000», «V1», «Chipa». La información es el dato interpretado en su contexto: «la venta V1 totalizó G. 14.000». Y el conocimiento es lo que se aprende juntando información: «los salados son la categoría que más recauda». La figura ordena los tres niveles.



Figura 5.1 — De abajo hacia arriba crece el valor: dato, información y conocimiento con ejemplos del Copetín.

Una base de datos guarda datos, pero su propósito es producir información y, con el tiempo, conocimiento para decidir mejor. Todo lo que vas a hacer en este libro — diseñar, normalizar, consultar — es el camino de los datos hacia las decisiones.

El archivo tradicional y sus límites

Antes de las bases de datos, cada programa guardaba sus propios archivos. Ese esquema, llamado sistema de archivos tradicional, funciona hasta que la información crece: los mismos datos terminan copiados en archivos distintos, cada programa los escribe a su manera y nadie garantiza que coincidan.

Aspecto	Archivos tradicionales	Base de datos
Ubicación de los datos	Repartidos por programa	Centralizados y compartidos
Redundancia	Alta: cada archivo repite datos	Mínima: cada dato, una vez
Consistencia	Difícil de garantizar	Controlada por el sistema
Acceso	Cada programa con su formato	Un lenguaje de consulta común
Seguridad	Por archivo, dispareja	Permisos centralizados

Ejemplo resuelto — del renglón del cuaderno al dato estructurado

El cuaderno del Copetín dice: «3 de marzo — Rosa llevó 2 chipas y una gaseosa, pagó 14.000». Transformemos ese renglón en datos estructurados, paso a paso.

1. Identificar las cosas que aparecen: una clienta (Rosa), una fecha (03/03/2026), dos productos (chipa, gaseosa), dos cantidades (2 y 1) y un total (G. 14.000).
2. Separar cada cosa en su lugar: la clienta pertenece a los clientes; los productos, con su precio, a los productos; la operación en sí, a las ventas.
3. Asignar códigos que identifiquen sin ambigüedad: Rosa será C01; la chipa, P01; la gaseosa, P06; la venta, V1.
4. Registrar la venta como datos: V1, del 03/03/2026, del cliente C01, con 2 unidades de P01 y 1 unidad de P06.
5. Verificar que nada se perdió: con los precios (P01 a G. 3.000 y P06 a G. 8.000) se reconstruye el total del renglón: 2 × 3.000 + 1 × 8.000 = G. 14.000. El renglón y los datos dicen exactamente lo mismo, pero los datos se pueden consultar, sumar y cruzar.

Concepto clave — estructurar es separar sin perder

Estructurar la información es separarla en sus piezas (quién, cuándo, qué, cuánto) sin perder la posibilidad de reconstruir el hecho completo. Si desde los datos no se puede volver a armar el renglón original, la estructura está mal diseñada.

Metadatos: los datos sobre los datos

Además de los datos, una base guarda metadatos: la descripción de su propia estructura. Que el precio es numérico, que el código de cliente tiene tres caracteres, que la fecha es una fecha — todo eso es metadato. Sin metadatos, «14000» sería una tira de dígitos sin estatus; con ellos, es un importe en guaraníes dentro del campo precio.

Dato del Copetín	Metadato que lo describe
P03	Código de producto: texto de 3 caracteres, clave principal
6.000	Precio: número entero positivo, en guaraníes
04/03/2026	Fecha de venta: fecha del calendario
C01	Cliente: texto de 3 caracteres, debe existir en Clientes

Concepto clave — la base se describe a sí misma

Los metadatos viven dentro de la propia base, en el llamado diccionario de datos. Por eso cualquier programa puede conectarse y saber qué tablas y campos existen sin que nadie se lo cuente: la base carga su propio manual.

Persistencia: los datos sobreviven al programa

Un programa vive mientras se ejecuta: al cerrarlo, sus variables desaparecen. Los datos de una base, en cambio, son persistentes: quedan guardados en el disco y esperan al próximo programa que los consulte. Esa diferencia — memoria volátil del programa contra almacenamiento persistente de la base — explica el reparto de tareas de todo sistema: el programa calcula, la base recuerda.

Ejemplo resuelto — qué recuerda y qué olvida el sistema del Copetín

1. Durante el cobro de la venta V3, el programa calcula los subtotales (18.000 y 16.000) en variables: existen solo en la memoria, mientras el programa corre.
2. Al confirmar la venta, el sistema registra V3 y sus renglones de DetalleVenta con cantidad y precio_unitario. El importe G. 34.000 puede reconstruirse sumando los subtotales; no necesita almacenarse como campo total de Ventas.
3. Se corta la luz y el programa se cierra de golpe. Las variables se pierden — no importa: eran cálculo intermedio.
4. Al volver la energía, se abre el sistema: la base sigue teniendo las ventas confirmadas, V3 incluida. La recaudación acumulada se recalcula desde los datos persistentes, no desde la memoria perdida.
5. Regla de diseño que se desprende: todo lo que deba sobrevivir a un cierre — ventas, clientes, precios — va a la base; lo demás puede vivir y morir en la memoria del programa.

Aplicación profesional — la memoria del negocio

Para un comercio, la persistencia es la memoria comercial y tributaria: qué se vendió, a quién, por cuánto. Perder esa memoria por no usar una base (o por no respaldarla) no es un problema informático: es un problema del negocio entero.

La calidad de los datos: cinco preguntas

Guardar datos no alcanza: tienen que ser buenos datos. La calidad de los datos se evalúa con cinco preguntas, y cada una tiene su patología típica en el mostrador.

Dimensión	Pregunta	Falla típica en un copetín
Exactitud	¿El dato refleja la realidad?	Se anotó G. 41.000 donde fue G. 14.000
Completitud	¿Está todo lo que debe estar?	Renglones de venta sin producto anotado
Actualidad	¿Está al día?	La lista de precios del año pasado
Consistencia	¿Dice lo mismo en todas partes?	«Rosa Duarte» acá y «R. Duarte» allá
Unicidad	¿Cada cosa está una sola vez?	La misma venta anotada dos veces

Un informe de recaudación construido sobre datos incompletos o duplicados no es neutro: induce decisiones equivocadas con total seriedad aparente. Por eso los sistemas serios validan en la entrada (tipos, listas, formatos): la calidad se defiende en la puerta, no se repara en el informe.

Actividades de aplicación

Ejercitá:

1. Definí «base de datos» con tus palabras.
2. Escribí tres características de una base de datos organizada.
3. Indicá un problema de anotar las ventas repitiendo el precio en cada renglón.
4. Dado un conjunto de fichas sueltas sin ninguna relación, decidí si es o no una base de datos y justificá.

Resolvé (caso Copetín Karumbé):

5. El cuaderno del Copetín repite «Gaseosa G. 8.000» en cada venta. Explicá qué característica de una base de datos evita ese problema.

6. Proponé qué datos del Copetín conviene guardar una sola vez (no en cada venta).

7. Clasificá como dato, información o conocimiento: (a) «17000»; (b) «la venta V4 totalizó G. 17.000»; (c) «los martes se venden más chipas que cualquier otro día».

8. En el cuaderno, la venta V2 aparece anotada dos veces y en otro renglón falta el producto. ¿Qué dimensiones de la calidad de los datos fallan?

Pensá y decidí (justificá tu respuesta):

9. «Una base de datos es simplemente un montón de datos guardados juntos.»

10. «La no redundancia significa guardar cada dato una sola vez.»

Desafío:

11. Mirá el cuaderno de notas de una materia y proponé cómo lo organizarías como base de datos: qué datos guardarías una sola vez y cuáles se repetirían.

Práctica 5 — Del cuaderno de préstamos a datos estructurados

Nombre y Apellido: _____ Curso: _____ Fecha: ____ / ____ / ____

Competencia de la práctica

Transformar los registros desordenados de un cuaderno en datos estructurados, y detectar redundancias, inconsistencias y problemas de calidad.

Entorno de trabajo: papel y lápiz.

Lo que necesitás saber

Estructurar es separar cada hecho en sus piezas (quién, qué, cuándo) sin perder la posibilidad de reconstruirlo.

La calidad de un dato se juzga por su exactitud, completitud, actualidad, consistencia y unicidad.

Este es el cuaderno de préstamos de la biblioteca del colegio, copiado tal como está escrito.

Actividad 1 — Identificar las piezas

Objetivo: *Separar socios, libros y préstamos, y asignar un código a cada uno.*

Modelo — Cuaderno de la biblioteca

1) 03/03 – Lucía Ortiz (3.º A) llevó «Hijo de hombre», de Augusto Roa Bastos.
2) 03/03 – Marcos Rojas (2.º B) llevó «El trueno entre las hojas», de Roa Bastos.
3) 04/03 – Lucia Ortiz (3A) llevó «La babosa», de Gabriel Casaccia.
4) 05/03 – Marcos Rojas (2.º B) devolvió «El trueno entre las hojas».
5) 05/03 – Ana Paula Vera (1.º C) llevó «Hijo de Hombre», de A. Roa Bastos.
6) 05/03 – Ana Paula Vera (1.º C) llevó «Hijo de hombre».

Pasos:

1. Hacé tres listas: socios, libros y préstamos.
2. Asigná códigos: S01, S02… a los socios y L01, L02… a los libros.
3. Reescribí cada préstamo como (código de socio, código de libro, fecha).

Punto de control: Ningún código se repite y ninguna persona ni libro aparece con dos códigos. Para cada renglón del cuaderno podés decir si generó un préstamo y por qué.

Actividad 2 — Detectar los problemas del cuaderno

Objetivo: *Señalar las redundancias, las inconsistencias y las fallas de calidad.*

Modelo — Tabla para completar

Renglón · Problema · Dimensión de calidad afectada · Cómo lo evita una base de datos

Pasos:

1. Compará los renglones 1 y 3, y los renglones 5 y 6.
2. Buscá un mismo dato escrito de dos maneras distintas.
3. Decidí qué tiene de especial el renglón 4.

Punto de control: Encontraste al menos cuatro problemas distintos y cada uno tiene su dimensión de calidad.

Actividad 3 — Dato, información y conocimiento

Objetivo: *Clasificar enunciados sobre la biblioteca y decidir qué se guarda una sola vez.*

Modelo — Enunciados

(a) «L01»
(b) «Lucía Ortiz llevó Hijo de hombre el 03/03»
(c) «Las novelas de Roa Bastos son las más pedidas por 3.er curso»
(d) «05/03»

Pasos:

1. Clasificá cada enunciado.
2. Hacé la lista de los datos que la biblioteca guardaría una sola vez.

Punto de control: Cada clasificación está justificada con la definición de dato, información o conocimiento.

Transferencia y revisión entre pares (30 a 40 minutos)

Objetivo: *aplicar lo mismo a un segundo caso, con datos diferentes, y mejorar el trabajo con la revisión de otro equipo.*

Modelo — Mini-caso: Libreta de fiados del almacén de la esquina

1) 10/03 – Doña Petrona Ayala lleva 2 kg de azúcar (G. 6.000 el kilo).
2) 10/03 – Petrona Ayala lleva 1 paquete de yerba (G. 12.000).
3) 11/03 – Don Ramón Sosa lleva 3 panificados (G. 2.000 c/u).
4) 11/03 – Doña Petrona pagó G. 12.000.
5) 12/03 – R. Sosa lleva 1 paquete de yerba (G. 12.000).

Pasos:

1. Separá clientes, productos y fiados con códigos, como en la Actividad 1, y anotá los problemas de calidad de la libreta con su dimensión.
2. Intercambiá tu resolución con la de otro equipo.
3. Revisá la resolución que recibiste y señalá por escrito al menos un error o una decisión sin justificar.
4. Con la revisión que te devolvieron, corregí tu versión.
5. Escribí en tres o cuatro renglones por qué tu versión final es correcta.

Punto de control: Tu versión final incorpora la corrección que recibiste (o explica por escrito por qué no correspondía) y tu justificación usa los conceptos de la clase.

Desafío final (para quienes terminan antes)

El renglón 4 registra una devolución. Proponé cómo guardarla en tu estructura sin crear un préstamo nuevo.

Clase 6 — Tipos y aplicaciones de bases de datos

Capacidad	Reconoce los conceptos requeridos en la programación de base de datos utilizando un gerenciador de base de datos.
Tema	Tipos de bases de datos. El modelo relacional. Aplicaciones.
Indicadores de logro	• Menciona tipos de bases de datos según su modelo. • Explica qué organiza el modelo relacional. • Asocia una base de datos a un campo de aplicación.

El modelo relacional: todo en tablas

Existen distintos modelos de bases de datos, pero el más usado es el modelo relacional: organiza la información en tablas, donde cada fila es un registro y cada columna un dato. Access, MySQL y PostgreSQL son SGBD relacionales que permiten crear y gestionar bases de datos de este modelo.

El nombre «relacional» viene de que las tablas se relacionan entre sí: la tabla de ventas se relaciona con la de clientes, la de detalle con la de productos. Así se evita repetir y se mantiene el orden.

La base de datos del Copetín

El Copetín Karumbé organizará su información en varias tablas relacionadas. Empecemos por la tabla de productos, donde cada producto se guarda una sola vez con su precio.

Código	Nombre	Categoría	Precio (G.)
P01	Chipa	Panificados	3.000
P02	Mbeju	Panificados	5.000
P03	Empanada	Salados	6.000
P04	Milanesa	Salados	15.000
P05	Cocido	Bebidas	4.000
P06	Gaseosa	Bebidas	8.000

Aplicaciones

Las bases de datos están detrás de casi todo sistema: gestión comercial, bancos, salud, educación, redes sociales. En cada caso guardan y relacionan la información que el sistema necesita.

Ejemplo — una tabla es una relación

En el modelo relacional, cada tabla representa un tipo de cosa: la tabla Productos guarda productos; la tabla Clientes, clientes. Las columnas son los datos (nombre, precio) y cada fila, un caso concreto (la chipa a G. 3.000).

Concepto clave — modelo relacional

Organiza la información en tablas relacionadas entre sí. Es el modelo más usado y el que trabajaremos en este libro.

En el laboratorio vas a trabajar con Microsoft Access como gestor de base de datos relacional de escritorio. Permite practicar tablas, claves, relaciones y consultas en un entorno integrado antes de trasladar los mismos conceptos a otros gestores.

Otros modelos: jerárquico, en red y no relacional

El modelo relacional no fue el primero ni es el único. El modelo jerárquico organiza los datos como un árbol: cada dato cuelga de un solo «padre». El modelo en red permite que un dato tenga varios padres. Y en los últimos años crecieron los modelos no relacionales (conocidos como NoSQL), que guardan documentos completos o pares de clave y valor, útiles cuando los datos no encajan bien en tablas. La figura compara los mismos datos organizados como árbol jerárquico y como tablas relacionadas.

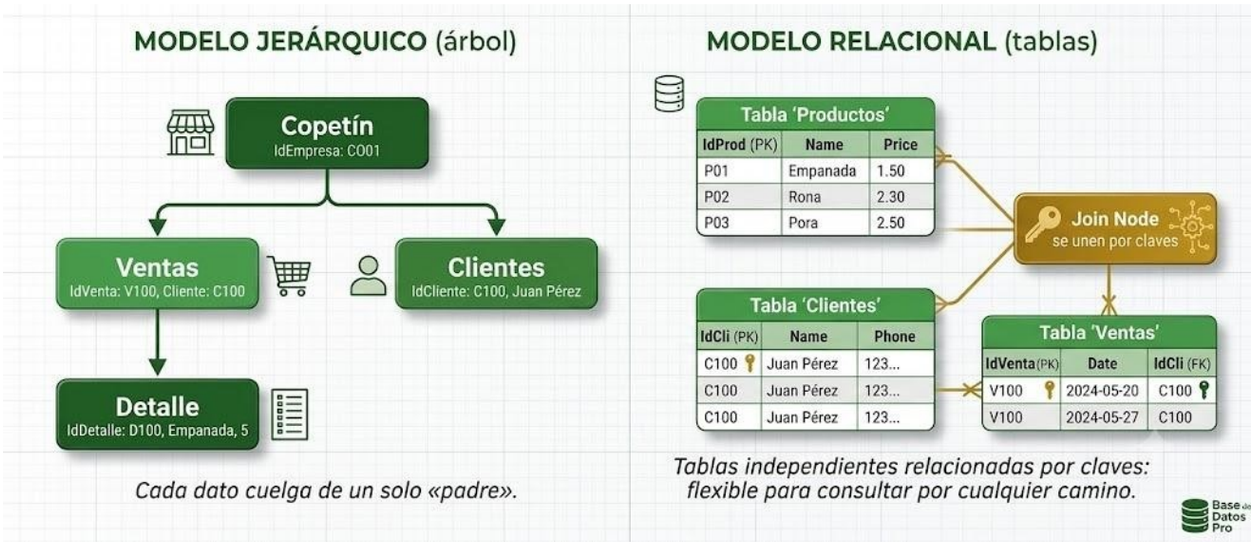


Figura 6.1 — Dos formas de organizar los mismos datos: el árbol del modelo jerárquico y las tablas relacionadas del modelo relacional.

Modelo	Organiza los datos como	Dónde brilla
Jerárquico	Un árbol con un padre por dato	Estructuras fijas tipo organigrama
En red	Un grafo con varios padres	Relaciones múltiples y complejas
Relacional	Tablas unidas por claves	La gestión general: es el estándar
No relacional (NoSQL)	Documentos o pares clave-valor	Volúmenes enormes y datos variables

Concepto clave — el relacional sigue siendo la base

Aunque existan alternativas, el modelo relacional domina la gestión de negocios: ventas, clientes, inventarios y facturación viven en tablas. Los demás modelos no lo reemplazan: lo complementan en problemas específicos.

Ejemplo resuelto — la tabla Clientes del Copetín

Ya conocés la tabla Productos. Construyamos ahora, con el mismo criterio, la tabla Clientes: cada cliente una sola vez, con un código propio.

1. Listar los clientes reales del Copetín: Rosa Duarte, Luis Benítez, Ana Gómez y Miguel Ortiz.
2. Asignar un código único a cada uno: C01, C02, C03 y C04, en orden de llegada.
3. Definir las columnas: código, nombre y algún dato de contacto útil (por ejemplo, el barrio de entrega si el copetín reparte).
4. Cargar los registros: C01 Rosa Duarte; C02 Luis Benítez; C03 Ana Gómez; C04 Miguel Ortiz.
5. Verificar la regla de oro: ningún cliente repetido, ningún código repetido, ningún código vacío. La tabla queda lista para que las ventas la señalen por código.

Código	Nombre	Observación
C01	Rosa Duarte	Clienta frecuente
C02	Luis Benítez	—
C03	Ana Gómez	—
C04	Miguel Ortiz	Compras grandes

Las bases de datos que usás sin darte cuenta

Cada vez que una aplicación te muestra tu saldo, tu historial o tus mensajes, hay una base de datos respondiendo por detrás. Los bancos consultan tablas de cuentas y movimientos; las billeteras electrónicas, tablas de usuarios y transferencias; los organismos públicos, registros de personas y contribuyentes. La diferencia con el Copetín es la escala, no la idea.

Aplicación profesional — registros organizados

Los organismos públicos y privados administran grandes registros de personas, vehículos, contribuyentes u operaciones mediante sistemas de información. Su implementación concreta puede variar, pero todos necesitan mecanismos de identificación, validación, relación y control de los datos.

El gestor y la aplicación: quién es quién

Conviene distinguir dos capas que suelen confundirse. El gestor (el SGBD) guarda y sirve los datos. La aplicación es el programa con el que trabaja el usuario: las pantallas, los botones, los informes. La aplicación le pide datos al gestor y le entrega los nuevos; el usuario final ve la aplicación y ni se entera del gestor que trabaja detrás.

Access es un caso particular y didáctico: reúne las dos capas en una sola herramienta — el motor que guarda los datos y el taller para construir formularios e informes. En sistemas grandes, las capas se separan: el gestor corre en un servidor y las aplicaciones, en las computadoras de cada usuario.

Bases locales y bases en servidor

Cuando la base vive en el equipo del usuario, es local: simple, sin red, ideal para un solo puesto de trabajo. Cuando vive en un servidor, muchos usuarios la comparten a la vez desde distintas máquinas: es el esquema de bancos, supermercados y sistemas web. La lógica que aprendés no cambia entre uno y otro: cambian la escala y la infraestructura.

Aspecto	Base local	Base en servidor
Dónde viven los datos	En el equipo del usuario	En un servidor compartido
Usuarios simultáneos	Uno o muy pocos	Decenas, miles o más
Requiere red	No	Sí
Administración	Mínima	Especializada
Ejemplo típico	El sistema de un copetín	El sistema de una cooperativa

Ejemplo resuelto — clasificar cuatro escenarios

1. El Copetín Karumbé con una sola computadora en caja: base local. Simple y suficiente.

2. Una farmacia con tres cajas que comparten el mismo inventario: base en servidor (local de la farmacia), porque tres puestos escriben a la vez.

3. Una billetera electrónica con miles de usuarios: base en servidores, con toda la ingeniería de concurrencia y respaldo que viste en el SGBD.

4. El Copetín dentro de cinco años, con local nuevo y reparto: probablemente migre a servidor — y la migración será llevadera justamente porque el diseño relacional es el mismo en ambos mundos.

Concepto clave — el diseño viaja, la infraestructura cambia

Las tablas, claves y relaciones que diseñás valen igual en una base local que en un servidor gigante. Por eso este libro invierte tanto en diseño: es la parte del conocimiento que no caduca cuando cambia la tecnología.

Qué hay dentro del archivo de una base

En Access, toda la base vive en un único archivo (extensión .accdb) que agrupa objetos de cuatro familias: las tablas (los datos y su estructura), las consultas (preguntas guardadas), los formularios (pantallas de carga) y los informes (salidas impresas). El panel de navegación, a la izquierda de la ventana, es el índice de esos objetos.

La consecuencia práctica es doble: respaldar la base es copiar UN archivo — trivial y sin excusas — y transportarla es llevar ese archivo a otra máquina con Access. La contracara: ese único archivo ES el negocio digitalizado; protegerlo (copias, contraseña) no es opcional.

Ejemplo — la base del Copetín, objeto por objeto

Terminado el libro, el archivo del Copetín contendrá: 4 tablas (Productos, Clientes, Ventas, DetalleVenta), media docena de consultas (ventas por fecha/cliente, consultas paramétricas, recaudación por categoría y tabla cruzada), y — si el proyecto de feria prospera — formularios de carga e informes imprimibles. Un negocio entero en un archivo.

Actividades de aplicación

Ejercitá:

1. Indicá qué organiza el modelo relacional.
2. Nombrá dos gestores de bases de datos relacionales.
3. En una tabla, ¿qué es una fila y qué es una columna?
4. Asociá: (a) sistema de un banco; (b) red social. ¿Usan base de datos? Justificá.

Resolvé (caso Copetín Karumbé):

5. Una cooperativa abre una sucursal y necesita que sus tres cajas consulten los mismos socios a la vez. ¿Base local o en servidor? Justificá con dos aspectos de la tabla de la clase.

6. Indicá qué otras dos tablas necesitará el Copetín además de Productos y por qué.

7. Indicá qué modelo conviene en cada caso: (a) el organigrama fijo de una empresa; (b) millones de mensajes de una app, con formatos muy variados; (c) ventas, clientes y productos de un comercio.

Pensá y decidí (justificá tu respuesta):

8. «En el modelo relacional cada columna es un registro.»

9. «Las tablas de una base de datos relacional pueden relacionarse entre sí.»

Desafío:

10. Elegí un sistema que uses (una app, el sistema del colegio) y proponé dos tablas que podría tener y qué columnas guardaría cada una.

Autoevaluación antes de la Práctica 6

Marcá con una X. Si algo queda en «Todavía no», repasá esa parte de la clase antes de empezar la Práctica 6.

Puedo…	Sí	Todavía no
Menciona tipos de bases de datos según su modelo.		
Explica qué organiza el modelo relacional.		
Asocia una base de datos a un campo de aplicación.		

Práctica 6 — El catálogo de la biblioteca como tabla relacional

Nombre y Apellido: _____ Curso: _____ Fecha: ____ / ____ / ____

Competencia de la práctica

Construir una tabla relacional nueva, reconocer sus partes y decidir el modelo y la infraestructura adecuados para distintos escenarios.

Entorno de trabajo: papel y lápiz.

Lo que necesitás saber

En el modelo relacional cada tabla guarda un tipo de cosa: una fila por ejemplar del tipo y una columna por dato.

Base local: un solo puesto, sin red. Base en servidor: muchos usuarios a la vez, con red.

Datos de cinco libros: Hijo de hombre, Augusto Roa Bastos, 1960, novela · El trueno entre las hojas, Augusto Roa Bastos, 1953, cuentos · La babosa, Gabriel Casaccia, 1952, novela · Yo el Supremo, Augusto Roa Bastos, 1974, novela · Don Quijote de la Mancha, Miguel de Cervantes, 1605, novela.

Actividad 1 — La tabla Libros

Objetivo: *Armar la tabla Libros con código, título, autor, año y género.*

Modelo — Encabezado de la tabla

código | título | autor | año | género

Pasos:

1. Asigná los códigos L01 a L05 en el orden de la lista.
2. Cargá una fila por libro.
3. Pintá una fila (registro) y una columna (campo) con dos colores distintos.
4. Marcá qué columna serviría como clave principal y por qué el título no sirve.

Punto de control: Tu tabla tiene una columna por dato y una fila por libro; ninguna celda tiene dos valores.

Actividad 2 — Modelo e infraestructura

Objetivo: *Elegir el modelo de datos y decidir entre base local o en servidor para cinco escenarios.*

Modelo — Escenarios

(a) La biblioteca escolar, con una sola computadora.

(b) La red de bibliotecas municipales de un departamento, que comparten el catálogo.

(c) Una app de lectura con millones de reseñas de formatos muy variados.

(d) El organigrama fijo del colegio.

(e) Un supermercado con ocho cajas.

Pasos:

1. Para cada escenario elegí el modelo (jerárquico, en red, relacional, NoSQL).
2. Decidí local o servidor, salvo en (d).
3. Justificá con una característica de la tabla de la clase.

Punto de control: Cada decisión está justificada con una característica (usuarios simultáneos, red, tipo de datos), no con «porque sí».

Actividad 3 — El archivo de la base, por dentro

Objetivo: *Planificar qué objetos tendría el archivo Biblioteca.accdb.*

Modelo — Familias de objetos

tablas · consultas · formularios · informes

Pasos:

1. Proponé dos objetos de cada familia para la biblioteca.
2. Clasificá los objetos según su función principal: datos y consulta (tablas y consultas) o interfaz y presentación (formularios e informes). Recordá que Access integra ambas funciones en una misma herramienta.

Punto de control: Tenés ocho objetos y cada uno tiene un nombre que dice qué hace.

Transferencia y revisión entre pares (30 a 40 minutos)

Objetivo: *aplicar lo mismo a un segundo caso, con datos diferentes, y mejorar el trabajo con la revisión de otro equipo.*

Modelo — Mini-caso: Inventario del laboratorio de informática

PC-01 · computadora · Lenovo · 2019 · 8 GB de memoria · sala 1

PC-02 · computadora · HP · 2021 · 8 GB de memoria · sala 1

PC-03 · computadora · Lenovo · 2019 · 4 GB de memoria · sala 2

IMP-01 · impresora · Epson · 2020 · (sin memoria informada) · sala 2

Escenarios: (a) solo la encargada del laboratorio usa el inventario, en su computadora; (b) los colegios del distrito comparten un único inventario.

Pasos:

1. Armá la tabla Equipos con un campo por dato, elegí la clave principal y decidí base local o en servidor para los escenarios (a) y (b).
2. Intercambiá tu resolución con la de otro equipo.
3. Revisá la resolución que recibiste y señalá por escrito al menos un error o una decisión sin justificar.
4. Con la revisión que te devolvieron, corregí tu versión.
5. Escribí en tres o cuatro renglones por qué tu versión final es correcta.

Punto de control: Tu versión final incorpora la corrección que recibiste (o explica por escrito por qué no correspondía) y tu justificación usa los conceptos de la clase.

Desafío final (para quienes terminan antes)

Proponé una segunda tabla, Socios, y explicá con qué tabla tendría que unirse para saber quién tiene cada libro.

Clase 7 — El SGBD y los problemas que resuelve

Capacidad	Utiliza conceptos de sistema de gestión de base de datos, propósitos, inconvenientes, redundancia e inconsistencia de datos en la creación de Bases de Datos.
Tema	El SGBD. Redundancia, inconsistencia, integridad, concurrencia y seguridad.
Indicadores de logro	• Explica qué es un SGBD. • Menciona tres problemas del manejo desordenado de datos. • Asocia un problema con la solución que aporta un SGBD.

Qué es un SGBD

Un Sistema de Gestión de Base de Datos (SGBD) es el software que crea, guarda, protege y consulta la base de datos. Access, MySQL y PostgreSQL son SGBD. La base de datos es la información; el SGBD es el programa que la maneja.

Los problemas que resuelve

Guardar los datos «a mano» o en planillas sueltas trae problemas que un SGBD evita: la redundancia (mismo dato repetido), la inconsistencia (copias que no coinciden), la dificultad para acceder a la información, y problemas de integridad, de acceso concurrente (varias personas a la vez) y de seguridad.

Problema	Qué es	Cómo ayuda el SGBD
Redundancia	Mismo dato repetido	Se guarda una sola vez
Inconsistencia	Copias que no coinciden	Un único dato válido
Acceso a datos	Cuesta encontrar la información	Consultas rápidas
Integridad	Datos inválidos o incompletos	Reglas que los controlan
Concurrencia	Dos personas editan a la vez	Controla el acceso simultáneo
Seguridad	Cualquiera ve o cambia todo	Permisos por usuario

Ejemplo — redundancia e inconsistencia en el Copetín

Si el precio de la gaseosa está copiado en 200 renglones del cuaderno y sube de G. 8.000 a G. 9.000, habría que corregir 200 veces (redundancia). Si se corrigen solo algunos, quedan precios distintos para el mismo producto (inconsistencia). El SGBD guarda el precio una vez y se corrige una vez.

Concepto clave — SGBD

Software que gestiona la base de datos: la crea, la guarda, la protege y permite consultarla. La base de datos son los datos; el SGBD es el programa.

Las funciones del SGBD: definir, manipular y controlar

El trabajo del SGBD se agrupa en tres funciones. Definir: crear las tablas, sus campos y sus reglas. Manipular: cargar, modificar, borrar y consultar los datos. Controlar: vigilar quién accede, mantener la integridad, atender a varios usuarios a la vez y recuperar los datos ante una falla.

Función	Qué abarca	Ejemplo en el Copetín
Definición	Crear tablas, campos y reglas	Crear la tabla Productos con su clave
Manipulación	Cargar, modificar, consultar	Registrar la venta V1; consultar la recaudación
Control	Permisos, integridad, respaldo	Impedir borrar a Rosa si tiene ventas

Seguridad y permisos: quién puede hacer qué

En una base real no todos pueden todo. El SGBD administra permisos por usuario: el dueño del Copetín puede crear tablas y ver todo; el cajero puede registrar ventas y consultar, pero no borrar clientes; una aplicación para clientes podría consultar el menú y nada más. La figura muestra al SGBD como único portero de los datos.



Figura 7.1 — El SGBD como intermediario: cada usuario llega con sus permisos y nadie toca los datos directamente.

Concepto clave — un solo portero

La regla de seguridad más importante es arquitectónica: los datos tienen UNA sola puerta, el SGBD. Si cada programa pudiera abrir los archivos por su cuenta, los permisos no servirían de nada.

Ejemplo resuelto — un día del SGBD en el Copetín

Sigamos al SGBD durante una mañana de trabajo para ver sus funciones en acción, una por una.

8:00 — El dueño consulta la recaudación de ayer. El SGBD verifica su permiso de lectura y responde.

9:15 — El cajero registra la venta V1 de Rosa. Manipulación: se insertan los datos y el SGBD confirma.

9:16 — Casi al mismo tiempo, desde otra terminal se registra la venta V2 de Luis. Control de concurrencia: el SGBD ordena las dos operaciones para que ninguna pise a la otra; ambas quedan guardadas completas.

11:30 — Alguien intenta registrar una venta de un cliente inexistente (C09). Control de integridad: el SGBD la rechaza y explica el motivo.

13:00 — Corte de energía. Al volver, el SGBD revisa su registro interno de operaciones y garantiza que las ventas confirmadas están intactas y ninguna quedó por la mitad.

Cierre — El respaldo automático copia la base completa. Si mañana se dañara el equipo, los datos se recuperan del respaldo.

Aplicación profesional — la copia de seguridad no es opcional

Un comercio necesita conservar durante años sus registros de ventas y comprobantes, por control interno y por sus obligaciones tributarias. Un respaldo periódico y probado (verificar que la copia realmente se puede restaurar) es la diferencia entre un susto y una pérdida irreparable.

Transacciones: todo o nada

Hay operaciones que solo tienen sentido completas. Registrar la venta V5 implica insertar la venta Y sus tres renglones de detalle: si la energía se corta a mitad de camino, quedaría una venta sin contenido o renglones sin venta. La transacción resuelve el dilema: agrupa las operaciones y el SGBD garantiza que se ejecutan TODAS o NINGUNA.

Propiedad de una transacción	Qué garantiza
Atomicidad	Todo o nada: sin mitades
Consistencia	La base pasa de un estado válido a otro válido
Aislamiento	Las transacciones simultáneas no se mezclan
Durabilidad	Lo confirmado no se pierde, ni con corte de luz

Ejemplo resuelto — la venta V5 como transacción

1. Inicio de la transacción: el cajero confirma la venta de Miguel.
2. Operación 1: insertar la venta V5 (05/03/2026, cliente C04).
3. Operaciones 2 a 4: insertar los renglones — 2 milanesas, 2 gaseosas, 1 cocido.

4. Confirmación: las cuatro operaciones terminaron bien; el SGBD sella la transacción y los G. 50.000 quedan registrados de forma durable.

5. Escenario alternativo: la operación 3 falla. El SGBD deshace TODO lo hecho — también la venta y el renglón ya insertados — y la base vuelve al estado previo, como si nada hubiera empezado. Se reintenta completa o no queda nada.

Concepto clave — sin transacciones no hay confianza

Los sistemas de pago, los bancos y la facturación viven de esta garantía: una transferencia debita en una cuenta Y acredita en otra, o no toca ninguna. La atomicidad no es un lujo técnico: es la base de la confianza en los sistemas.

El diccionario de datos

El SGBD mantiene el diccionario de datos: el catálogo interno con la descripción de cada tabla, campo, tipo, clave y permiso — los metadatos de la clase anterior, organizados y consultables. Es la fuente de la verdad sobre la estructura: cuando la aplicación pregunta «¿qué campos tiene Ventas?», la respuesta sale de ahí.

Cuando se audita un sistema (por control interno o requerimiento externo), el diccionario de datos es la primera parada: dice qué se guarda, con qué reglas y quién puede tocarlo. Un sistema con diccionario prolijo se audita en días; uno indocumentado, en meses.

Actividades de aplicación

Ejercitá:

1. Explicá la diferencia entre base de datos y SGBD.
2. Nombrá tres problemas que evita un SGBD.
3. Definí redundancia y definí inconsistencia.
4. Asociá el problema con su solución: (a) mismo dato repetido; (b) varios editan a la vez. Soluciones: control de concurrencia / no redundancia.

Resolvé (caso Copetín Karumbé):

5. La cantina del colegio anota los precios en una planilla para la caja y en otra para el depósito. El jugo figura a G. 5.000 en una y a G. 5.500 en la otra. Nombrá los dos problemas y explicá cómo los evita un SGBD.
6. Solo el dueño debería cambiar precios; los empleados solo registran ventas. ¿Qué problema resuelve esto y con qué herramienta del SGBD?
7. Al registrar la venta V2 (1 milanesa y 1 cocido) se corta la luz justo después de guardar el renglón de la milanesa. Según la atomicidad, ¿qué queda guardado al volver la energía?

Pensá y decidí (justificá tu respuesta):

8. «La base de datos y el SGBD son lo mismo.»
9. «La inconsistencia aparece cuando copias del mismo dato no coinciden.»

Desafío:

10. Pensá una situación de tu colegio donde el mismo dato esté repetido en varios lugares y explicá qué problema podría causar.

Práctica 7 — La cantina del colegio: problemas que resuelve un SGBD

Nombre y Apellido: _____ Curso: _____ Fecha: ____ / ____ / ____

Competencia de la práctica
Diagnosticar los problemas del manejo desordenado de datos y proponer la función del SGBD que los resuelve: permisos, transacciones y concurrencia.
Entorno de trabajo: papel y lápiz.

Lo que necesitás saber

Redundancia: un dato repetido. Inconsistencia: copias que no coinciden. El SGBD guarda una sola vez, controla permisos, agrupa operaciones en transacciones y ordena los accesos simultáneos.

La cantina del colegio lleva dos planillas: una en la caja y otra en el depósito.

Actividad 1 — Dos planillas, un problema

Objetivo: *Detectar redundancia, inconsistencia y dificultad de acceso comparando las planillas.*

Modelo — Planillas de la cantina
Caja: Jugo G. 5.000 · Sándwich G. 7.000 · Agua G. 3.000
Depósito: Jugo G. 5.500 · Sándwich G. 7.000 · Agua mineral G. 3.000 · Stock: Jugo 12, Sándwich 4, Agua mineral 20

Pasos:

1. Marcá los datos que están en las dos planillas.
2. Marcá los que no coinciden.
3. Escribí qué pregunta no se puede responder sin juntar las dos planillas a mano.

Punto de control: Cada problema que nombraste tiene un ejemplo concreto de las planillas y la consecuencia que tiene para la cantina.

Actividad 2 — Quién puede qué

Objetivo: *Armar la matriz de permisos de la cantina aplicando el mínimo privilegio.*

Modelo — Matriz para completar (Sí/No)
Roles: dueña · cajera · repositor
Operaciones: consultar precios · registrar venta · cambiar precio · cargar stock · borrar producto

Pasos:

1. Completá la matriz.
2. Justificá un «No» de cada rol.

Punto de control: Ninguna persona tiene un permiso que no necesita para su trabajo, y toda operación la puede hacer al menos una persona.

Actividad 3 — Todo o nada

Objetivo: *Simular una transacción con falla y relacionar cada propiedad ACID con un hecho.*

Modelo — Venta de la cantina
Insertar venta 501 · renglón 1: 2 jugos · renglón 2: 1 sándwich · renglón 3: 1 agua mineral · falla al guardar el renglón 3

Pasos:

1. Escribí qué queda guardado al volver el sistema y por qué.
2. Asociá: (a) «la venta confirmada no se pierde con un corte de luz»; (b) «la venta no queda por la mitad»; (c) «dos cajas que venden a la vez no se pisan»; (d) «el stock nunca queda negativo».

Punto de control: Tu respuesta sobre la falla y cada asociación están justificadas con el nombre de la propiedad y su definición.

Desafío final (para quienes terminan antes)

Quedan 1 sándwich en stock y dos cajeras lo venden en el mismo segundo. Explicá qué debe hacer el SGBD y qué ve cada cajera.

Clase 8 — Tablas, campos y registros. Clave principal y foránea

Capacidad	Analiza la composición de una base de datos.
Tema	Tablas, campos y registros. Clave principal, clave foránea y el término «clave secundaria».
Indicadores de logro	• Identifica campos y registros en una tabla. • Reconoce la clave principal de una tabla. • Explica para qué sirve una clave foránea.

Las piezas: tabla, campo y registro

Una tabla se compone de columnas y filas. Cada columna es un campo (un dato: código, nombre, precio) y cada fila es un registro (un caso concreto: la chipa). El nombre de la tabla dice qué tipo de cosa guarda.

La clave principal

La clave principal es el campo (o combinación de campos) que identifica a cada registro de forma única: no se repite y no queda vacío. En la tabla Productos, el código (P01, P02…) es la clave principal: dos productos no pueden tener el mismo código.

Código	Nombre	Categoría	Precio (G.)
P01	Chipa	Panificados	3.000
P02	Mbeju	Panificados	5.000
P03	Empanada	Salados	6.000
P04	Milanesa	Salados	15.000
P05	Cocido	Bebidas	4.000
P06	Gaseosa	Bebidas	8.000

La clave foránea

La clave foránea es un campo que apunta a la clave principal de otra tabla, y así las relaciona. En la tabla Ventas, el campo cliente guarda el código de un cliente (C01, C02…): es una clave foránea que apunta a la clave principal de la tabla Clientes.

En la figura, las dos tablas del Copetín se muestran como grillas: cada columna es un campo y cada fila un registro. La clave principal va en ámbar y la clave foránea, en azul, apunta con su flecha a la tabla Clientes.

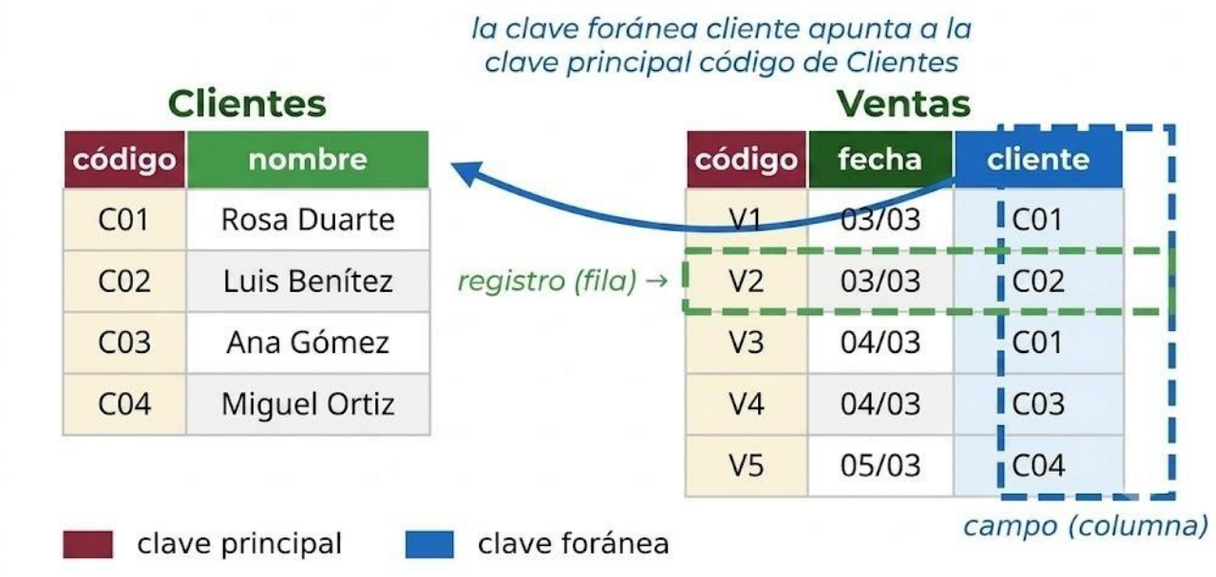


Figura 8.1 — Campo, registro y claves: el campo cliente de Ventas apunta al código de Clientes.

Venta	Fecha	Cliente (clave foránea)
V1	03/03/2026	C01
V2	03/03/2026	C02
V3	04/03/2026	C01
V4	04/03/2026	C03
V5	05/03/2026	C04

Ejemplo — la clave foránea conecta dos tablas

La venta V1 tiene cliente = C01. Para saber quién es, seguimos la clave foránea hasta la tabla Clientes, donde C01 es Rosa Duarte. Así se relacionan las tablas sin repetir el nombre del cliente en cada venta.

Concepto clave — clave principal y clave foránea

Clave principal: identifica de forma única cada registro de su tabla. Clave foránea: apunta a la clave principal de otra tabla y las relaciona.

Concepto clave — el término «clave secundaria»

El programa oficial emplea el término «clave secundaria» sin definirlo en este apartado. En este libro usaremos el término clave foránea para el campo que referencia la clave principal de otra tabla. En otros textos, «clave secundaria» también puede referirse a una clave alternativa o a un índice secundario.

Ante la duda, preguntate qué hace el campo: si apunta a otra tabla, es foránea; si identifica sin haber sido elegido como principal, es alternativa; si solo sirve para buscar más rápido, es un índice.

Elegir bien la clave principal: requisitos y errores comunes

Una buena clave principal cumple tres requisitos: es única (no se repite jamás), es estable (no cambia con el tiempo) y nunca queda vacía. Elegirla mal es uno de los errores de diseño más caros, porque todo el esquema se apoya en ella.

Candidata a clave de Clientes	¿Sirve?	Por qué
Nombre y apellido	No	Puede repetirse (dos Rosa Duarte) y puede cambiar
Teléfono	No	Cambia de dueño, puede quedar vacío
Código propio (C01, C02…)	Sí	Único, estable, nunca vacío: lo controla la base
Número de cédula	Depende	Único y estable, pero no siempre se quiere pedir

Concepto clave — la clave artificial

Cuando ningún dato natural cumple los tres requisitos, se inventa un código propio (C01, P01, V1). Se llama clave artificial y es la solución más usada en la práctica: la base la controla por completo y ningún cambio de la vida real la afecta.

Ejemplo resuelto — la tabla Ventas completa, clave por clave

Armemos la tabla Ventas del Copetín con sus cinco registros y analicemos cada clave. La figura muestra la tabla terminada con la clave principal y la foránea señaladas.

1. Clave principal: el código de venta (V1 a V5). Único, estable y nunca vacío: identifica cada operación.
2. Clave foránea: el campo cliente, que guarda el código del cliente que compró. No identifica la venta: la conecta con la tabla Clientes.
3. Cargar en Ventas sólo sus datos propios: V1 (03/03/2026, C01), V2 (03/03/2026, C02), V3 (04/03/2026, C01), V4 (04/03/2026, C03) y V5 (05/03/2026, C04). Los importes G. 14.000, 19.000, 34.000, 17.000 y 50.000 son valores calculados de referencia que se obtienen después desde DetalleVenta.
4. Verificar la clave principal: cinco códigos distintos, ninguno vacío. Correcta.
5. Verificar la clave foránea: C01, C02, C03 y C04 existen todos en Clientes. Ninguna venta apunta al vacío.
6. Control aritmético de referencia: al reconstruir cada venta desde sus renglones de detalle, los cinco importes suman G. 134.000. Ese total volverá a aparecer por otros caminos en la Unidad 4.

VENTAS: CLAVES DEL ESQUEMA E IMPORTE CALCULADO DE REFERENCIA

Venta (PK)	Fecha	Cliente (FK)	Importe calculado*
V1	03/03	C01	14.000
V2	03/03	C02	19.000
V3	04/03	C01	34.000
V4	04/03	C03	17.000
V5	05/03	C04	50.000

*** El importe se reconstruye desde DetalleVenta. No es un campo almacenado en Ventas.**

Figura 8.2 — La tabla Ventas con sus claves y los importes calculados de referencia; el importe se reconstruye desde DetalleVenta.

Claves compuestas: cuando un solo campo no alcanza

A veces ningún campo identifica solo a los registros y la clave se forma con dos o más campos juntos: es una clave compuesta. Ejemplo escolar: en un horario de clases, ni el curso ni el día ni la hora identifican solos una celda del horario; la combinación curso + día + hora, sí. En la próxima unidad te vas a cruzar con una clave compuesta muy importante para el Copetín.

En Paraguay — códigos que identifican

En la vida diaria existen identificadores como la cédula, el RUC o la matrícula de un vehículo. Sirven como analogía para comprender las claves, pero en una base de datos la elección de la clave principal depende de las reglas del sistema y de que el valor sea estable, único y no nulo.

Índices: encontrar sin recorrer todo

Buscar la venta V4 recorriendo las cinco filas del Copetín es instantáneo; buscar una venta entre dos millones, no. El índice resuelve el problema: es una estructura auxiliar, ordenada, que apunta a cada registro — como el índice de un libro, que evita hojear página por página. La clave principal se indexa automáticamente; otros campos muy buscados (la fecha, el cliente) pueden indexarse a pedido.

El índice tiene costo: ocupa espacio y se actualiza con cada inserción. Por eso no se indexa todo: se indexan los campos por los que realmente se busca. Es la primera decisión de rendimiento que toma un diseñador.

Ejemplo — el índice del cuaderno

Si el cuaderno del Copetín tuviera al final una lista ordenada de clientes con los números de página donde aparecen, encontrar «todas las compras de Rosa» sería directo. Esa lista final ES un índice: nadie reescribió el cuaderno, solo se agregó un camino rápido para llegar.

Tipos de datos: el molde de cada campo

Cada campo declara su tipo de dato: el molde que define qué valores acepta y qué operaciones permite. Elegirlo bien previene errores y habilita cálculos: un precio numérico se suma; un precio guardado como texto, no. Los tipos esenciales se repiten en todos los gestores, con nombres parecidos.

Tipo de dato	Guarda	Ejemplo en el Copetín
Texto	Palabras y códigos	nombre del producto, código C01
Número	Cantidades para calcular	cantidad vendida
Moneda	Importes con precisión monetaria	precio de catálogo, precio_unitario e importes calculados
Fecha y hora	Fechas y horas del calendario	fecha de la venta
Sí/No	Verdadero o falso	¿la venta fue a crédito?
Autonumeración	Un número que el gestor asigna solo	posible código interno de venta

Ejemplo resuelto — asignar el tipo a cada campo del Copetín

1. código (P01, C01, V1): Texto — aunque tenga números, no se calcula con él; se identifica.
2. nombre y categoría: Texto — palabras.
3. precio: Moneda — importe en guaraníes que se suma y se multiplica.
4. fecha: Fecha y hora — habilita filtrar por día y por rango.
5. cantidad: Número — participa del cálculo del subtotal.
6. Verificación cruzada: con estos tipos, el subtotal (cantidad × precio_unitario) es calculable y la consulta por fecha del 04/03/2026 devuelve V3 y V4 sin trucos. El molde correcto hace posible todo lo que viene en la Unidad 4.

Concepto clave — el código numérico que no es número

Regla práctica: si con el dato no se hace aritmética, no es Número aunque tenga dígitos. Un código, un teléfono o un número de documento son Texto: nadie suma dos teléfonos, y el tipo Texto conserva ceros iniciales que el numérico perdería.

Claves naturales y claves artificiales

Las claves se consiguen de dos maneras. La clave natural ya existe en la realidad: el número de RUC de una empresa, la cédula de una persona. La clave artificial se inventa para el sistema: los códigos P01 o V1 del Copetín, o el campo Autonumeración que Access numera solo. La natural trae significado; la artificial trae estabilidad garantizada — nadie la cambia porque no significa nada.

Criterio	Clave natural	Clave artificial
Origen	Ya existe en la realidad	La inventa el sistema
Ejemplo	Cédula, RUC	P01, V1, Autonumeración
Ventaja	Significativa, verificable afuera	Estable, corta, sin excepciones
Riesgo	Puede cambiar o tener excepciones	No dice nada por sí sola

Ejemplo — por qué el Copetín eligió códigos

¿Por qué P01 y no el nombre «Chipa» como clave? Porque el nombre puede corregirse («Chipa tradicional»), tener homónimos o variar de escritura — y una clave que cambia arrastra a todas las tablas que la referencian. El código artificial, mudo pero inmutable, es el ancla perfecta; el nombre queda como atributo, libre de cambiar sin romper nada.

Actividades de aplicación

Ejercitá:

1. Definí campo y registro.
2. ¿Qué dos condiciones cumple una clave principal?
3. Indicá cuál es la clave principal de la tabla Productos.
4. Explicá con tus palabras qué hace una clave foránea.
5. En la tabla Ventas, ¿qué campo es la clave foránea y a qué tabla apunta?

Resolvé (caso Copetín Karumbé):

6. Seguí la clave foránea de la venta V4 y decí a quién pertenece. Luego indicá cuántas ventas tiene esa persona en la tabla Ventas.
7. Proponé una clave principal para la tabla Clientes y justificá por qué sirve.
8. Asigná el tipo de dato (Texto, Número, Moneda, Fecha y hora o Sí/No) a: teléfono del cliente, cantidad vendida, precio_unitario, fecha de la venta y «¿pagó con tarjeta?».

Pensá y decidí (justificá tu respuesta):

9. «La clave principal puede repetirse entre registros.»
10. «La clave foránea relaciona una tabla con otra.»

Desafío:

11. Diseñá una tabla Empleados para el Copetín: proponé tres campos e indicá cuál sería la clave principal.

Práctica 8 — Claves de la biblioteca

Nombre y Apellido: _____ Curso: _____ Fecha: ____ / ____ / ____

Competencia de la práctica

Elegir claves principales con criterio, reconocer claves alternativas, foráneas y compuestas, y seguir las referencias entre tablas.

Entorno de trabajo: papel y lápiz.

Lo que necesitás saber

Clave principal: única, estable y nunca vacía. Clave alternativa: otro campo que también sería único. Clave foránea: apunta a la clave principal de otra tabla. Clave compuesta: dos o más campos juntos.

La biblioteca organizó sus datos en tres tablas.

Actividad 1 — Candidatas a clave

Objetivo: *Evaluar los candidatos a clave principal de Socios y Libros.*

Modelo — Tablas de la biblioteca

Socios(cod_socio, cédula, nombre, curso, email)

Libros(cod_libro, ISBN, título, autor)

Préstamos(nro_préstamo, cod_socio, cod_libro, fecha)

Pasos:

1. Para cada campo de Socios, respondé: ¿es único?, ¿es estable?, ¿puede quedar vacío?
2. Elegí la clave principal de cada tabla y nombrá las claves alternativas.

Punto de control: Cada tabla tiene exactamente una clave principal; las alternativas cumplen la unicidad.

Actividad 2 — Seguir las flechas

Objetivo: *Marcar las claves foráneas y recorrerlas con datos.*

Modelo — Datos

Socios: S01 Lucía Ortiz (3.º A) · S02 Marcos Rojas (2.º B) · S03 Ana Paula Vera (1.º C)

Libros: L01 Hijo de hombre · L02 El trueno entre las hojas · L03 La babosa

Préstamos: 101 (S01, L01, 03/03) · 102 (S02, L02, 03/03) · 103 (S01, L03, 04/03) · 104 (S03, L01, 05/03)

Pasos:

1. Dibujá las tres tablas y trazá una flecha desde cada clave foránea hacia la clave principal que referencia.
2. Respondé: ¿quiénes llevaron L01?, ¿qué libros llevó S01?, ¿a qué curso pertenece quien hizo el préstamo 102?

Punto de control: Cada flecha sale de una clave foránea y llega a la clave principal que referencia, y respondiste las tres preguntas recorriendo las flechas.

Actividad 3 — Una clave de dos campos

Objetivo: *Encontrar la clave de una tabla de horarios y asignar tipos de datos.*

Modelo — Horario (fragmento)

3.º A · lunes · 1.ª hora · Algorítmica

3.º A · lunes · 2.ª hora · Algorítmica

3.º B · lunes · 1.ª hora · Matemática

3.º A · martes · 1.ª hora · Inglés

Pasos:

1. Probá si curso, día u hora identifican solos una fila.
2. Encontrá la combinación mínima que identifica.
3. Asigná un tipo de dato a cada campo de Préstamos.

Punto de control: Tu clave compuesta no tiene campos de más: si quitás uno, deja de identificar.

Transferencia y revisión entre pares (30 a 40 minutos)

Objetivo: *aplicar lo mismo a un segundo caso, con datos diferentes, y mejorar el trabajo con la revisión de otro equipo.*

Modelo — Mini-caso: Taller mecánico del barrio

Clientes(cédula, nombre, teléfono)

Vehículos(nro_chasis, chapa, marca, cédula_dueño)

Reparaciones(nro_orden, nro_chasis, fecha, monto)

Dato del taller: la chapa de un vehículo puede cambiar si se lo vuelve a empadronar; el número de chasis no cambia.

Pasos:

1. Marcá en cada tabla la clave principal, las claves alternativas y las claves foráneas, y dibujá las flechas FK → PK.
2. Intercambiá tu resolución con la de otro equipo.
3. Revisá la resolución que recibiste y señalá por escrito al menos un error o una decisión sin justificar.
4. Con la revisión que te devolvieron, corregí tu versión.
5. Escribí en tres o cuatro renglones por qué tu versión final es correcta.

Punto de control: Tu versión final incorpora la corrección que recibiste (o explica por escrito por qué no correspondía) y tu justificación usa los conceptos de la clase.

Desafío final (para quienes terminan antes)

La biblioteca compra un segundo ejemplar de «Hijo de hombre». ¿Puede el ISBN seguir siendo clave? Proponé una solución.

Clase 9 — Integridad referencial

Capacidad	Analiza la composición de una base de datos.
Tema	Integridad referencial: reglas que protegen la coherencia entre tablas.
Indicadores de logro	• Explica qué es la integridad referencial. • Reconoce una operación que la viola. • Justifica por qué protege la coherencia de los datos.

No se puede apuntar a lo que no existe

La integridad referencial es la regla que asegura que toda clave foránea apunte a un registro que realmente existe. Si la venta dice cliente = C09 pero C09 no está en la tabla Clientes, la relación está rota.

Las dos reglas básicas

De esa idea salen dos reglas: no se puede registrar una venta con un cliente que no existe, y no se puede borrar un cliente que todavía tiene ventas registradas (quedarían ventas «huérfanas»). El SGBD hace cumplir estas reglas automáticamente.

El esquema muestra al SGBD actuando como guardián: acepta la operación que respeta las relaciones y rechaza las dos que las romperían.



Figura 9.1 — La integridad referencial en acción: qué acepta y qué rechaza el SGBD.

Operación	¿La permite el SGBD?	Por qué
Venta con cliente C01 (existe)	Sí	La clave foránea apunta a un registro real
Venta con cliente C09 (no existe)	No	Violaría la integridad referencial
Borrar cliente C01 con ventas	No	Dejaría ventas huérfanas
Borrar cliente sin ventas	Sí	No rompe ninguna relación

Ejemplo — una venta huérfana

Si borráramos a Rosa (C01) sin más, las ventas V1 y V3 quedarían apuntando a un cliente que ya no existe: nadie sabría de quién fueron. La integridad referencial impide ese borrado mientras existan esas ventas.

Concepto clave — integridad referencial

Regla que garantiza que toda clave foránea apunte a un registro existente. Protege la coherencia entre las tablas relacionadas.

En Paraguay — integridad y facturación

En la facturación electrónica paraguaya, las reglas sobre identificación del receptor dependen del tipo de operación. En operaciones B2B y B2G la identificación del receptor es obligatoria; en operaciones B2C existen casos de consumidor final. Por eso una base de datos debe representar correctamente las reglas del negocio y las exigencias vigentes para cada tipo de comprobante. La integridad referencial protege aquellas relaciones que el modelo define como obligatorias.

Qué hacer ante el conflicto: restringir, propagar o anular

Cuando una orden amenaza la integridad referencial, el SGBD puede restringirla, propagar el cambio o anular la referencia, según la configuración. En Access, «Exigir integridad referencial» es la opción que impide dejar registros huérfanos. Si está activada y las dos opciones de cascada permanecen desmarcadas, Access restringe la actualización o eliminación que rompería la relación. «Actualizar en cascada los campos relacionados» propaga los cambios de la clave principal; «Eliminar en cascada los registros relacionados» borra también los registros dependientes. Anular la referencia existe en otros gestores y no se configura desde esa ventana.

Reacción	Qué hace	Cuándo conviene
Restringir (rechazar)	No permite la orden mientras existan registros que apuntan	Cuando se exige integridad referencial sin cascadas.
En cascada	Borra también los registros que apuntan	Solo si los datos dependientes no valen nada sin el principal
Anular referencia	Deja la clave foránea vacía	Cuando tiene sentido conservar el registro «sin dueño»

Concepto clave — la cascada es peligrosa

Borrar en cascada al cliente C01 borraría también sus ventas V1 y V3 — y con ellas, G. 48.000 de historia comercial. Por eso la reacción en cascada se usa con muchísimo cuidado: un borrado chico puede arrastrar medio negocio.

Ejemplo resuelto — tres intentos contra la integridad

Pongamos a prueba al SGBD del Copetín (configurado para restringir) con tres órdenes, y veamos cómo decide. La figura resume el razonamiento que aplica ante cada borrado.

Intento 1 — borrar el cliente C01 (Rosa). El SGBD busca ventas que apunten a C01: encuentra V1 y V3. Resultado: RECHAZADO. Primero habría que decidir qué hacer con esas ventas.

Intento 2 — registrar una venta del cliente C09. El SGBD busca C09 en Clientes: no existe. Resultado: RECHAZADO. No se puede apuntar a lo que no existe.

Intento 3 — borrar la venta V2 (de Luis). Si en este momento analizamos solamente la relación Clientes→Ventas y todavía no cargamos DetalleVenta, ninguna fila hija apunta a V2 y el borrado podría permitirse. Una vez creados los renglones de DetalleVenta, esos renglones sí referencian V2 y una relación configurada para restringir el borrado debe rechazar la operación hasta resolver sus dependencias.

Balance: la integridad referencial no molesta por molestar; cada rechazo evitó datos huérfanos o inventados.



Figura 9.2 — El razonamiento del SGBD ante la orden de borrar un cliente: verificar antes de ejecutar.

Las otras dos guardianas: integridad de entidad y de dominio

La integridad referencial tiene dos hermanas. La integridad de entidad exige que toda tabla tenga clave principal y que esa clave jamás quede vacía ni repetida: sin ella, habría registros indistinguibles. La integridad de dominio exige que cada valor pertenezca al dominio de su campo: precios positivos, fechas reales, categorías del catálogo. Las tres juntas forman el sistema inmunológico de la base.

Integridad	Qué protege	Violación típica
De entidad	La identidad de cada registro	Dos productos con el código P03
De dominio	La validez de cada valor	Un precio de −500 o una fecha 35/03
Referencial	Las conexiones entre tablas	Una venta del cliente inexistente C09

Actualizar en cascada: el caso amable

La cascada, tan peligrosa al borrar, tiene una versión amable al actualizar. Si el código de un cliente cambiara (C01 pasa a CL01, por ejemplo), la actualización en cascada corrige automáticamente todas las ventas que lo referencian: nada queda apuntando al código viejo. Como las claves bien elegidas son estables, el caso es raro — pero cuando ocurre, la cascada de actualización evita una corrección manual propensa a olvidos.

Concepto clave — borrar no es actualizar

Las dos cascadas no se parecen en riesgo: la de borrado DESTRUYE registros dependientes; la de actualización solo CORRIGE referencias. Por eso es común configurar actualización en cascada y borrado restringido: lo mejor de cada mundo.

Ejemplo resuelto — el chequeo integral de la base del Copetín

Pasemos a la base completa por las tres integridades, como haría una auditoría.

1. Entidad: Productos (P01–P06), Clientes (C01–C04) y Ventas (V1–V5) tienen claves únicas, sin vacíos ni repetidos.
2. Dominio: los seis precios son positivos; las categorías son solo Panificados, Salados y Bebidas; las fechas (03/03/2026 a 05/03/2026) son reales; las cantidades van de 1 a 4.
3. Referencial (Ventas → Clientes): C01, C02, C03 y C04 existen todos.
4. Referencial (DetalleVenta → Ventas y Productos): las once filas apuntan a ventas V1–V5 y productos P01–P06 existentes.
5. Dictamen: base íntegra en las tres dimensiones. Este chequeo, que acá se hace a ojo, el SGBD lo ejecuta automáticamente en cada operación, millones de veces por día si hace falta.

El valor nulo: el tercer estado

Entre «tiene valor» y «vale cero» existe un tercer estado: el nulo — la ausencia de dato. Que el teléfono de un cliente esté nulo no significa que su teléfono sea cero ni una cadena vacía: significa que no se conoce. Los gestores tratan el nulo con lógica propia: no participa de las sumas, no coincide con nada en las comparaciones comunes y se pregunta con un criterio especial (Es Nulo).

1. Caso: al registrar a Ana Gómez no se le pidió el teléfono. El campo queda nulo — no cero, no vacío: desconocido.
2. Efecto en consultas: el criterio «teléfono = 0» NO la encuentra; el criterio «Es Nulo» sí. Confundir ambos es un clásico de resultados incompletos.
3. Decisión de diseño: los campos esenciales (código, nombre, precio) se declaran obligatorios — el gestor rechaza el nulo; los accesorios (teléfono) lo admiten. La obligatoriedad se decide campo por campo, en el diseño.

Concepto clave — nulo no es cero

Un precio de G. 0 es un dato (¡gratis!); un precio nulo es la falta del dato. Sumar ventas ignorando la diferencia produce totales silenciosamente incompletos. Regla profesional: los campos que participan de cálculos, siempre obligatorios.

Actividades de aplicación

Ejercitá:

1. Definí integridad referencial.
2. Escribí las dos reglas básicas que impone.
3. Indicá si el SGBD permite registrar una venta con un cliente que no existe.

4. Explicá qué es una venta «huérfana».

Resolvé (caso Copetín Karumbé):

5. Se intenta registrar una venta del cliente C07, que no está en la tabla Clientes. ¿Qué hace el SGBD configurado para restringir y por qué?

6. El dueño quiere borrar a Ana Gómez (C03), que tiene la venta V4 (G. 17.000). Indicá qué pasaría con cada configuración: (a) restringir; (b) eliminar en cascada.

7. Clasificá cada violación como de entidad, de dominio o referencial: (a) dos clientes con el código C02; (b) una cantidad de −2; (c) un renglón de detalle con el producto P09, que no existe.

Pensá y decidí (justificá tu respuesta):

8. «La integridad referencial permite claves foráneas que apuntan a registros inexistentes.»

9. «No se puede borrar un cliente que aún tiene ventas registradas.»

Desafío:

10. Proponé una tercera situación (además de las dos reglas vistas) donde la integridad referencial proteja los datos del Copetín.

Práctica 9 — La integridad referencial pone orden en la biblioteca

Nombre y Apellido: _____ Curso: _____ Fecha: ____ / ____ / ____

Competencia de la práctica

Predecir la reacción del SGBD ante operaciones que afectan relaciones, con distintas configuraciones, y reconocer las tres integridades y el valor nulo.

Entorno de trabajo: papel y lápiz.

Lo que necesitás saber

Restringir rechaza la orden mientras haya registros que apuntan; eliminar en cascada borra también los dependientes; actualizar en cascada corrige las referencias.

Usá las tablas y los datos de la Práctica 8 (socios S01–S03, libros L01–L03, préstamos 101–104).

Actividad 1 — ¿Acepta o rechaza?

Objetivo: *Decidir la reacción del SGBD (configurado para restringir) ante seis operaciones.*

Modelo — Operaciones

(a) Registrar el préstamo 105 del socio S04.

(b) Borrar al socio S02.

(c) Borrar el libro L03.

(d) Agregar el libro L04 «Yo el Supremo».

(e) Borrar el préstamo 102.

(f) Cambiar el código S01 por S10, con «Actualizar en cascada» activado.

Pasos:

1. Para cada operación, buscá qué registros apuntan al afectado.
2. Escribí «acepta» o «rechaza» y nombrá el registro que provoca el rechazo.

Punto de control: Cada rechazo nombra al registro que lo provoca; cada aceptación explica por qué no rompe ninguna relación.

Actividad 2 — La cascada, con números

Objetivo: *Calcular qué se borra si la relación Socios–Préstamos se configura con «Eliminar en cascada».*

Modelo — Orden

Borrar al socio S01.

Pasos:

1. Listá los préstamos que se borrarían.
2. Contá cuántos préstamos quedan en la base.
3. Decidí si conviene esa configuración en una biblioteca y justificá.

Punto de control: La cantidad de préstamos que quedan, más los borrados, da los préstamos que había.

Actividad 3 — Tres integridades y un nulo

Objetivo: *Clasificar violaciones e interpretar el valor nulo.*

Modelo — Situaciones

(a) Dos libros con el código L02.

(b) Un préstamo con fecha 31/02/2026.

(c) Un préstamo del libro L09, que no existe.

(d) El préstamo 104 tiene la fecha de devolución vacía (nula).

Pasos:

1. Clasificá (a), (b) y (c) como de entidad, de dominio o referencial.
2. Explicá qué significa el nulo de (d) y cómo lo buscarías.

Punto de control: Cada clasificación nombra la regla que se viola y el dato que la viola.

Transferencia y revisión entre pares (30 a 40 minutos)

Objetivo: *aplicar lo mismo a un segundo caso, con datos diferentes, y mejorar el trabajo con la revisión de otro equipo.*

Modelo — Mini-caso: Docentes y materias del colegio

Docentes: D1 Benítez · D2 Ruiz

Materias: M1 Algorítmica (docente D1) · M2 Matemática (docente D2) · M3 Inglés (docente D1)

Operaciones: (a) borrar a D2, con la relación configurada para restringir; (b) cambiar el código D1 por D10, con «Actualizar en cascada» activado; (c) agregar la materia M4 con el docente D7; (d) borrar a D1, con «Eliminar en cascada» activado.

Pasos:

1. Para cada operación, decidí si el SGBD la acepta o la rechaza y qué registros quedan afectados.
2. Intercambiá tu resolución con la de otro equipo.
3. Revisá la resolución que recibiste y señalá por escrito al menos un error o una decisión sin justificar.
4. Con la revisión que te devolvieron, corregí tu versión.
5. Escribí en tres o cuatro renglones por qué tu versión final es correcta.

Punto de control: Tu versión final incorpora la corrección que recibiste (o explica por escrito por qué no correspondía) y tu justificación usa los conceptos de la clase.

Desafío final (para quienes terminan antes)

Proponé la configuración completa (exigir integridad, actualizar en cascada, eliminar en cascada) para la relación Socios–Préstamos y para Libros–Préstamos, y justificá.

Clase 10 — La base por dentro: abstracción, usuarios y lenguajes

Capacidades	• Identifica los diferentes niveles de abstracción de los datos. • Analiza los tipos de usuarios de base de datos. • Reconoce los diferentes lenguajes de manipulación de datos más utilizados.
Tema	Niveles de abstracción (físico, lógico, vistas). Tipos de usuario. DDL y DML.
Indicadores de logro	• Nombra los tres niveles de abstracción de datos. • Distingue tipos de usuario de una base de datos. • Diferencia un lenguaje de definición de uno de manipulación de datos.

Tres niveles de abstracción

Observá la figura de arriba hacia abajo: cada usuario mira la base a través de su vista; las vistas se arman sobre las tablas del nivel lógico, y las tablas viven guardadas en el archivo del nivel físico. Seguí la flecha punteada: un cambio abajo no obliga a cambiar lo de arriba.

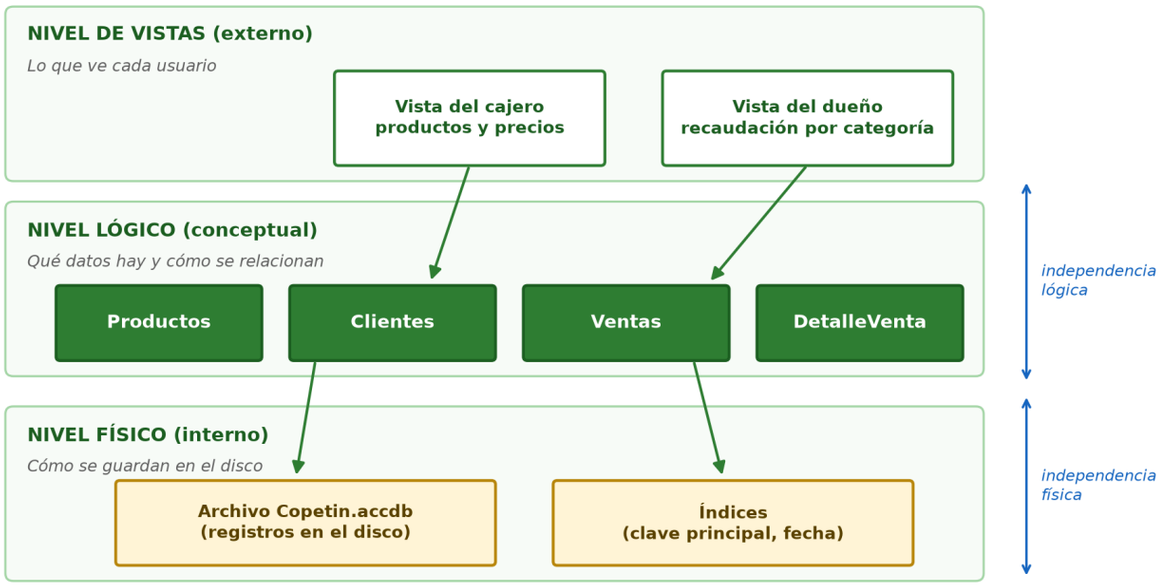


Figura 10.1 — Los tres niveles de abstracción aplicados a la base del Copetín Karumbé.

Una base de datos se puede mirar a distintos niveles. El nivel físico es cómo se guardan los datos en el disco (lo más técnico). El nivel lógico es qué datos hay y cómo se relacionan (las tablas y sus campos). El nivel de vistas es lo que ve cada usuario: una parte de la base, adaptada a lo que necesita.

Nivel	Qué describe	Quién lo usa
Físico	Cómo se guardan los datos en el disco	El sistema / técnicos
Lógico	Qué tablas y relaciones hay	Diseñadores y programadores
Vistas	La porción que ve cada usuario	Cada usuario final

Tipos de usuario

No todos usan la base igual: el programador de aplicaciones la conecta a un sistema; los usuarios sofisticados escriben sus propias consultas; los especializados desarrollan aplicaciones concretas; y los usuarios normales solo usan el sistema (registran ventas, consultan reportes) sin ver la base por dentro.

Lenguajes: definir y manipular

Para trabajar con la base se usan dos tipos de lenguaje. El de definición de datos (DDL) crea y modifica la estructura: tablas, campos, claves. El de manipulación de datos (DML) trabaja con el contenido: agregar, consultar, modificar y borrar registros.

Ejemplo — una vista para el Copetín

El empleado que atiende la caja no necesita ver toda la base: le alcanza una vista con los productos y sus precios para cobrar. El dueño, en cambio, ve además los reportes de recaudación. Misma base, distintas vistas.

Concepto clave — DDL y DML

DDL define la estructura (crear tablas y claves). DML manipula el contenido (insertar, consultar, actualizar y borrar registros).

Ejemplo cotidiano — DDL y DML en Access

En Access, diseñar una tabla y sus campos es trabajo de tipo DDL; cargar datos y hacer consultas es trabajo de tipo DML, aunque Access te lo facilite con menús y asistentes.

DDL y DML en la práctica: las órdenes típicas

Los dos lenguajes del SGBD tienen vocabularios distintos. El DDL (lenguaje de definición) habla de la estructura: crear, modificar o eliminar tablas. El DML (lenguaje de manipulación) habla de los datos: insertarlos, actualizarlos, borrarlos y — la orden más usada de todas — consultarlos.

Lenguaje	Órdenes típicas	Sobre qué actúan
DDL	Crear tabla · Modificar tabla · Eliminar tabla	La estructura (el molde)
DML	Insertar · Actualizar · Eliminar · Consultar	Los datos (el contenido)

Concepto clave — el molde y el contenido

El DDL toca el molde; el DML, el contenido. Eliminar la tabla Ventas (DDL) destruye el molde y todo lo que contenía; eliminar la venta V2 (DML) saca un registro del molde, que sigue intacto. Confundirlos es confundir demoler la casa con sacar un mueble.

El administrador de la base: tareas de todos los días

El administrador de la base de datos es quien responde por su salud. Sus tareas típicas: crear usuarios y asignar permisos, programar y verificar los respaldos, vigilar el rendimiento (que las consultas respondan rápido), aplicar los cambios de estructura que pida el negocio y planificar el crecimiento. En un emprendimiento chico ese rol lo cumple una sola persona; en una empresa grande, un equipo entero.

Ejemplo resuelto — clasificar las tareas del Copetín

Clasifiquemos seis tareas reales del sistema del Copetín según el lenguaje que usan y el tipo de usuario que las ejecuta.

1. Crear la tabla DetalleVenta → DDL (define estructura) · la ejecuta el administrador.
2. Registrar la venta V5 de Miguel → DML (inserta datos) · la ejecuta el usuario final (el cajero).
3. Agregar el campo teléfono a Clientes → DDL (modifica estructura) · administrador.
4. Consultar la recaudación del día → DML (consulta datos) · usuario final o dueño.
5. Corregir el nombre de un cliente mal tipeado → DML (actualiza datos) · usuario autorizado.
6. Dar permiso de consulta a un nuevo empleado → tarea de control del administrador, sobre los permisos y no sobre tablas ni datos.

La administración de bases de datos es un área profesional importante: organizaciones de distintos sectores necesitan datos íntegros, seguros, disponibles y respaldados. Estas mismas ideas se aplican tanto a sistemas pequeños como a infraestructuras de mayor escala.

Las vistas: ventanas a medida

El nivel externo de la arquitectura se materializa en las vistas: consultas guardadas que muestran a cada usuario SOLO la porción de datos que le corresponde, como si fuera una tabla propia. El cajero del Copetín podría trabajar sobre una vista con las ventas del día, sin ver la recaudación histórica; un socio, ver totales sin los datos personales de los clientes.

La vista no duplica datos: es una ventana con marco. Los datos siguen viviendo una sola vez en las tablas; la vista decide qué se ve y qué no. Seguridad y simplicidad en una sola herramienta.

Independencia de datos: cambiar sin romper

El premio mayor de los tres niveles es la independencia de datos: poder cambiar un nivel sin arrastrar a los demás. Si la base se reorganiza físicamente (nuevos índices, otro disco), las tablas y las vistas ni se enteran: independencia física. Si el esquema lógico suma un campo nuevo, las vistas existentes siguen funcionando: independencia lógica. Sin esta separación, cada mejora interna rompería todas las aplicaciones.

Cambio que ocurre	Nivel afectado	¿Quién lo nota?
Se agrega un índice para acelerar búsquedas	Interno (físico)	Nadie: solo mejora la velocidad
Se agrega el campo teléfono a Clientes	Conceptual (lógico)	Solo quien quiera usar el campo nuevo
Se crea una vista de ventas del día	Externo	Solo el usuario que la recibe

Ejemplo resuelto — un cambio en cada nivel del Copetín

1. Cambio físico: se indexa el campo fecha de Ventas para acelerar las consultas por día. Las tablas, vistas y aplicaciones quedan idénticas; solo cambia la velocidad. Independencia física en acción.

2. Cambio lógico: se agrega el campo barrio a Clientes para organizar repartos. La vista del cajero (que no usa el barrio) sigue intacta. Independencia lógica en acción.

3. Cambio externo: se crea la vista «resumen del día» para el dueño, con la recaudación por categoría. Nadie más se entera; las tablas no se tocaron.

4. Conclusión: tres cambios reales, cero roturas. Esa es la promesa cumplida de la arquitectura de tres niveles.

Concepto clave — separar para poder evolucionar

La independencia de datos es el mismo principio que verás en todo buen diseño de sistemas: separar las capas para que cada una pueda mejorar sin pedirle permiso a las demás. Las bases de datos lo aplicaron primero y mejor que nadie.

Actividades de aplicación

Ejercitá:

1. Nombrá los tres niveles de abstracción de datos.
2. Indicá qué describe el nivel lógico.
3. Diferenciá un usuario normal de un programador de aplicaciones.
4. Asociá: (a) crear una tabla; (b) consultar registros. Lenguajes: DDL / DML.

Resolvé (caso Copetín Karumbé):

5. El empleado de la caja del Copetín solo necesita ver productos y precios. ¿Qué nivel de abstracción describe eso y qué tipo de usuario es?
6. Clasificá como DDL o DML: (a) borrar la tabla Proveedores; (b) corregir el precio de la empanada; (c) crear el campo barrio en Clientes; (d) eliminar la venta V2.
7. Para acelerar las búsquedas se indexa el campo cliente de Ventas. ¿Qué nivel cambia y qué tipo de independencia permite que la vista del cajero siga funcionando sin tocarla?

Pensá y decidí (justificá tu respuesta):

8. «El nivel de vistas muestra toda la base de datos a todos los usuarios.»
9. «El DDL define la estructura y el DML manipula los datos.»

Desafío:

10. Proponé dos vistas distintas de la base del Copetín: una para el empleado y otra para el dueño, e indicá qué datos vería cada uno.

Práctica 10 — Lenguajes, usuarios y vistas de la biblioteca

Nombre y Apellido: _____ Curso: _____ Fecha: ____ / ____ / ____

Competencia de la práctica

Clasificar tareas en DDL y DML, reconocer tipos de usuario y diseñar vistas respetando la independencia de datos.

Entorno de trabajo: papel y lápiz.

Lo que necesitás saber

DDL define la estructura; DML trabaja con el contenido. Cada usuario ve la base a través de su vista; la independencia permite cambiar un nivel sin romper los otros.

Trabajamos sobre la base de la biblioteca de las prácticas anteriores.

Actividad 1 — ¿DDL o DML?

Objetivo: *Clasificar ocho tareas.*

Modelo — Tareas

(a) Crear la tabla Socios.
(b) Agregar el campo email a Socios.
(c) Registrar el préstamo 105.
(d) Corregir el título mal escrito de L03.
(e) Borrar la tabla Temporal.
(f) Consultar los libros prestados hoy.
(g) Eliminar al socio S03, que no tiene préstamos.
(h) Crear un índice sobre el título de Libros.

Pasos:

1. Marcá DDL o DML.
2. Para (e) y (g), explicá la diferencia entre borrar el molde y sacar un registro.

Punto de control: Cada tarea que modifica la estructura (tablas, campos, índices) quedó como DDL.

Actividad 2 — ¿Qué tipo de usuario es?

Objetivo: *Clasificar a cinco personas que usan la base.*

Modelo — Personas

(a) La bibliotecaria, que registra préstamos con un formulario.
(b) El profesor de Informática, que escribe sus propias consultas para un informe.
(c) Una estudiante que programa una app que se conecta a la base.
(d) Un técnico que desarrolla un sistema especial de catalogación con imágenes de portadas.
(e) Quien crea los usuarios, asigna permisos y programa los respaldos.

Pasos:

1. Asigná a cada persona su tipo de usuario.
2. Indicá qué nivel de abstracción usa cada una.

Punto de control: Cada asignación está justificada con lo que hace la persona y con la forma en que accede a la base.

Actividad 3 — Dos vistas y tres cambios

Objetivo: *Diseñar dos vistas y predecir qué nivel afecta cada cambio.*

Modelo — Vistas pedidas y cambios

Vista para estudiantes: catálogo disponible.

Vista para la dirección: cantidad de préstamos por curso.

Cambios: (1) mover la base a un disco más rápido; (2) agregar el campo editorial a Libros; (3) crear la vista de la dirección.

Pasos:

1. Para cada vista, listá los campos que muestra y los que oculta.

2. Indicá el nivel afectado por cada cambio y si la otra vista se entera.

Punto de control: Ninguna vista muestra datos personales a quien no los necesita.

Transferencia y revisión entre pares (30 a 40 minutos)

Objetivo: *aplicar lo mismo a un segundo caso, con datos diferentes, y mejorar el trabajo con la revisión de otro equipo.*

Modelo — Mini-caso: La base de la cantina del colegio

(a) Agregar el campo stock_mínimo a la tabla Productos.
(b) Registrar que llegaron 20 jugos.
(c) Crear para la cajera una vista que muestre solo producto y precio.
(d) La dueña consulta las ventas del día desde un formulario.
(e) Mudar el archivo de la base a un disco nuevo.

Pasos:

1. Clasificá cada tarea como DDL o DML, indicá el nivel de abstracción que afecta y el tipo de usuario de la cajera y de la dueña.
2. Intercambiá tu resolución con la de otro equipo.
3. Revisá la resolución que recibiste y señalá por escrito al menos un error o una decisión sin justificar.
4. Con la revisión que te devolvieron, corregí tu versión.
5. Escribí en tres o cuatro renglones por qué tu versión final es correcta.

Punto de control: Tu versión final incorpora la corrección que recibiste (o explica por escrito por qué no correspondía) y tu justificación usa los conceptos de la clase.

Desafío final (para quienes terminan antes)

Escribí con tus palabras la orden DDL que crearía la tabla Socios (nombre de la tabla, campos, tipos y clave).

Evaluación de la Unidad 2

Bases de datos y modelo relacional

Nombre y Apellido: _____ Curso: _____ Fecha: ____ / ____ / ____

Parte A — Diagnóstico rápido (marcá la opción correcta):

1. El campo que identifica de forma única a cada registro es la:
   a) clave foránea
   b) vista
   c) clave principal

2. El software que gestiona la base de datos se llama:
   a) SGBD
   b) DDL
   c) registro

3. Que una clave foránea apunte siempre a un registro existente es:
   a) redundancia
   b) integridad referencial
   c) una vista

Parte B — Resolución y aplicación:

4. En la tabla Ventas, el campo cliente guarda C01, C02… Indicá qué tipo de clave es y a qué tabla apunta.

5. Explicá con un ejemplo la diferencia entre redundancia e inconsistencia.

6. La venta V2 tiene cliente C02. Seguí la clave foránea y decí quién es (Clientes: C01 Rosa Duarte, C02 Luis Benítez, C03 Ana Gómez, C04 Miguel Ortiz).

7. Se intenta borrar al cliente C04, que tiene la venta V5. ¿Qué hace el SGBD configurado para restringir? ¿Qué pasaría si la relación estuviera configurada para eliminar en cascada?

8. Clasificá como DDL o DML: (a) crear la tabla Empleados; (b) registrar una venta nueva; (c) eliminar el campo teléfono de Clientes; (d) consultar las ventas de Rosa.

Evaluación integradora de la 1.ª etapa

Unidades 1 y 2 · Cierre de la 1.ª etapa (junio)

Nombre y Apellido: _____ Curso: _____ Fecha: ____ / ____ / ____

Parte A — Diagnóstico rápido (marcá la opción correcta):

1. El paradigma en el que el programa espera y responde a clics y teclas del usuario es el:
   a) lógico
   b) dirigido por eventos
   c) funcional

2. En el modelo relacional, la información se organiza en:
   a) listas sueltas
   b) tablas relacionadas
   c) un solo archivo de texto

3. Guardar cada dato una sola vez evita la:
   a) redundancia
   b) clave principal
   c) vista

Parte B — Resolución y aplicación:

4. Explicá por qué un programa escrito en alto nivel necesita un traductor y nombrá dos estrategias de ejecución.

5. La secretaría de un club deportivo necesita un sistema de gestión de socios y cuotas. Elegí un lenguaje razonable, justificá con dos criterios e indicá su paradigma típico.

6. Diseñá la tabla Socios de ese club con tres registros inventados, indicá la clave principal y explicá por qué el nombre no sirve como clave.

7. La tabla Pagos del club usa nro_socio como clave foránea hacia Socios. Explicá qué es la integridad referencial y da un ejemplo de operación que la violaría.

8. Calculá el total de una venta de 2 milanesas (G. 15.000 c/u) y 1 cocido (G. 4.000), mostrando las órdenes paso a paso.

UNIDAD 3 — Diseño: Entidad-Relación y normalización

En esta unidad vas a diseñar antes de construir: modelar con entidades, atributos y relaciones, dibujar el diagrama Entidad-Relación, traducirlo a tablas y comprobar con la normalización que cada dato vive en su lugar.

Clase 11 — Modelos de datos: el plano antes de construir

Capacidad	Analiza los diferentes modelos de datos.
Tema	Modelos de datos. El diseño como plano previo a la base.
Indicadores de logro	• Explica qué es un modelo de datos. • Menciona al menos dos modelos de datos. • Justifica por qué conviene diseñar antes de crear las tablas.

Diseñar antes de crear

Antes de construir una casa se hace un plano. Con una base de datos pasa lo mismo: antes de crear las tablas conviene diseñar qué información se guardará y cómo se relaciona. Ese diseño se llama modelo de datos.

Un modelo de datos es una representación de la información y sus relaciones, hecha en papel o en un diagrama, independiente del programa que después la implemente.

Distintos modelos

Existen varios modelos de datos. El modelo entidad-relación describe la realidad con entidades (cosas) y relaciones entre ellas; es el que usaremos para diseñar. También están el orientado a objetos y los modelos lógicos basados en registros o en objetos, más cercanos a la implementación.

Modelo de datos	Idea central
Entidad-relación	Entidades (cosas) y relaciones entre ellas
Orientado a objetos	Objetos con datos y comportamiento
Lógico basado en registros	Datos organizados en registros/tablas
Lógico basado en objetos	Los datos se guardan como objetos con atributos y métodos (bases orientadas a objetos)

Ejemplo — el plano del Copetín

Antes de abrir Access, el Copetín piensa: «guardo productos, clientes y ventas; cada venta es de un cliente y tiene varios productos». Ese razonamiento, dibujado, es el modelo de datos: el plano de la base.

Concepto clave — modelo de datos

Representación de la información y sus relaciones, previa e independiente del programa que la implemente.

Las tres fases del diseño: conceptual, lógico y físico

Diseñar una base de datos es un camino de tres fases. En la fase conceptual se dibuja el DER: qué entidades hay, con qué atributos y cómo se relacionan — sin pensar todavía en ningún programa. En la fase lógica el DER se traduce a un esquema de tablas con sus claves. Y en la fase física ese esquema se crea en un gestor concreto, con tipos de datos reales. La figura muestra el recorrido.



Figura 11.1 — Las tres fases del diseño: del DER en papel a la base creada en el gestor.

Fase	Producto	Herramienta típica
Conceptual	Diagrama Entidad-Relación	Papel, pizarra o diagramador
Lógica	Esquema de tablas con claves	Papel o documento
Física	Base creada y funcionando	El gestor (Access, en este libro)

Concepto clave — pensar antes de crear

El orden de las fases no es capricho: corregir un error en el DER cuesta una goma de borrar; corregirlo con la base cargada de datos cuesta horas y riesgo de pérdidas. Cuanto antes se detecta un error de diseño, más barato sale.

Ejemplo resuelto — delimitar el mini-mundo del Copetín

Antes de modelar hay que decidir QUÉ parte de la realidad entra en el modelo. Esa porción se llama mini-mundo. Delimitemos el del sistema de ventas del Copetín, paso a paso.

1. Escuchar la necesidad: «quiero saber qué vendo, a quién y cuánto recaudo».
2. Listar candidatos a entrar: productos, precios, clientes, ventas, fechas, proveedores, recetas, empleados, horarios.
3. Aplicar el filtro de la necesidad: ¿los proveedores hacen falta para saber qué se vende y cuánto se recauda? No por ahora. ¿Las recetas? Tampoco. ¿Los empleados y horarios? No para este sistema.
4. Mini-mundo resultante: productos (con categoría y precio), clientes, ventas (con fecha) y el detalle de qué productos lleva cada venta.
5. Registrar lo que quedó afuera y por qué: si mañana el Copetín quiere controlar compras a proveedores, el modelo se ampliará — pero será otra decisión, documentada como esta.

Aplicación profesional — sistemas empaquetados y mini-mundos

Los sistemas comerciales que se venden listos (facturación, inventario, clientes) son mini-mundos empaquetados: alguien decidió qué entidades incluir para el comercio típico. Cuando un negocio tiene necesidades distintas, ese recorte deja de servir, y ahí empieza el trabajo del diseñador de bases de datos.

Los requisitos: escuchar antes de dibujar

El mini-mundo no se adivina: se releva. Las técnicas clásicas son tres. La entrevista con quienes usarán el sistema revela qué necesitan saber y decidir. El análisis de documentos (cuadernos, boletas, planillas existentes) revela qué datos ya circulan y con qué forma. Y la observación del trabajo real revela lo que nadie cuenta porque «siempre se hizo así».

Técnica	Qué revela	En el Copetín
Entrevista	Necesidades y decisiones	«Quiero saber cuánto recaudo y qué se vende más»
Análisis de documentos	Datos que ya existen y su forma	El cuaderno de ventas con sus renglones
Observación	Lo que se hace y no se dice	El cajero anota el nombre del cliente solo si es conocido

Concepto clave — el modelo responde preguntas

Un modelo de datos se diseña PARA responder preguntas concretas. La lista de preguntas del dueño («¿cuánto recaudé?, ¿quién me compra más?, ¿qué categoría rinde?») es la vara con la que se mide el DER: si una pregunta de la lista no se puede responder desde el modelo, el modelo está incompleto.

Ejemplo resuelto — la entrevista al dueño del Copetín

Reconstruyamos la entrevista de relevamiento, pregunta por pregunta, y su efecto en el modelo.

1. Pregunta 1: «¿Qué decisiones querés tomar con el sistema?» — Respuesta: qué producir más y a quién cuidar como cliente. Consecuencia: el modelo necesita ventas por producto y por cliente.

2. Pregunta 2: «¿Qué anotás hoy y qué no?» — Respuesta: producto, cantidad y total; el cliente, solo si es conocido. Consecuencia: la relación venta-cliente existe, y habrá que decidir cómo tratar al cliente ocasional.

3. Pregunta 3: «¿Los precios cambian seguido?» — Respuesta: una o dos veces al año. Consecuencia: El precio de catálogo actual vive en Productos; queda fuera del mini-mundo un historial independiente de cambios del catálogo. Cada renglón de venta sí conserva en precio_unitario el importe realmente cobrado en esa operación.

4. Pregunta 4: «¿Quién va a cargar los datos?» — Respuesta: el propio dueño, desde una computadora. Consecuencia: base local, pantallas simples.

Cierre: las respuestas se documentan y se convierten en la lista de preguntas que el modelo deberá responder — la vara de la caja anterior.

En un relevamiento real, quien ejecuta el trabajo cotidiano conoce detalles que el diseñador puede pasar por alto. Por eso conviene observar, preguntar y validar el modelo con las personas usuarias antes de convertirlo en una base de datos.

Actividades de aplicación

Ejercitá:

1. Definí «modelo de datos».
2. Nombrá dos modelos de datos.
3. Explicá por qué conviene diseñar antes de crear las tablas.
4. Compará: ¿qué es al plano de una casa lo que el modelo de datos es a la base?

Resolvé (caso Copetín Karumbé):

5. Escribí, en una frase, qué información guardará el Copetín y cómo se relaciona (productos, clientes, ventas).
6. Indicá qué modelo de datos usaremos para diseñar y por qué es el adecuado para empezar.
7. El dueño pide: «quiero saber cuántas chipas me quedan en el horno». ¿Entra en el mini-mundo del sistema de ventas tal como fue delimitado? ¿Qué habría que agregar?

Pensá y decidí (justificá tu respuesta):

8. «El modelo de datos depende del programa que se use para crear la base.»
9. «Diseñar antes de crear las tablas ayuda a evitar errores costosos.»

Desafío:

10. Elegí algo que conozcas (una biblioteca, un club) y escribí en una frase qué entidades y relaciones tendría su modelo de datos.

Práctica 11 — El mini-mundo del club deportivo

Nombre y Apellido: _____ Curso: _____ Fecha: ____ / ____ / ____

Competencia de la práctica

Relevar requisitos a partir de una entrevista, delimitar el mini-mundo y formular las preguntas que el modelo debe responder.

Entorno de trabajo: papel y lápiz.

Lo que necesitás saber

El mini-mundo es la porción de la realidad que entra en el modelo. Se delimita con la necesidad del cliente y se documenta lo que queda afuera.

Leé la entrevista con el presidente del Club Atlético Yvága, un club de barrio inventado para esta práctica.

Actividad 1 — Leer la entrevista y delimitar

Objetivo: *Decidir qué entra y qué queda afuera del mini-mundo.*

Modelo — Entrevista (resumen)

«Tenemos unos 200 socios y tres disciplinas: fútbol, vóley y básquet. Cada socio paga una cuota mensual y quiero saber quién está al día.»

«Un socio puede anotarse en varias disciplinas; cada disciplina tiene un solo entrenador, pero un entrenador puede tener más de una.»

«La cantina la maneja otra comisión, y el arreglo de la cancha lo vemos aparte.»

«Quiero saber cuántos socios hay por disciplina y quién debe cuotas.»

Pasos:

1. Subrayá en la entrevista los sustantivos que podrían ser entidades.
2. Hacé dos listas: «entra» y «queda afuera», con el motivo de cada decisión.

Punto de control: Todo lo que pusiste en «queda afuera» tiene un motivo escrito.

Actividad 2 — Las preguntas que el modelo debe responder

Objetivo: *Escribir la lista de preguntas del presidente y verificar el mini-mundo contra ella.*

Modelo — Formato

Pregunta del club → datos del mini-mundo que la responden

Pasos:

1. Escribí al menos cuatro preguntas que el sistema debe responder.
2. Al lado de cada una, anotá qué datos del mini-mundo usa.
3. Indicá qué modelo de datos vas a usar y en qué fase del diseño estás.

Punto de control: Cada pregunta se responde con algo de tu mini-mundo y nada de tu mini-mundo sobra.

Desafío final (para quienes terminan antes)

El club quiere sumar en el futuro la venta de la cantina. Escribí cómo documentarías hoy esa decisión para que nadie la olvide.

Clase 12 — Entidades y atributos

Capacidad	Aplica los conceptos fundamentales en la construcción de entidades.
Tema	Entidades, atributos y tipos de atributos.
Indicadores de logro	• Identifica entidades y sus atributos en un caso. • Clasifica atributos (simple/compuesto, univalorado/multivalorado, derivado). • Reconoce el atributo que sirve como clave.

Entidades y atributos

Una entidad es una cosa de la realidad sobre la que guardamos datos: un producto, un cliente, una venta. Los atributos son los datos de esa entidad: del cliente guardamos código, nombre y teléfono.

Tipos de atributos

Los atributos pueden ser simples (un solo valor: precio) o compuestos (se dividen en partes: un nombre completo en nombre y apellido). Pueden ser univalorados (un valor por registro: fecha de nacimiento) o multivalorados (varios: los teléfonos de un cliente). Y pueden ser derivados: se calculan a partir de otros (la edad se deriva de la fecha de nacimiento).

Tipo de atributo	Ejemplo
Simple	precio del producto
Compuesto	nombre completo (nombre + apellido)
Univalorado	código del producto
Multivalorado	varios teléfonos de un cliente
Derivado	edad (se calcula desde la fecha de nacimiento)

La figura muestra cómo se dibuja cada tipo de atributo en la notación clásica. Fijate en tres detalles: el identificador va subrayado, el multivaluado lleva doble elipse y el derivado, contorno punteado.

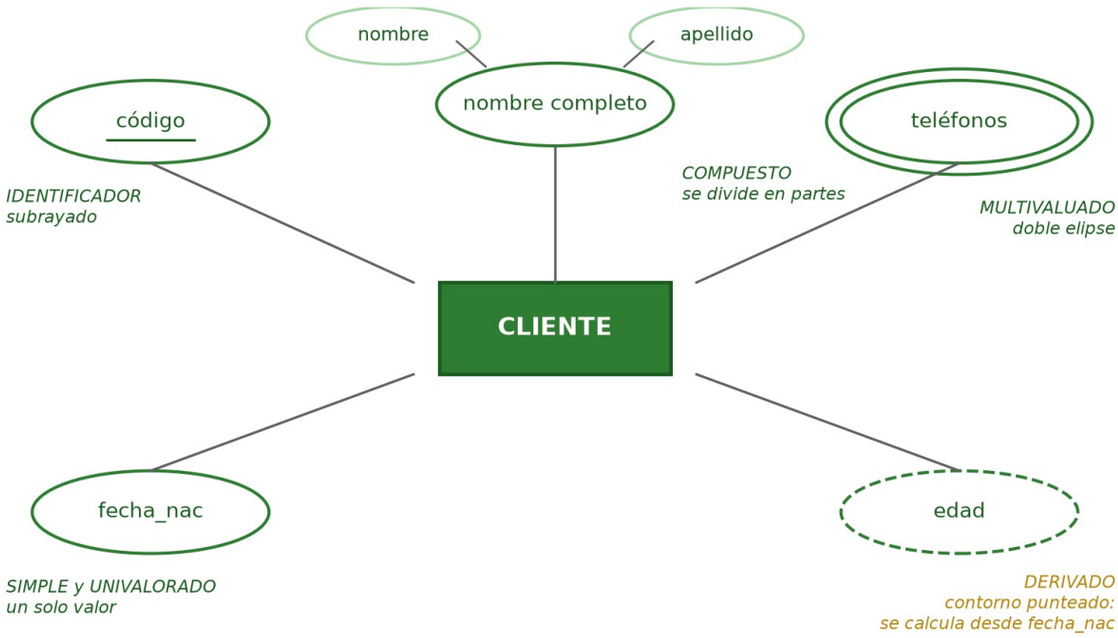


Figura 12.1 — Notación de los tipos de atributo, sobre la entidad CLIENTE.

Ejemplo — las entidades del Copetín

El Copetín tiene tres entidades claras: Producto (código, nombre, categoría, precio), Cliente (código, nombre, teléfono) y Venta (código, fecha). El precio es un atributo simple; el nombre del cliente podría verse como compuesto.

Concepto clave — entidad y atributo

Entidad: cosa sobre la que guardamos datos. Atributo: cada dato de esa entidad. Un atributo suele servir de clave (identifica el registro).

Entidades fuertes y débiles

Una entidad fuerte existe por sí misma y tiene un identificador propio: un producto o un cliente del Copetín no dependen de nadie para tener identidad. Una entidad débil, en cambio, depende de otra para existir y para identificarse: el «renglón» de una venta (2 chipas dentro de V1) no existe sin la venta que lo contiene.

Concepto	Qué significa	En el Copetín
Entidad fuerte	Existe por sí misma y tiene identificador propio	PRODUCTO (código), VENTA (número)
Entidad débil	No existe sin otra entidad, de la que depende (dependencia de existencia)	RENGLÓN de una venta
Discriminante (identificador parcial)	Distingue a las entidades débiles que dependen de una misma entidad fuerte, pero no alcanza solo	nro_renglón: 1, 2, 3… dentro de cada venta
Relación identificadora	Une la débil con su fuerte y le «presta» la clave	VENTA tiene RENGLÓN

En el DER, la entidad débil se dibuja con doble rectángulo, la relación identificadora con doble rombo y el discriminante se subraya con línea discontinua. La clave de la entidad débil se forma con la clave de la fuerte más el discriminante: (venta, nro_renglón).

Entidad débil o relación N:M con atributos

El renglón de venta admite dos dibujos correctos. Uno, como entidad débil RENGLÓN, que depende de VENTA. Otro, como los datos (cantidad y precio_unitario) de la relación N:M «contiene» entre VENTA y PRODUCTO. La diferencia está en qué identifica a cada renglón: en la relación N:M, la pareja venta + producto; en la entidad débil, la venta más su propio discriminante.

En este caso, si la regla del negocio establece que un mismo producto aparece como máximo una vez dentro de cada venta, ambas alternativas pueden conducir a DetalleVenta con clave venta + producto. Si el mismo producto puede aparecer en dos renglones separados, RENGLÓN necesita un discriminante propio, por ejemplo nro_renglón, y ambos modelos ya no producen exactamente la misma clave.

Caso	Regla del negocio	Modelo	Clave de DetalleVenta
A	Cada producto aparece una sola vez por venta (si se piden más, se suma la cantidad)	Relación N:M «contiene» con atributos	(venta, producto)
B	Un producto puede aparecer en dos renglones distintos (por ejemplo, 2 empanadas cobradas y 1 de cortesía a G. 0)	Entidad débil RENGLÓN con discriminante nro_renglón	(venta, nro_renglón); producto queda como clave foránea común

El Copetín sigue la regla del Caso A; por eso, en las Clases 13 a 15, el renglón se dibuja como la relación «contiene» con sus atributos.

A veces una relación N:M empieza a tener vida propia: se la numera, se la consulta por sí misma o se relaciona con otras entidades. Entonces conviene dibujarla como entidad asociativa, un rectángulo que nace de la relación (en algunos textos, un rombo dentro de un rectángulo). Pasa, por ejemplo, con la inscripción de un alumno a un taller cuando además se le registran las asistencias.

Ejemplo — fuerte y débil en el Copetín

La chipa (P01) existe aunque hoy nadie la compre: entidad fuerte. El renglón «2 chipas» solo existe dentro de la venta V1: si se lo modela como entidad, es débil. Si V1 se anulara, sus renglones desaparecerían con ella; la chipa seguiría existiendo.

El dominio de un atributo: qué valores acepta

Cada atributo tiene un dominio: el conjunto de valores que puede tomar. Definir dominios es ponerle límites sanos a los datos: un precio no puede ser negativo, una categoría solo puede ser una de las tres del Copetín, una fecha debe ser una fecha real. El gestor usará los dominios para rechazar datos absurdos antes de que entren.

Atributo	Dominio	Ejemplo válido	Ejemplo inválido
código de producto	P + dos dígitos	P03	X9
precio	Número entero positivo	6.000	−500
categoría	Panificados / Salados / Bebidas	Salados	Lácteos
fecha de venta	Fecha del calendario	04/03/2026	35/03
cantidad	Entero de 1 en adelante	3	0

Ejemplo resuelto — la ficha de la entidad PRODUCTO

Armemos la ficha de diseño completa de la entidad PRODUCTO, decisión por decisión.

1. Nombrar la entidad en singular y mayúsculas: PRODUCTO.
2. Listar los atributos que el mini-mundo necesita: código, nombre, categoría, precio.
3. Clasificarlos: código es identificador; nombre y categoría son simples; precio es simple y numérico.
4. Definir el dominio de cada uno (como en la tabla anterior).
5. Elegir el identificador: código, porque es único, estable y nunca vacío.
6. Validar la ficha contra los datos reales: los seis productos (P01 a P06) encajan sin excepciones — ninguno tiene dos categorías ni precio no numérico. La ficha está lista para el DER.

Aplicación profesional — dominios sujetos a reglas externas

Algunos dominios no dependen solo del diseñador: pueden estar condicionados por normas, estándares o reglas externas. Cuando un sistema maneja datos regulados, el diseño debe consultar la fuente oficial vigente y convertir esas reglas en validaciones documentadas.

Atributos derivados: calcular en vez de guardar

Un atributo derivado se obtiene de otros por cálculo: el subtotal de un renglón (cantidad × precio_unitario), el total de una venta (suma de sus subtotales), la edad (desde la fecha de nacimiento). La regla de diseño es clara: lo derivado, en general, NO se guarda — se calcula al momento de mostrarlo. Guardarlo duplicaría información y abriría la puerta a incoherencias entre el dato guardado y el recalculado.

Ejemplo — el subtotal que no se guarda

El renglón (V3, empanada, 3) no guarda «18.000»: cuando alguien lo consulta, el sistema multiplica 3 × 6.000 y muestra G. 18.000. Si mañana un renglón se corrigiera de 3 a 2 empanadas, el subtotal mostrado sería 12.000 sin que nadie tenga que acordarse de actualizarlo. Calcular siempre desde la fuente elimina toda posibilidad de desfase.

Atributos multivaluados y cómo tratarlos

Un atributo multivaluado toma varios valores a la vez para el mismo registro: los teléfonos de un cliente, por ejemplo. Meter «tres teléfonos en una celda» viola la atomicidad que exigirá la primera forma normal. El tratamiento correcto lo vas a reconocer enseguida: el atributo multivaluado se saca a una entidad aparte, relacionada con la original — exactamente el movimiento que la normalización hará sistemático en las clases finales de esta unidad.

Tipo de atributo	Ejemplo	Tratamiento en el diseño
Simple	precio, fecha	Queda como campo común
Compuesto	nombre (nombre + apellido)	Se decide: junto o separado, según el uso
Derivado	subtotal, edad	No se guarda: se calcula
Multivaluado	teléfonos del cliente	Entidad aparte relacionada

Ejemplo resuelto — cuatro decisiones de atributos en el Copetín

1. El total de la venta: derivado (suma de cantidad × precio_unitario de sus renglones). Decisión: no se guarda; se calcula. Los totales que manejás (14.000, 19.000, 34.000, 17.000, 50.000) son siempre reconstruibles.
2. El nombre del cliente: compuesto (nombre y apellido). Decisión: separado en dos campos, porque el dueño quiere ordenar por apellido.
3. Los teléfonos del cliente: multivaluado. Decisión: si el reparto prospera, entidad Teléfono aparte; por ahora, un solo teléfono principal como campo simple.

4. La categoría del producto: simple, con dominio cerrado (tres valores). Decisión: campo común con regla de validación.

5. Cada decisión quedó anotada en la ficha de la entidad: el DER que viene se dibuja sobre decisiones, no sobre supuestos.

Concepto clave — la excepción del derivado guardado

Hay una excepción honrosa: los documentos legales. Una factura emitida guarda su total impreso aunque sea derivado, porque el documento debe quedar congelado tal como se emitió, incluso si un precio cambia después. La regla general calcula; el documento histórico, conserva.

Requisitos de un buen identificador

No cualquier atributo sirve de identificador: se le exigen tres virtudes. Unicidad: dos ejemplares jamás comparten el valor. Estabilidad: el valor no cambia con el tiempo. Minimalidad: no sobra nada — si con un campo alcanza, no se usan dos. Evaluar candidatos con esta vara es un ejercicio breve que evita dolores largos.

1. Candidato 1 para PRODUCTO — el nombre: único hoy (no hay dos «Chipa»), pero inestable (puede corregirse) y con riesgo futuro de homónimos. Rechazado.

2. Candidato 2 — el precio: ni siquiera único (dos productos pueden costar lo mismo). Rechazado de plano.

3. Candidato 3 — el código P01: único por construcción, estable porque no significa nada que deba actualizarse, mínimo (un solo campo). Elegido.

4. El mismo examen consagra a C01 en CLIENTE y a V1 en VENTA: en los tres casos gana el código artificial, y no por casualidad.

Concepto clave — el subrayado es una promesa

Subrayar un atributo en el DER es prometer que ese valor identificará al ejemplar para siempre. Por eso la elección se toma con la vara de las tres virtudes y no por comodidad: las promesas del diseño se pagan durante toda la vida del sistema.

Actividades de aplicación

Ejercitá:

1. Definí entidad y definí atributo.
2. Clasificá: precio (simple o compuesto); teléfonos de un cliente (univalorado o multivalorado).
3. Indicá un atributo derivado y de qué otro se calcula.
4. Listá los atributos de la entidad Producto del Copetín.

Resolvé (caso Copetín Karumbé):

5. En el sistema de un colegio, clasificá los atributos de ALUMNO: cédula, nombre completo, fecha de nacimiento, edad y teléfonos de los padres.
6. En la entidad Cliente, indicá qué atributo puede ser la clave y por qué.
7. Definí el dominio de: cantidad vendida, categoría y código de cliente.

Pensá y decidí (justificá tu respuesta):

8. «Un atributo multivalorado guarda un solo valor por registro.»
9. «Cada entidad se convierte luego en una tabla.»

Desafío:

10. Diseñá la entidad Empleado del Copetín con cuatro atributos e indicá el tipo de cada uno (simple, compuesto, derivado…).

Práctica 12 — Las entidades del club: socios y disciplinas

Nombre y Apellido: _____ Curso: _____ Fecha: ____ / ____ / ____

Competencia de la práctica

Clasificar atributos, definir dominios, elegir identificadores y dibujarlos con la notación del DER.

Entorno de trabajo: papel y lápiz.

Lo que necesitás saber

Simple/compuesto, univalorado/multivalorado y derivado son tres preguntas independientes sobre cada atributo. El identificador va subrayado; el multivaluado, con doble elipse; el derivado, punteado.

Seguimos con el Club Atlético Yvága de la Práctica 11.

Actividad 1 — La ficha de SOCIO

Objetivo: *Clasificar los atributos de SOCIO y definir sus dominios.*

Modelo — Atributos relevados

nro_socio · cédula · nombre completo · fecha de nacimiento · edad · dirección (calle, número y barrio) · teléfonos · categoría (infantil, juvenil o mayor, según la edad) · fecha de alta

Pasos:

1. Armá una tabla: atributo, simple o compuesto, univalorado o multivalorado, ¿derivado?, dominio.
2. Elegí el identificador y explicá por qué no elegiste la cédula.
3. Dibujá SOCIO con todos sus atributos en notación clásica.

Punto de control: Los atributos derivados están punteados y en tu tabla dice que no se guardan.

Actividad 2 — DISCIPLINA y sus horarios

Objetivo: *Decidir cómo tratar un atributo multivaluado.*

Modelo — Atributos de DISCIPLINA

código · nombre · cuota mensual · horarios (por ejemplo, lunes 18:00 y jueves 18:00)

Pasos:

1. Clasificá cada atributo.
2. Proponé el tratamiento de horarios en el diseño.
3. Escribí el dominio de cuota mensual.

Punto de control: Tu propuesta para horarios no deja dos valores en una misma celda.

Desafío final (para quienes terminan antes)

Cada cuota se identifica por el socio y el número de mes (cuota 3 del socio 15). ¿Es CUOTA una entidad fuerte o débil? Dibujala.

Clase 13 — Relaciones y cardinalidad

Capacidad	Trabaja con el modelo Entidad Relación en la representación de los datos.
Tema	Relaciones entre entidades. Cardinalidad: 1:1, 1:N, N:M.
Indicadores de logro	• Identifica la relación entre dos entidades. • Determina la cardinalidad de una relación. • Reconoce una relación de muchos a muchos.

Las entidades se relacionan

Las entidades no viven aisladas: se relacionan. Un cliente realiza ventas; una venta contiene productos. La relación es ese vínculo entre entidades.

Cardinalidad: cuántos con cuántos

La cardinalidad (o ligadura de correspondencia) dice cuántos registros de una entidad se vinculan con cuántos de la otra. Es de uno a uno (1:1) cuando a cada uno le corresponde uno solo; de uno a muchos (1:N) cuando uno se vincula con varios; y de muchos a muchos (N:M) cuando varios se vinculan con varios. El programa nombra también la ligadura «varios a uno» (N:1): no es una cuarta categoría, sino la misma 1:N leída desde el otro extremo («muchas ventas pertenecen a un cliente»).

En la figura, cada punto es un registro y cada línea un vínculo: así se ven, dibujadas, las tres cardinalidades con ejemplos del Copetín.

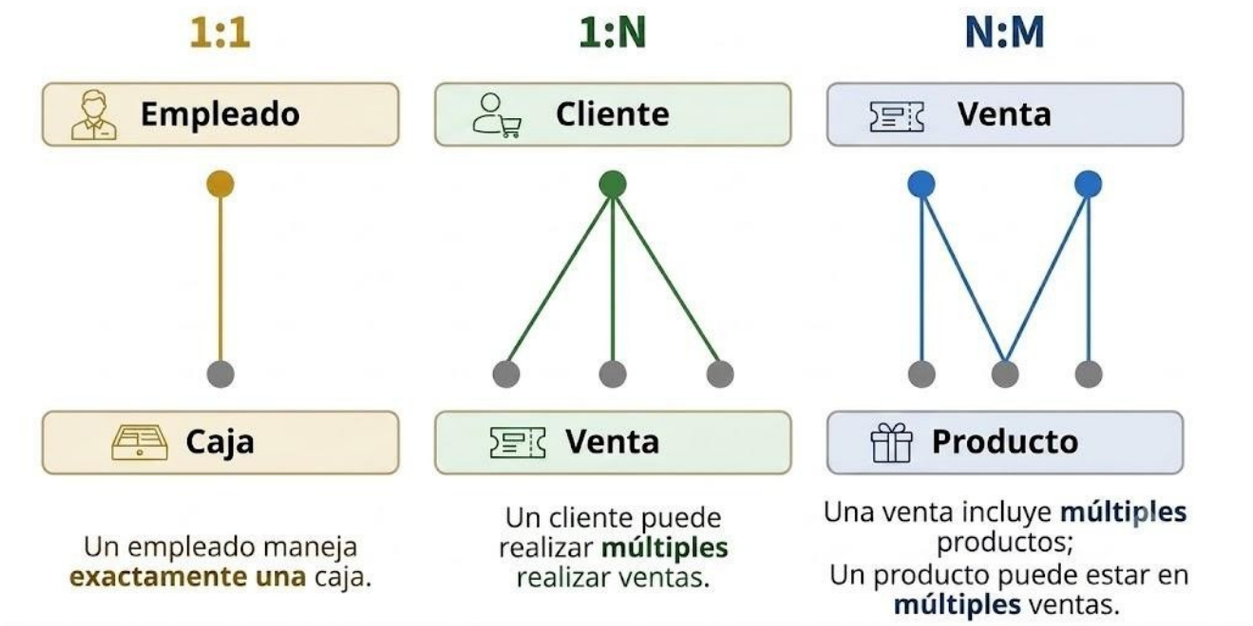


Figura 13.1 — Las tres cardinalidades: 1:1, 1:N y N:M, registro por registro.

Cardinalidad	Ejemplo en el Copetín
1:N (uno a muchos)	Un cliente realiza muchas ventas
N:M (muchos a muchos)	Una venta tiene muchos productos y un producto está en muchas ventas
1:1 (uno a uno)	Cada empleado maneja una caja y cada caja, un empleado

Ejemplo — la venta y los productos: N:M

La venta V5 tiene tres productos (milanesa, gaseosa, cocido) y la gaseosa está en varias ventas (V1, V3, V5). Muchas ventas con muchos productos: es una relación de muchos a muchos (N:M), la más interesante de resolver.

Concepto clave — cardinalidad

Indica cuántos registros de una entidad se relacionan con cuántos de la otra: 1:1, 1:N o N:M.

Las relaciones de muchos a muchos no se pueden guardar directamente en dos tablas: necesitan una tabla intermedia. Lo vas a ver en la clase «del diagrama a las tablas».

Participación total y parcial

La cardinalidad dice cuántos; la participación dice si es obligatorio participar. En la relación «Cliente realiza Venta», todo depende del lado: una venta SIEMPRE pertenece a un cliente (participación total u obligatoria: mínimo 1), pero un cliente puede existir sin haber comprado todavía (participación parcial u opcional: mínimo 0). La figura lo muestra con la notación de mínimos y máximos.

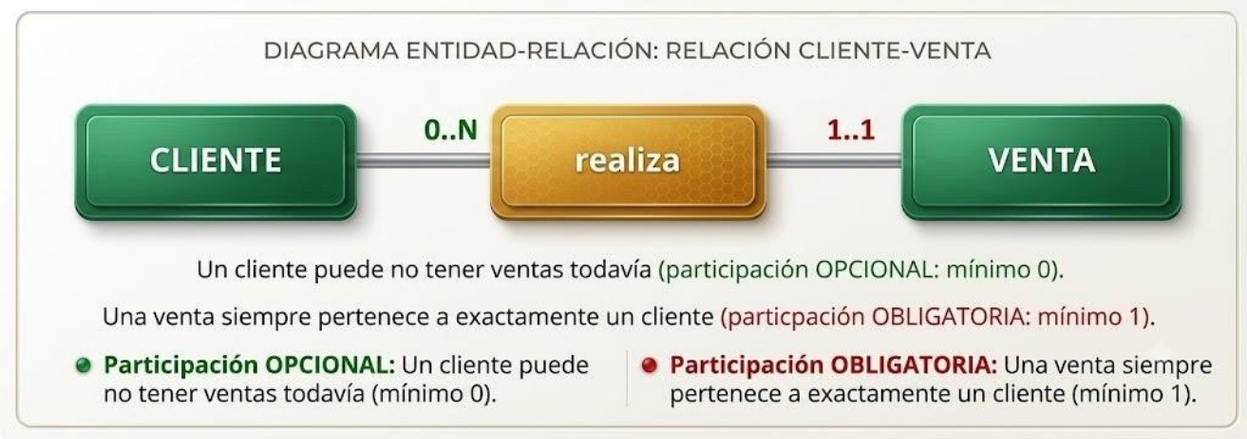


Figura 13.2 — Participación en «Cliente realiza Venta»: opcional del lado del cliente, obligatoria del lado de la venta.

> **Concepto clave — leé la relación en las dos direcciones**
> Toda relación se lee dos veces: «un cliente realiza cero o muchas ventas» y «una venta es realizada por exactamente un cliente». Si las dos lecturas suenan verdaderas en el mini-mundo, la cardinalidad y la participación están bien puestas.

Ejemplo resuelto — leer la cardinalidad en los datos reales

La cardinalidad no se declama: se verifica contra los datos. Tomemos las cinco ventas del Copetín y comprobemos, registro por registro, que la relación Cliente–Venta es 1:N.

1. Listar las ventas con su cliente: V1 → C01, V2 → C02, V3 → C01, V4 → C03, V5 → C04.
2. Verificar el lado «una venta, un cliente»: cada venta apunta a exactamente un código. Ninguna venta tiene dos clientes ni cero.
3. Verificar el lado «un cliente, muchas ventas»: C01 (Rosa) aparece en dos ventas (V1 y V3); C02, C03 y C04, en una cada uno. Un cliente PUEDE repetirse.
4. Conclusión: máximo 1 de un lado, máximo N del otro → la relación es 1:N, con participación obligatoria del lado de la venta.
5. Contraste: si existiera una venta compartida por dos clientes, la relación ya no sería 1:N — y el diseño entero cambiaría. Por eso se verifica ANTES de seguir.

Lectura	¿Se cumple en los datos?	Evidencia
Una venta pertenece a un solo cliente	Sí	V1 a V5: un código cada una
Un cliente puede tener varias ventas	Sí	C01 tiene V1 y V3
Un cliente puede no tener ventas aún	Sí	Un cliente nuevo entra sin comprar

> **Ejemplo cotidiano — cardinalidades de la vida real**
> Las cardinalidades están en todos lados: una cédula pertenece a una sola persona (1:1); una cooperativa tiene muchos socios (1:N); los estudiantes cursan muchas materias y cada materia tiene muchos estudiantes (N:M). Modelar es reconocer estos patrones en el negocio que tengas enfrente.

Relaciones recursivas: la entidad que se relaciona consigo misma

Nada impide que una entidad se relacione consigo misma: un empleado supervisa a otros empleados; un producto es combo de otros productos. Son las relaciones recursivas, y se dibujan con el rombo conectado dos veces a la misma entidad, con roles en cada extremo (supervisa / es supervisado). Si el Copetín armara promociones combinadas — «combo merienda: chipa + cocido» — la relación Producto-compone-Producto sería recursiva.

Grado de una relación: unaria, binaria y ternaria

Cuántas entidades participan en una relación es su grado. Una relación recursiva, como la del empleado que supervisa a otros empleados, es de grado uno (unaria). «Cliente realiza Venta» es de grado dos (binaria), el caso

más frecuente. Y existen relaciones de grado tres (ternarias), en las que el hecho solo tiene sentido si se nombran tres entidades a la vez.

Compará los tres dibujos de la figura: lo que cambia es cuántos rectángulos toca el rombo. En la ternaria, el rombo «dicta» une tres entidades a la vez.

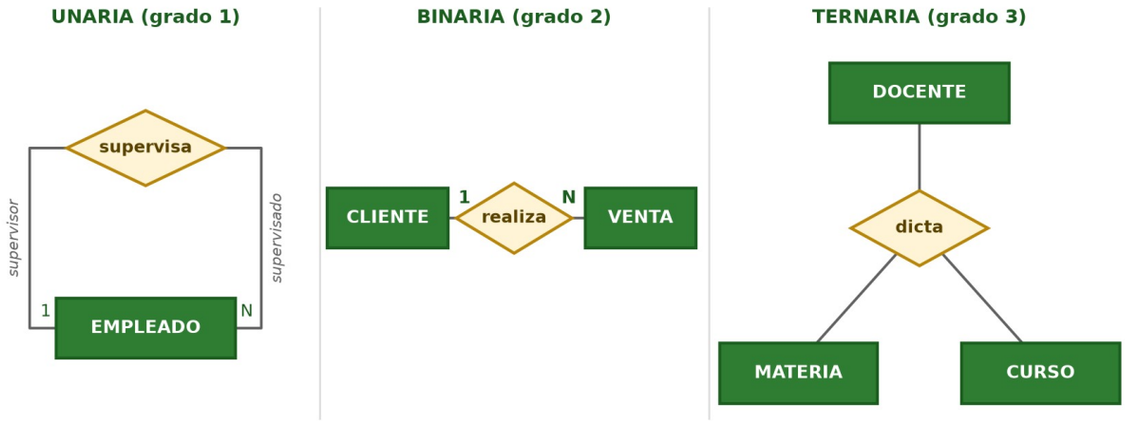


Figura 13.3 — Grado de una relación: unaria, binaria y ternaria.

Grado	Entidades que participan	Ejemplo
Unaria (1)	Una entidad consigo misma	EMPLEADO supervisa EMPLEADO
Binaria (2)	Dos entidades	CLIENTE realiza VENTA
Ternaria (3)	Tres entidades a la vez	DOCENTE dicta MATERIA en CURSO

Ejemplo — por qué la ternaria no se parte en dos

En un colegio, la profesora Benítez dicta Algorítmica en 3.º A y Matemática en 3.º B. Si guardáramos solo «Benítez dicta Algorítmica» y «Benítez enseña en 3.º A», no sabríamos qué materia da en cada curso: el hecho «quién dicta qué y dónde» necesita las tres entidades juntas. Esa es la señal de una relación ternaria.

Atributos de relación: dónde viven cantidad y precio_unitario

Algunos atributos no pertenecen a ninguna entidad sino al encuentro entre dos. En el Copetín, cantidad y precio_unitario son hechos del PAR venta-producto: una misma venta tiene varios productos y un mismo producto puede venderse en operaciones distintas, incluso a precios históricos diferentes. En el DER, esos atributos pertenecen a la relación «contiene». Ubicarlos bien evita confundir el precio de catálogo actual con lo efectivamente cobrado.

Atributo dudoso	¿De quién es?	Dónde se dibuja
cantidad	Del par venta-producto	Sobre el rombo «contiene»
fecha	De la venta	En la entidad VENTA
precio de catálogo	Del producto	En la entidad PRODUCTO
descuento de una venta puntual	Del par (si es por renglón) o de la venta (si es global)	Donde la realidad del negocio diga
precio_unitario cobrado	Del par venta-producto	Sobre el rombo «contiene»

Ejemplo resuelto — el interrogatorio del atributo

Para ubicar un atributo dudoso, se lo interroga con dos preguntas. Apliquémoslas primero a la cantidad y luego al precio_unitario.

1. Pregunta A: ¿la cantidad queda fija sabiendo SOLO la venta? No — V1 tiene 2 chipas y 1 gaseosa: dos cantidades. No es atributo de VENTA.

2. Pregunta B: ¿queda fija sabiendo SOLO el producto? No — la gaseosa aparece con cantidad 1 (V1) y con cantidad 2 (V3 y V5). No es atributo de PRODUCTO.

3. Pregunta C: ¿queda fija sabiendo el PAR venta-producto? Sí — (V1, gaseosa) determina cantidad 1, sin ambigüedad. Es atributo de la relación.

4. Repetí el interrogatorio con precio_unitario: saber sólo el producto no fija lo cobrado si el precio puede cambiar entre ventas; saber sólo la venta tampoco identifica un renglón. La pareja venta+producto sí lo determina. Esta es una dependencia funcional de la clave completa.

Concepto clave — el rombo también tiene datos

Que las relaciones puedan tener atributos sorprende al principio y después ordena todo: la mitad de los errores de diseño novato son atributos de relación forzados dentro de una entidad. Cuando un dato «no encaja» en ninguna entidad, preguntate si no pertenece al encuentro entre dos.

Cardinalidad máxima y mínima, leídas juntas

La cardinalidad que conocés (1:N, N:M) es la máxima: cuántos como MUCHO. La participación de la sección anterior es la mínima: cuántos como POCO (cero u uno). Las notaciones profesionales las escriben juntas, como un par (mínima, máxima) en cada extremo de la relación — y leerlas de corrido describe el negocio con precisión de contrato.

Extremo de la relación	Par (mín, máx)	Lectura en voz alta
CLIENTE en «realiza»	(0, N)	Un cliente puede no tener compras, o tener muchas
VENTA en «realiza»	(1, 1)	Toda venta tiene exactamente un cliente
VENTA en «contiene»	(1, N)	Toda venta tiene al menos un producto
PRODUCTO en «contiene»	(0, N)	Un producto puede no haberse vendido aún

Ejemplo — el contrato leído contra los datos

Verificá el par (0, N) de PRODUCTO con el caso: cinco productos registran ventas, y si mañana se agregara el jugo sin vender, la base lo aceptaría sin protestar — el mínimo 0 lo permite. En cambio, una venta sin cliente violaría el (1, 1) de VENTA y el gestor la rechazaría. El DER es un contrato, y los datos lo cumplen.

Errores frecuentes — al determinar cardinalidades

1. Mirar un solo sentido. «Un cliente hace muchas ventas» no alcanza: falta preguntar cuántos clientes tiene una venta. La cardinalidad sale de las dos lecturas juntas.
2. Decidir por los datos de hoy. Si hoy cada cliente compró una sola vez, igual la relación es 1:N: la regla del negocio permite volver a comprar.
3. Confundir máximo con mínimo. «Un producto puede no haberse vendido» habla del mínimo (0); no cambia que el máximo sea N.
4. Inventar una 1:1 que es 1:N. Antes de declarar 1:1, buscá un caso real donde un lado tenga dos: si existe, la relación es 1:N.

Actividades de aplicación

Ejercitá:

1. Definí «relación» y «cardinalidad».
2. Indicá la cardinalidad: un cliente y sus ventas.
3. Indicá la cardinalidad: ventas y productos.
4. Dado «cada país tiene una capital y cada capital pertenece a un país», indicá la cardinalidad.

Resolvé (caso Copetín Karumbé):

5. En el Copetín, determiná la cardinalidad entre Cliente y Venta, y entre Venta y Producto.
6. Justificá con los datos que Venta–Producto es N:M usando la chipa y la venta V2.
7. Indicá el grado de cada relación: (a) un socio de la biblioteca recomienda a otro socio; (b) un cliente realiza ventas; (c) un docente dicta una materia en un curso.
8. Escribí el par (mín, máx) de cada extremo de «CURSO tiene ALUMNO», sabiendo que todo alumno está inscripto en exactamente un curso y que puede haber cursos recién abiertos sin alumnos.

Pensá y decidí (justificá tu respuesta):

9. «Una relación de muchos a muchos se guarda directamente en dos tablas.»
10. «Un cliente puede realizar muchas ventas: la relación Cliente-Venta es 1:N.»

Desafío:

11. Proponé una relación 1:1 y una 1:N en un sistema escolar, e indicá las entidades de cada una.

Práctica 13 — Cardinalidades del club, leídas en los datos

Nombre y Apellido: _____ Curso: _____ Fecha: ____ / ____ / ____

Competencia de la práctica

Determinar cardinalidad, participación y grado de relaciones nuevas a partir de datos y reglas del negocio.

Entorno de trabajo: papel y lápiz.

Lo que necesitás saber

Toda relación se lee en los dos sentidos. El par (mín, máx) de cada extremo combina participación (0 o 1) y cardinalidad (1 o N).

Datos del club: socios 1 Alan, 2 Belén, 3 Carlos y 4 Diana; disciplinas FUT, VOL y BAS; entrenadores Ruiz y Paredes.

Actividad 1 — Leer las cardinalidades

Objetivo: *Determinar la cardinalidad de SOCIO–DISCIPLINA y de ENTRENADOR–DISCIPLINA.*

Modelo — Registros

Inscripciones: Alan–FUT · Alan–BAS · Belén–VOL · Carlos–FUT · Diana–VOL · Diana–FUT

Entrena: Ruiz–FUT · Paredes–VOL · Paredes–BAS

Regla del club: cada disciplina tiene un solo entrenador

Pasos:

1. Para cada relación, contá cuántos del otro lado tiene cada uno (en los dos sentidos).
2. Escribí la cardinalidad y el par (mín, máx) de cada extremo.
3. Indicá cuál es la relación N:M y qué evidencia la prueba.

Punto de control: Para cada relación escribiste dos lecturas, una por sentido, y una evidencia de los datos.

Actividad 2 — Grado y relaciones especiales

Objetivo: *Clasificar relaciones por su grado.*

Modelo — Enunciados

(a) Un socio es padrino de otros socios nuevos.

(b) Un entrenador dirige una disciplina en una cancha determinada, y el dato solo tiene sentido con los tres.

(c) Un socio paga cuotas.

Pasos:

1. Indicá el grado de cada relación.
2. Dibujá (a) con sus dos roles.

Punto de control: Para cada enunciado contaste cuántas entidades participan y, en (a), nombraste los dos roles.

Desafío final (para quienes terminan antes)

El club decide que un socio puede anotarse en dos disciplinas como máximo. ¿Sigue siendo N:M la relación? ¿Cómo lo anotarías con el par (mín, máx)?

Clase 14 — El diagrama Entidad-Relación

Capacidad	Trabaja con el modelo Entidad Relación en la representación de los datos.
Tema	El diagrama Entidad-Relación (DER): entidades, atributos y relaciones.
Indicadores de logro	• Reconoce los símbolos del diagrama entidad-relación. • Interpreta un DER sencillo. • Dibuja un DER a partir de un caso.

Un dibujo del diseño

El diagrama entidad-relación (DER) es el dibujo del modelo. Representa las entidades como rectángulos, los atributos como óvalos (o listas dentro del rectángulo) y las relaciones como rombos o líneas que unen las entidades, con la cardinalidad anotada.

El DER del Copetín

Con tres entidades y sus vínculos, el DER del Copetín queda así: Cliente se une a Venta con la relación «realiza» (1:N), y Venta se une a Producto con la relación «contiene» (N:M), que lleva los atributos cantidad y precio_unitario.

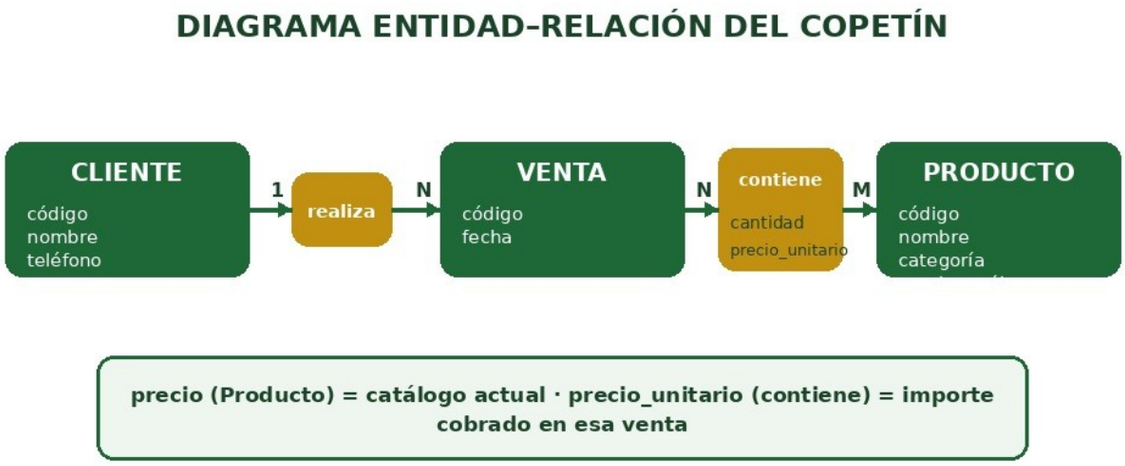


Figura 14.1 — Diagrama Entidad-Relación del Copetín: la relación contiene incorpora cantidad y precio_unitario como atributos de la operación.

Entidad	Atributos	Se relaciona con
Cliente	código, nombre, teléfono	Venta (realiza, 1:N)
Venta	código, fecha	Cliente y Producto
Producto	código, nombre, categoría, precio	Venta (contiene, N:M)

Ejemplo — leer el DER del Copetín

El DER se lee así: «un Cliente realiza muchas Ventas; una Venta contiene muchos Productos y un Producto aparece en muchas Ventas, indicando la cantidad y el precio unitario cobrado». Ese dibujo resume todo el diseño en una hoja.

Concepto clave — diagrama entidad-relación

Representación gráfica del modelo: rectángulos (entidades), óvalos o listas (atributos) y líneas/rombos (relaciones) con su cardinalidad.

Las convenciones de dibujo: rectángulos, elipses y rombos

El DER se dibuja con un vocabulario visual acordado, para que cualquier diseñador lo lea sin explicaciones. Las convenciones clásicas son pocas y precisas.

Símbolo	Representa	Ejemplo del Copetín
Rectángulo	Entidad	CLIENTE, PRODUCTO, VENTA
Elipse	Atributo	nombre, precio, fecha
Elipse con subrayado	Atributo clave	código
Rombo	Relación	realiza, contiene
Letras en los extremos	Cardinalidad	1, N, M

Ejemplo resuelto — construir el DER paso a paso

Un DER no se dibuja de un tirón: se construye por capas. Recorramos el método con el mini-mundo del Copetín.

Paso 1 — Entidades: dibujar los rectángulos CLIENTE, VENTA y PRODUCTO, bien separados.

Paso 2 — Relaciones: unir CLIENTE y VENTA con el rombo «realiza»; VENTA y PRODUCTO con el rombo «contiene».

Paso 3 — Cardinalidades: «realiza» lleva 1 del lado del cliente y N del lado de la venta; «contiene» lleva N y M, porque una venta tiene muchos productos y un producto está en muchas ventas.

Paso 4 — Atributos: colgar las elipses de cada entidad (código subrayado en las tres) y los atributos cantidad y precio_unitario sobre el rombo «contiene»: pertenecen a la relación, no a las entidades.

Paso 5 — Lectura de control: recorrer el diagrama leyendo cada relación en las dos direcciones. Si todas las lecturas suenan verdaderas, el DER está listo para traducirse a tablas.

Errores frecuentes al dibujar un DER

Hay errores que se repiten en todos los primeros diagramas: relaciones sin nombre ni cardinalidad, el mismo atributo repetido en dos entidades, atributos que en realidad son entidades (una elipse «cliente» colgando de VENTA en lugar de la entidad CLIENTE) y claves sin subrayar. La figura contrasta un fragmento mal dibujado con su versión correcta.



Figura 14.2 — Un mismo fragmento del modelo, dibujado mal y dibujado bien.

Concepto clave — el atributo que pide ser entidad

Cuando un atributo tiene, a su vez, datos propios (el cliente tiene nombre, el producto tiene precio), no es un atributo: es una entidad que merece su rectángulo y su relación. Detectar esos «atributos disfrazados» es una de las habilidades centrales del modelador.

Cuestiones de diseño: ¿entidad, atributo o relación?

El programa pide resolver dos «cuestiones de diseño». La primera ya la viste: un dato que tiene datos propios merece ser entidad y no atributo. La segunda es más sutil: ¿un hecho del negocio se dibuja como entidad o como relación? La pregunta que decide es si ese hecho tiene identidad propia, es decir, si se lo nombra, se lo numera y se lo consulta por sí mismo.

Situación	Se modela como	Por qué
El cliente tiene código, nombre y teléfono	Entidad CLIENTE	Tiene datos propios: no puede ser una elipse de VENTA
El teléfono del cliente	Atributo de CLIENTE	Es un dato suelto, sin datos propios
La venta tiene número, fecha y cliente	Entidad VENTA	Se la numera y se la consulta por sí misma
Qué productos lleva cada venta	Relación N:M «contiene»	No tiene número propio: es el encuentro venta-producto
El préstamo de un libro tiene número de recibo y fecha de devolución	Entidad PRÉSTAMO	Tiene identidad (el recibo) y vida propia (se devuelve, se renueva)

Concepto clave — la prueba del número propio

Si el negocio le pone número al hecho (venta N.º 1, recibo de préstamo N.º 58), probablemente es una entidad. Si el hecho solo existe como vínculo entre dos cosas y se identifica por ellas, es una relación, y sus datos van sobre el rombo.

Otras notaciones: la pata de gallo

La notación clásica (rectángulos, elipses y rombos) es ideal para aprender, pero en la industria vas a cruzarte con la notación de pata de gallo: las entidades son cajas con sus atributos adentro, y la cardinalidad se dibuja en los extremos de la línea — tres «deditos» para el lado muchos, una raya para el lado uno. Dicen lo mismo con otra caligrafía; quien domina los conceptos lee cualquiera de las dos.

Elemento	Notación clásica	Notación pata de gallo
Entidad	Rectángulo	Caja con los atributos listados
Atributo	Elipse colgante	Renglón dentro de la caja
Clave principal	Subrayado	Marcada arriba, separada por una línea
Relación	Rombo con nombre	Línea directa entre cajas
Cardinalidad N	Letra N	Tres trazos («pata de gallo») en el extremo

Concepto clave — la notación es acento, no idioma

Cambiar de notación es cambiar de acento, no de idioma: entidades, atributos, claves y cardinalidades son los mismos en todas. Aprendé los conceptos a fondo y las notaciones se vuelven detalle de un día de práctica.

Validar el DER antes de seguir

El DER terminado no se archiva: se valida. Primero contra los datos reales (¿las cinco ventas del cuaderno entran sin forzar nada?), después contra las preguntas del dueño (¿el modelo puede responder cuánto recaudó cada categoría?) y por último con el propio dueño, leyéndole el diagrama en castellano llano.

Ejemplo resuelto — la lista de control del DER del Copetín

1. ¿Toda entidad tiene clave subrayada? CLIENTE, VENTA y PRODUCTO tienen su código.
2. ¿Toda relación tiene nombre y cardinalidad en ambos extremos? «realiza» (1:N) y «contiene» (N:M).
3. ¿Ningún atributo está repetido en dos entidades? El precio vive solo en PRODUCTO; la fecha, solo en VENTA.
4. ¿Los atributos de relación están sobre el rombo? Cantidad y precio_unitario, sobre «contiene».
5. ¿Los datos reales entran? Las cinco ventas, los cuatro clientes y los seis productos se representan sin excepciones ni parches.
6. ¿Responde las preguntas del dueño? Recaudación total, por cliente, por producto y por categoría: todas alcanzables recorriendo el diagrama. Conclusión: DER validado; se puede traducir a tablas.

Aplicación profesional — leerle el diagrama al cliente

La validación con el dueño se hace en castellano, no en jerga: «un cliente puede hacer muchas compras, cada compra es de un solo cliente, ¿es así tu negocio?». Cuando el que atiende el mostrador asiente frase por frase, el DER está listo. Si duda en alguna, ahí había un error que ningún software habría encontrado.

Del boceto al diagrama definitivo

Ningún DER sale definitivo al primer trazo: el proceso real es iterativo. Primer boceto a mano alzada mientras se escucha al cliente; segunda versión ordenada, ya con cardinalidades; tercera, tras validar con datos reales y descubrir el atributo olvidado. El borrador tachado no es fracaso: es el método funcionando.

La herramienta acompaña el momento: papel y lápiz para pensar, pizarra para discutir en equipo, y software de diagramación para la versión final que se archiva con el proyecto. Lo importante es indeleble: la notación correcta y completa, en cualquier soporte.

Actividades de aplicación

Ejercitá:

1. Indicá con qué símbolo se representa una entidad y con cuál una relación.
2. Nombrá las tres entidades del DER del Copetín.
3. Indicá qué atributos lleva la relación «contiene».
4. Escribí en palabras qué dice el DER del Copetín.

Resolvé (caso Copetín Karumbé):

5. Leé el DER del Copetín en castellano llano, en los dos sentidos de cada relación (cuatro oraciones).
6. Agregá al DER una entidad Empleado relacionada con Venta (1:N) e indicá cómo quedaría el vínculo.
7. ¿Entidad, atributo o relación? (a) la categoría del producto, si solo se usa su nombre; (b) la categoría, si además se guardan su descripción y el porcentaje de descuento de cada categoría; (c) «qué alumnos se inscribieron en qué taller».

Pensá y decidí (justificá tu respuesta):

8. «En un DER, las entidades se representan con rombos.»
9. «La relación entre Venta y Producto lleva los atributos cantidad y precio_unitario.»

Desafío:

10. Dibujá el DER de una biblioteca con Socio, Libro y Préstamo, indicando la cardinalidad de cada relación.

Práctica 14 — El DER de los talleres del colegio

Nombre y Apellido: _____ Curso: _____ Fecha: ____ / ____ / ____

Competencia de la práctica

Construir un DER completo y validado a partir de requisitos nuevos, con atributos de relación y cardinalidades en ambos extremos.

Entorno de trabajo: papel y lápiz.

Lo que necesitás saber

Rectángulo: entidad. Elipse: atributo (subrayado si es identificador). Rombo: relación, con sus atributos propios. Cardinalidad en los dos extremos.

Requisitos de la coordinación de talleres extracurriculares.

Actividad 1 — Del texto al diagrama

Objetivo: *Dibujar el DER a partir de los requisitos.*

Modelo — Requisitos

Cada alumno tiene código, nombre y curso. Cada taller tiene código, nombre y día de la semana.

Un alumno puede inscribirse en varios talleres y un taller tiene muchos alumnos; de cada inscripción interesa la fecha.

Cada taller lo dicta un solo docente, y un docente puede dictar varios talleres. Del docente interesan código, nombre y teléfono.

Cada taller usa un aula fija y cada aula aloja un solo taller. Del aula interesan código y capacidad.

Pasos:

1. Paso 1: dibujá las entidades.
2. Paso 2: uní con rombos y nombrá cada relación.
3. Paso 3: anotá las cardinalidades en los dos extremos.
4. Paso 4: colgá los atributos; ubicá la fecha de inscripción donde corresponde.

Punto de control: Hay tantas relaciones como vínculos describen los requisitos, y cada rombo tiene nombre y cardinalidad en los dos extremos.

Actividad 2 — Validar el diagrama

Objetivo: *Pasar tu DER por la lista de control de la clase y leerlo en voz alta.*

Modelo — Lista de control

¿Toda entidad tiene identificador subrayado?

¿Toda relación tiene nombre y cardinalidad en ambos extremos?

¿Algún atributo está repetido en dos entidades?

¿Los atributos de relación están sobre el rombo?

¿Responde «¿qué alumnos tiene el taller de Robótica y quién lo dicta?»?

Pasos:

1. Respondé cada pregunta con Sí o No.
2. Corregí lo que tenga No y volvé a controlar.
3. Leé cada relación en los dos sentidos ante un compañero.

Punto de control: Las cinco preguntas tienen Sí y tu compañero no encontró ninguna lectura falsa.

Desafío final (para quienes terminan antes)

La coordinación quiere registrar la asistencia de cada alumno a cada encuentro del taller (fecha y presente/ausente). Agregalo al DER y justificá si es entidad o relación.

Clase 15 — Del diagrama a las tablas

Capacidad	Trabaja con el modelo Entidad Relación en la representación de los datos.
Tema	Traducción del DER a tablas. Resolución de relaciones N:M.
Indicadores de logro	• Convierte una entidad en tabla con su clave principal. • Traduce una relación 1:N con clave foránea. • Resuelve una relación N:M con una tabla intermedia.

Cada entidad, una tabla

El DER se traduce a tablas con reglas simples. Cada entidad se convierte en una tabla, sus atributos en campos, y su identificador en clave principal. Cliente → tabla Clientes; Producto → tabla Productos; Venta → tabla Ventas.

Las relaciones 1:N

Una relación 1:N se resuelve poniendo la clave principal del lado «uno» como clave foránea en el lado «muchos». Como un cliente realiza muchas ventas, el código de cliente va como clave foránea en la tabla Ventas.

Las relaciones N:M

Una relación N:M no se puede guardar directamente en dos tablas: se crea una tabla intermedia (también llamada tabla puente o de unión). La relación «Venta contiene Producto» genera DetalleVenta, con las claves foráneas venta y producto y con los atributos propios de la relación: cantidad y precio_unitario.

La figura muestra la traducción: el rombo «contiene» del DER, con sus extremos N y M, se convierte en DetalleVenta. Sus dos claves foráneas forman la clave compuesta y la tabla conserva cantidad y precio_unitario, que dependen de la pareja venta-producto.

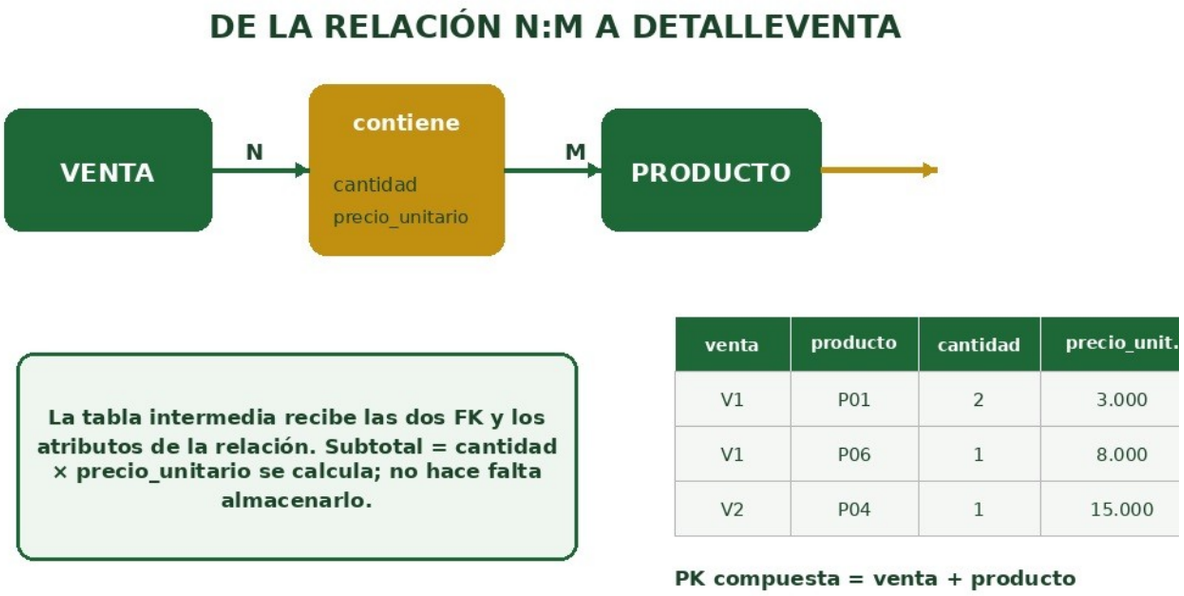


Figura 15.1 — De la relación N:M del DER a DetalleVenta, con PK compuesta venta+producto y atributos cantidad y precio_unitario.

Venta (FK)	Producto (FK)	Cantidad	Precio unitario (G.)
V1	P01 (Chipa)	2	3.000
V1	P06 (Gaseosa)	1	8.000
V2	P04 (Milanesa)	1	15.000
V2	P05 (Cocido)	1	4.000
V3	P03 (Empanada)	3	6.000
V3	P06 (Gaseosa)	2	8.000

Ejemplo — resolver la N:M del Copetín

«Venta contiene Producto» (N:M) se convierte en DetalleVenta: cada fila identifica venta y producto y conserva cantidad y precio_unitario. Así, V1 se registra en dos renglones: (V1,P01,2,3.000) y (V1,P06,1,8.000).

Concepto clave — la tabla intermedia

Toda relación N:M se resuelve con una tabla intermedia que contiene las claves foráneas de ambas entidades y los atributos propios de la relación.

Al terminar, el Copetín tiene cuatro tablas: Productos, Clientes, Ventas y DetalleVenta. Ese es el esquema que crearás en Access en la Unidad 4.

Las reglas de transformación, una por una

La traducción del DER a tablas sigue reglas mecánicas — tan mecánicas que, bien aprendidas, no dejan lugar a dudas. Las tres fundamentales aparecen en la tabla.

Regla	Qué genera	En el Copetín
Cada entidad → una tabla	Una tabla con los atributos de la entidad; su clave, la del DER	CLIENTE → Clientes; PRODUCTO → Productos; VENTA → Ventas
Relación 1:N → clave foránea	La clave del lado 1 se copia como campo del lado N	El código del cliente entra como campo de Ventas
Relación N:M → tabla intermedia	Una tabla nueva con las dos claves foráneas y los atributos de la relación	«contiene» → DetalleVenta (venta, producto, cantidad, precio_unitario)

El esquema resultante del Copetín tiene cuatro tablas conectadas por tres claves foráneas. La figura lo muestra completo, con cada flecha apuntando de la clave foránea a la clave principal que referencia.

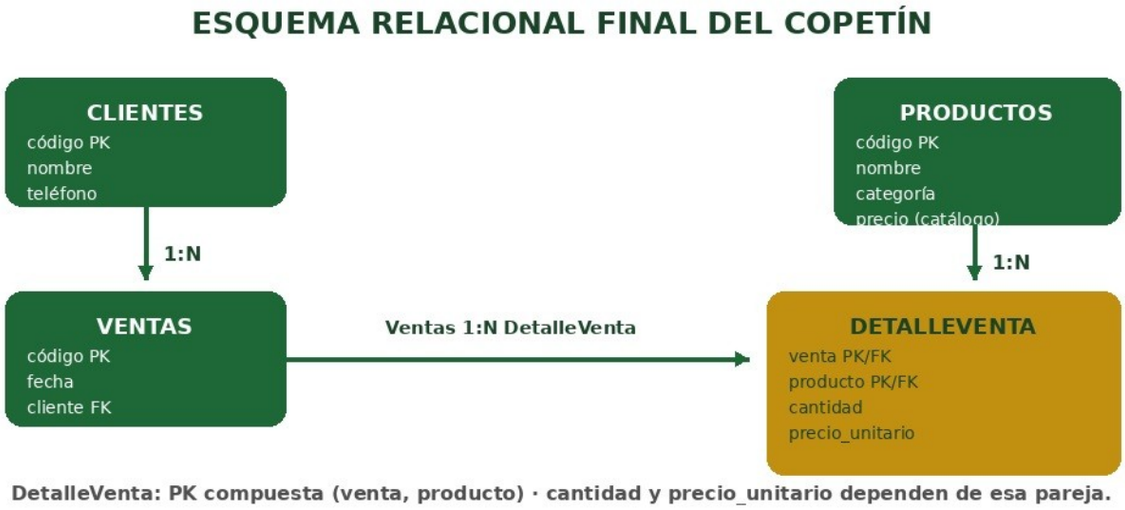


Figura 15.2 — Esquema relacional final del Copetín: cuatro tablas, PK compuesta en DetalleVenta y tres relaciones referenciales.

Ejemplo resuelto — DetalleVenta completo, venta por venta

Hasta ahora viste fragmentos de DetalleVenta. Construyamos la tabla COMPLETA para las cinco ventas del Copetín, y verifiquemos que reproduce exactamente la recaudación conocida.

V1 (Rosa, 03/03/2026): 2 chipas y 1 gaseosa → dos filas: (V1, P01, 2) y (V1, P06, 1). Subtotales: 6.000 + 8.000 = G. 14.000.

V2 (Luis, 03/03/2026): 1 milanesa y 1 cocido → (V2, P04, 1) y (V2, P05, 1). Subtotales: 15.000 + 4.000 = G. 19.000.

V3 (Rosa, 04/03/2026): 3 empanadas y 2 gaseosas → (V3, P03, 3) y (V3, P06, 2). Subtotales: 18.000 + 16.000 = G. 34.000.

V4 (Ana, 04/03/2026): 4 chipas y 1 mbeju → (V4, P01, 4) y (V4, P02, 1). Subtotales: 12.000 + 5.000 = G. 17.000.

V5 (Miguel, 05/03/2026): 2 milanesas, 2 gaseosas y 1 cocido → (V5, P04, 2), (V5, P06, 2) y (V5, P05, 1). Subtotales: 30.000 + 16.000 + 4.000 = G. 50.000.

1. Control final: 14.000 + 19.000 + 34.000 + 17.000 + 50.000 = G. 134.000. Las once filas de DetalleVenta reconstruyen, al guaraní, la recaudación total del Copetín.

Venta	Producto	Cantidad	Precio unit. (G.)	Subtotal (G.)
V1	P01 Chipa	2	3.000	6.000
V1	P06 Gaseosa	1	8.000	8.000
V2	P04 Milanesa	1	15.000	15.000
V2	P05 Cocido	1	4.000	4.000
V3	P03 Empanada	3	6.000	18.000
V3	P06 Gaseosa	2	8.000	16.000
V4	P01 Chipa	4	3.000	12.000
V4	P02 Mbeju	1	5.000	5.000
V5	P04 Milanesa	2	15.000	30.000
V5	P06 Gaseosa	2	8.000	16.000
V5	P05 Cocido	1	4.000	4.000

Concepto clave — la clave compuesta de DetalleVenta

En DetalleVenta, ni la venta ni el producto identifican solos una fila: V1 aparece dos veces y P06, en tres ventas. La clave es la combinación venta + producto — la clave compuesta que se anunció en la Unidad 2, ahora en acción.

Las relaciones 1:1: el caso menos frecuente

La relación 1:1 conecta registros de a uno: un empleado tiene UNA cuenta de usuario del sistema y esa cuenta pertenece a UN empleado. Al traducirla, la clave foránea puede ir en cualquiera de las dos tablas (conviene la del lado con participación obligatoria, para que no queden valores vacíos) y debe declararse sin duplicados: en Access, con la propiedad Indizado = «Sí (Sin duplicados)». Sin esa restricción, dos empleados podrían apuntar a la misma cuenta y la relación se convertiría, sin aviso, en 1:N. La otra salida es fusionar las dos entidades en una sola tabla, si siempre van juntas. Es la relación menos común: cuando aparezcan muchas 1:1 en un diseño, sospechá que algunas entidades se pueden unir.

Relación	Traducción a tablas	Regla mnemotécnica
1:1	FK única (sin duplicados) en el lado obligatorio, o fusión	«De a uno: elegí o fusioná»
1:N	La clave del lado 1 viaja como FK al lado N	«La del uno viaja al muchos»
N:M	Tabla intermedia con las dos FK	«Muchos con muchos: tabla nueva»

Verificar la traducción: el control del traductor

Terminada la traducción, el esquema se somete a un control de calidad propio, independiente del que se le hizo al DER: que no haya quedado ninguna relación N:M sin su tabla intermedia, que toda clave foránea apunte a una clave principal existente y del mismo tipo de dato, y que los atributos del rombo hayan aterrizado en la tabla intermedia y no en otro lado.

Ejemplo resuelto — el control final del esquema del Copetín

1. Conteo de tablas: 3 entidades + 1 relación N:M = 4 tablas. Coincide: Productos, Clientes, Ventas, DetalleVenta.
2. Claves foráneas: cliente en Ventas → Clientes; venta y producto en DetalleVenta → Ventas y Productos. Tres FK, las tres apuntando a claves existentes y del mismo tipo de dato que la clave a la que apuntan (Texto corto).
3. Atributos del rombo: cantidad y precio_unitario quedaron en DetalleVenta.
4. Relaciones sin traducir: ninguna — «realiza» se volvió la FK cliente; «contiene» se volvió DetalleVenta.

Prueba de fuego con datos: las cinco ventas del cuaderno se cargan completas en el esquema (las once filas de DetalleVenta que ya conocés) y desde él se reconstruye la recaudación de G. 134.000. La traducción no perdió información.

Concepto clave — dos controles, dos preguntas distintas

El control del DER pregunta «¿modelé bien la realidad?»; el control de la traducción pregunta «¿convertí bien el modelo?». Son errores de naturaleza distinta y por eso se revisan por separado: un DER perfecto mal traducido produce una base tan mala como un DER equivocado.

Errores frecuentes — al pasar del DER a las tablas

1. Poner la FK del lado equivocado. En una 1:N, la clave del «uno» viaja al «muchos». Si guardáramos el código de venta dentro de Clientes, cada cliente podría tener una sola venta.

2. Meter una lista en una celda. Resolver la N:M con un campo «productos» que diga «P01, P06» en Ventas viola la 1FN y no permite guardar cantidad ni precio de cada producto.

3. Olvidar los atributos del rombo. Si DetalleVenta queda solo con (venta, producto), se pierde cuántas unidades se vendieron y a qué precio.

4. Clave de la tabla puente sin control. Si DetalleVenta usa solo un campo Autonumeración como clave y nadie controla que la pareja venta + producto no se repita, el mismo producto puede cargarse dos veces en la misma venta. La clave compuesta lo impide por diseño.

Actividades de aplicación

Ejercitá:

1. Indicá en qué se convierte cada entidad al pasar a tablas.
2. Explicá cómo se resuelve una relación 1:N.
3. Explicá cómo se resuelve una relación N:M.
4. Nombrá las cuatro tablas del esquema final del Copetín.

Resolvé (caso Copetín Karumbé):

5. Traducí la relación «Cliente realiza Venta» (1:N) indicando qué clave foránea aparece y en qué tabla.
6. Una venta nueva, que el sistema numerará V12, lleva 3 empanadas y 1 cocido a los precios del catálogo. Escribí sus filas de DetalleVenta (venta, producto, cantidad, precio_unitario) y calculá el total.
7. Cada caja registradora del Copetín la usa un solo cajero y cada cajero usa una sola caja. Traducí esa 1:1 indicando dónde va la clave foránea y qué propiedad evita que se convierta en 1:N.

Pensá y decidí (justificá tu respuesta):

8. «Una relación N:M se puede guardar sin tabla intermedia.»
9. «En una relación 1:N, la clave del lado ‘uno’ va como foránea en el lado ‘muchos’.»

Desafío:

10. Traducí a tablas el DER de la biblioteca (Socio, Libro, Préstamo) que dibujaste, indicando claves principales y foráneas.

Práctica 15 — Del DER de los talleres a las tablas

Nombre y Apellido: _____ Curso: _____ Fecha: ____ / ____ / ____

Competencia de la práctica

Aplicar las reglas de transformación a un DER nuevo: entidades, 1:N, N:M con tabla puente y 1:1 con clave foránea única.
Entorno de trabajo: papel y lápiz.

Lo que necesitás saber

Entidad → tabla. 1:N → la clave del «uno» viaja como FK al «muchos». N:M → tabla puente con las dos FK y los atributos del rombo. 1:1 → FK sin duplicados en el lado obligatorio, o fusión.

Usá el DER validado de la Práctica 14.

Actividad 1 — El esquema de tablas

Objetivo: *Traducir el DER a tablas con sus claves.*

Modelo — Formato de cada tabla

NombreTabla(campo PK, campo, campo FK → Tabla, …)

Pasos:

1. Escribí una tabla por entidad.
2. Resolvé «dicta» (1:N) con una FK.
3. Resolvé «se inscribe» (N:M) con una tabla puente; indicá su clave.
4. Resolvé «usa» (1:1) e indicá la propiedad que la mantiene 1:1.

Punto de control: La cantidad de tablas es igual a la cantidad de entidades más la de relaciones N:M.

Actividad 2 — Cargar la tabla puente

Objetivo: *Escribir las filas de la tabla puente para tres inscripciones.*

Modelo — Inscripciones

Lucía Ortiz (A01) en Robótica (T1), el 02/03/2026
Lucía Ortiz (A01) en Teatro (T2), el 03/03/2026
Marcos Rojas (A02) en Robótica (T1), el 02/03/2026

Pasos:

1. Escribí una fila por inscripción, con códigos y fecha.
2. Verificá que la clave compuesta no se repita.
3. Intentá cargar de nuevo a Lucía en Robótica y explicá qué pasa.

Punto de control: Ninguna fila repite la pareja alumno + taller.

Desafío final (para quienes terminan antes)

Si un alumno puede volver a inscribirse en el mismo taller al año siguiente, ¿qué le pasa a la clave de la tabla puente? Proponé el cambio.

Clase 16 — Por qué normalizar: anomalías y dependencia funcional

Capacidad	Utiliza las reglas de la normalización (1FN- 2FN- 3FN) para la obtención de datos agrupados en diferentes entidades.
Tema	Anomalías de una tabla mal diseñada. Dependencia funcional.
Indicadores de logro	• Reconoce las anomalías de inserción, actualización y borrado. • Explica qué es una dependencia funcional. • Justifica por qué conviene normalizar.

Una tabla que junta todo

Imaginá que el Copetín guardara todo en una sola tabla gigante, repitiendo en cada renglón el nombre y el teléfono del cliente y el nombre, la categoría y el precio del producto. Esa tabla trae problemas llamados anomalías.

Anomalía	Qué pasa
De inserción	No se puede cargar un producto nuevo hasta que alguien lo compre
De actualización	Cambiar el precio obliga a corregir muchos renglones
De borrado	Al borrar la única venta de un producto o de un cliente, se pierden también sus datos

Dependencia funcional

Una dependencia funcional ocurre cuando un campo determina a otro: el código de producto determina su nombre, su categoría y su precio. Se escribe código → nombre, categoría, precio. Entender qué depende de qué es la clave para normalizar.

Ejemplo — la anomalía de actualización en el Copetín

Si la tabla gigante repite «Gaseosa, Bebidas, 8.000» en cada venta y la gaseosa sube a 9.000, hay que cambiar decenas de renglones; si se olvida alguno, quedan precios distintos. Separar el producto en su tabla elimina el problema.

Concepto clave — normalizar

Organizar los datos en tablas bien separadas para eliminar la redundancia y las anomalías, guiándose por las dependencias funcionales.

Concepto clave — dependencia funcional

Un campo A determina un campo B (se escribe A → B) si a cada valor de A le corresponde siempre un único valor de B. Ejemplo: código de producto → precio de catálogo.

Las tres anomalías con nombre propio

Las fallas de una tabla mal diseñada tienen nombre y apellido. La anomalía de inserción impide cargar un dato porque falta otro que no corresponde. La de actualización obliga a corregir el mismo dato en muchos lugares. La de borrado hace que, al eliminar un registro, se pierda información que no se quería perder. Las tres nacen de la misma raíz: datos que no dependen de la clave conviven en la misma tabla.

Anomalía	Qué pasa en la tabla única del Copetín	Consecuencia
Inserción	No se puede cargar el producto nuevo «jugo» hasta que alguien lo compre	Datos rehenes de otros datos
Actualización	Si la gaseosa sube de precio, hay que corregirla en cada renglón donde aparece	Riesgo de precios contradictorios
Borrado	Si se anula la única venta de Ana (V4), se pierde también a la clienta Ana	Información destruida por accidente

Ejemplo — la anomalía de actualización, con números

La gaseosa (P06) aparece en tres ventas del Copetín: V1, V3 y V5, con 5 unidades en total. En la tabla única, su precio figura repetido en cada renglón. Un aumento a G. 9.000 exigiría corregir todos; si uno se escapa, el mismo producto tendría dos precios a la vez — y ninguna consulta sería confiable.

Antes de leer el ejemplo, recorré la figura: cada flecha sale de lo que determina y llega a lo que queda determinado. Las flechas verdes nacen de la clave completa (venta + producto); las ámbar, de una sola parte de la clave, y la roja pasa por un campo que no es clave.

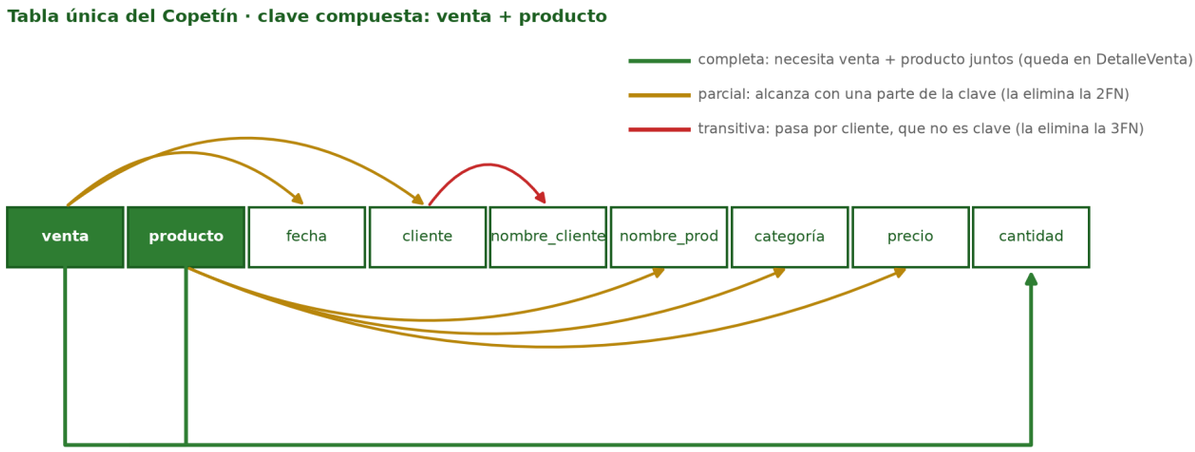


Figura 16.1 — Diagrama de dependencias funcionales de la tabla única del Copetín.

Ejemplo resuelto — detectar dependencias funcionales en la tabla única

La herramienta para diagnosticar la raíz del problema es la dependencia funcional: qué campos quedan determinados por cuáles. Analicemos la tabla única del Copetín (venta, fecha, cliente, nombre del cliente, producto, nombre del producto, categoría, precio, cantidad).

1. El código de venta determina la fecha y el cliente: sabiendo que es V3, la fecha es 04/03/2026 y el cliente, C01. Se escribe: venta → fecha, cliente.
2. El código de cliente determina su nombre: C01 → Rosa Duarte. Se escribe: cliente → nombre del cliente.
3. El código de producto determina su nombre, categoría y precio: P06 → Gaseosa, Bebidas, 8.000. Se escribe: producto → nombre, categoría, precio.
4. La cantidad NO depende de la venta sola (V1 tiene dos cantidades distintas) ni del producto solo (la gaseosa se vendió de a 1 y de a 2): depende de la combinación. Se escribe: venta + producto → cantidad.
5. Diagnóstico: en la tabla única conviven cuatro «dueños» de datos distintos (venta, cliente, producto y la pareja venta+producto). Cada dependencia funcional está pidiendo su propia tabla — exactamente las cuatro del esquema del Copetín.

> **Concepto clave — la dependencia funcional es el mapa**
> Detectar las dependencias funcionales no es un trámite teórico: es el mapa que indica cómo partir la tabla. En la próxima clase, las formas normales van a seguir ese mapa paso a paso.

El ensayo de operaciones: anomalías antes de que duelan

Hay una técnica simple para diagnosticar una tabla sospechosa: ensayar mentalmente las operaciones de todos los días — insertar, actualizar, borrar — y observar qué se rompe. Cada operación que falla o exige malabares delata una anomalía concreta, y cada anomalía delata una dependencia funcional mal ubicada.

Operación ensayada sobre la tabla única	Qué se rompe	Anomalía delatada
Cargar el producto nuevo «jugo» sin ventas	No hay renglón donde ponerlo	Inserción
Subir el precio de la gaseosa	Corregir en V1, V3 y V5 a la vez	Actualización
Anular la venta V4 (la única de Ana)	Ana desaparece del sistema	Borrado
Registrar un cliente nuevo sin compras	Igual que el jugo: sin renglón, no entra	Inserción

La redundancia: medirla para entenderla

La redundancia no es una abstracción: se cuenta. En la tabla única del Copetín, el precio de la gaseosa (G. 8.000) figura en tres renglones (V1, V3 y V5); el nombre de Rosa, en todos los renglones de sus dos ventas. Con cinco ventas el desperdicio es anecdótico; con cinco mil, cada dato repetido multiplica el espacio, el tiempo de corrección y — lo grave — la probabilidad de que alguna copia quede desactualizada.

Ejemplo — la cuenta de la redundancia

Se cuenta columna por columna, comparando cuántos renglones hay con cuántos valores distintos del «dueño» del dato existen. La fecha y el cliente dependen de la venta: 11 renglones para 5 ventas, 6 celdas repetidas en cada columna (12). El nombre del cliente depende del cliente: 11 renglones para 4 clientes, 7 repetidas. Nombre, categoría y precio dependen del producto: 11 renglones para 6 productos, 5 repetidas en cada columna (15).

Total: 12 + 7 + 15 = 34 celdas que repiten un dato ya escrito en otro renglón, contra 0 en el esquema normalizado, donde cada dato vive una sola vez y las claves hacen el resto.

Ejemplo resuelto — del síntoma a la dependencia

Cerremos el diagnóstico conectando cada síntoma del ensayo con la dependencia funcional que lo causa.

1. Síntoma «el jugo no entra sin venta»: los datos del producto dependen de producto (producto → nombre, categoría, precio), pero la tabla los encadena a la clave venta + producto. Dato preso de una clave que no le corresponde.

2. Síntoma «corregir el precio de catálogo de la gaseosa en tres lugares»: misma causa — ese precio de catálogo depende SOLO del producto y no debe repetirse por cada venta. Esto no se aplica a precio_unitario: ese campo registra lo realmente cobrado en una transacción concreta.

3. Síntoma «Ana desaparece con V4»: los datos del cliente dependen de cliente (cliente → nombre), no de la venta; al borrar la venta se llevan puesto lo que no era suyo.

4. Diagnóstico unificado: todos los síntomas son la MISMA enfermedad — dependencias parciales y transitivas conviviendo bajo una sola clave. El tratamiento sistemático es la próxima clase: las formas normales.

Concepto clave — la anomalía es el síntoma, la dependencia es la causa

Memorizar las tres anomalías sirve para el diagnóstico; entender las dependencias funcionales sirve para la cura. En la evaluación — y en la vida profesional — lo segundo vale el doble.

La dependencia parcial, con lupa

Cuando la clave es compuesta aparece un riesgo específico: la dependencia parcial — un atributo que depende de UNA PARTE de la clave, no de la clave entera. Es exactamente la enfermedad de la tabla única del Copetín, y verla con lupa deja la segunda forma normal a un paso.

1. La clave de un renglón es compuesta: (venta, producto).

2. La cantidad depende de la clave COMPLETA: para saber «cuántas gaseosas» hay que decir en qué venta (V1 llevó 1; V3 llevó 2). Dependencia total: la cantidad está bien donde está.

3. El precio de catálogo depende SOLO del producto y por eso no debe quedar en una tabla cuya clave es venta+producto: sería una dependencia parcial. En cambio, precio_unitario puede variar entre ventas y representa el importe cobrado en ese renglón; por eso depende de la clave completa venta+producto y sí pertenece a DetalleVenta.

4. El nombre y la categoría del producto: ídem — parciales.

5. La cura de la 2FN es literal: lo que depende sólo del producto —nombre, categoría y precio de catálogo— se muda a Productos. En el renglón de DetalleVenta quedan los hechos que dependen del par completo: cantidad y precio_unitario.

Concepto clave — la pregunta de la parcialidad

Ante clave compuesta, interrogá a cada atributo: «¿te alcanza con una parte de la clave?». Si la respuesta es sí, hay dependencia parcial y la 2FN exige mudanza. Si todos los atributos necesitan la clave entera, la tabla ya respira en segunda forma normal.

Dependencia funcional: regla, no coincidencia

Una dependencia funcional describe una regla del negocio, no una casualidad de los datos de hoy. En el Copetín, los seis precios son distintos (3.000, 5.000, 6.000, 15.000, 4.000 y 8.000), así que en esta tabla cada precio aparece con un solo producto. ¿Vale entonces «precio → producto»? No: mañana el jugo puede costar G. 6.000, igual que la empanada, y la supuesta regla se rompe. En cambio, «producto → precio de catálogo» vale siempre, porque el negocio decide un único precio de lista por producto.

El programa usa también el nombre dependencia funcional compuesta o completa para la que necesita toda una clave compuesta, como venta + producto → cantidad. Es la que la segunda forma normal quiere conservar; la parcial, la que quiere eliminar.

Errores frecuentes — al escribir dependencias funcionales

1. Leer la flecha al revés: «precio → producto» no es lo mismo que «producto → precio».
2. Tomar una coincidencia de los datos por una regla: que hoy no haya dos precios iguales no crea una dependencia.
3. Olvidar las dependencias de la clave compuesta: la cantidad no depende de la venta ni del producto por separado, sino de los dos juntos.
4. Confundir dependencia con relación entre tablas: la DF es entre campos y se usa para decidir en qué tabla va cada uno.

Actividades de aplicación

Ejercitá:

1. Nombrá las tres anomalías y explicá una.
2. Definí dependencia funcional.
3. Escribí la dependencia funcional entre código de producto y su precio de catálogo.
4. Indicá qué problema evita separar los productos en su propia tabla.

Resolvé (caso Copetín Karumbé):

5. En la tabla única, Luis Benítez tiene una sola venta (V2). Indicá qué anomalía aparece si se anula V2 y qué dato se pierde.
6. Escribí dos dependencias funcionales que existan en los datos del Copetín.
7. ¿Es una dependencia funcional «categoría → precio»? Justificá con los datos de Productos.
8. Clasificá como completa, parcial o transitiva, en la tabla única: (a) venta + producto → cantidad; (b) producto → categoría; (c) venta → nombre del cliente.

Pensá y decidí (justificá tu respuesta):

9. «Normalizar busca aumentar la redundancia de los datos.»
10. «El código de producto determina su precio de catálogo: es una dependencia funcional.»

Desafío:

11. Inventá una tabla mal diseñada de tu colegio (que junte todo) y señalá una anomalía de inserción, una de actualización y una de borrado.

Práctica 16 — Anomalías y dependencias en la planilla de talleres

Nombre y Apellido: _____ Curso: _____ Fecha: ____ / ____ / ____

Competencia de la práctica

Detectar anomalías y escribir y clasificar dependencias funcionales en una tabla nueva, y medir su redundancia.

Entorno de trabajo: papel y lápiz.

Lo que necesitás saber

Dependencia completa: necesita toda la clave. Parcial: alcanza con una parte. Transitiva: pasa por un campo que no es clave. Una DF es una regla, no una coincidencia de los datos.

La coordinación guarda las inscripciones en una sola planilla. Su clave es (cod_alumno, cod_taller).

Actividad 1 — Ensayar operaciones

Objetivo: *Encontrar una anomalía de cada tipo.*

Modelo — Planilla de inscripciones

A01 · Lucía Ortiz · 3.º A · T1 · Robótica · Prof. Ruiz · 0981 222 333 · 02/03/2026
A01 · Lucía Ortiz · 3.º A · T2 · Teatro · Prof. Benítez · 0982 444 555 · 03/03/2026
A02 · Marcos Rojas · 2.º B · T1 · Robótica · Prof. Ruiz · 0981 222 333 · 02/03/2026
A03 · Ana Paula Vera · 1.º C · T3 · Ajedrez · Prof. Ruiz · 0981 222 333 · 04/03/2026
A02 · Marcos Rojas · 2.º B · T3 · Ajedrez · Prof. Ruiz · 0981 222 333 · 05/03/2026
A04 · Diego Acosta · 3.º A · T2 · Teatro · Prof. Benítez · 0982 444 555 · 05/03/2026
Columnas: cod_alumno · nombre_alumno · curso · cod_taller · nombre_taller · docente · tel_docente · fecha_insc

Pasos:

1. Ensayá: crear el taller T4 «Coro» sin inscriptos; cambiar el teléfono del Prof. Ruiz; dar de baja la única inscripción de Ana Paula Vera.
2. Para cada ensayo, escribí qué se rompe y qué anomalía es.

Punto de control: Para cada anomalía señalaste en qué filas de la planilla ocurre.

Actividad 2 — El mapa de dependencias

Objetivo: *Escribir las dependencias funcionales, clasificarlas y medir la redundancia.*

Modelo — Modelo de diagrama

Cajas con las ocho columnas en fila; flechas desde lo que determina hacia lo determinado, como en la figura de dependencias de la clase.

Pasos:

1. Escribí todas las DF que valen como regla del colegio.
2. Clasificá cada una como completa, parcial o transitiva.
3. Dibujá el diagrama con tres colores.
4. Contá, columna por columna, las celdas que repiten un dato ya escrito en otra fila.

Punto de control: Toda DF que escribiste se cumple en las seis filas y tiene sentido como regla del colegio.

Desafío final (para quienes terminan antes)

¿Es «docente → cod_taller» una dependencia funcional? Justificá con los datos.

Clase 17 — Formas normales: 1FN, 2FN y 3FN

Capacidad	Utiliza las reglas de la normalización (1FN- 2FN- 3FN) para la obtención de datos agrupados en diferentes entidades.
Tema	Primera, segunda y tercera forma normal.
Indicadores de logro	• Aplica la 1FN (atomicidad, sin grupos repetidos). • Aplica la 2FN (sin dependencias parciales). • Aplica la 3FN (sin dependencias transitivas).

Primera forma normal (1FN)

Una tabla está en 1FN si cada campo tiene un solo valor (es atómico) y no hay grupos repetidos. En vez de guardar «2 chipas, 1 gaseosa» en un solo campo, se pone una fila por producto de la venta.

Segunda forma normal (2FN)

Una tabla está en 2FN si, estando en 1FN, ningún campo no clave depende sólo de una parte de una clave compuesta. En el caso del Copetín, el precio de catálogo depende sólo del producto y se ubica en Productos. El precio_unitario de una venta es otro hecho: puede variar por operación y depende de la pareja venta+producto, por lo que permanece en DetalleVenta.

Tercera forma normal (3FN)

Una tabla está en 3FN si, estando en 2FN, ningún campo depende de otro campo que no es la clave (dependencia transitiva). Si en Ventas guardáramos el teléfono del cliente, dependería del cliente, no de la venta: por eso el teléfono se mueve a la tabla Clientes.

El recorrido completo se ve en la figura: de la tabla única llena de redundancia se avanza, regla por regla, hasta el esquema en 3FN donde cada dato vive una sola vez.

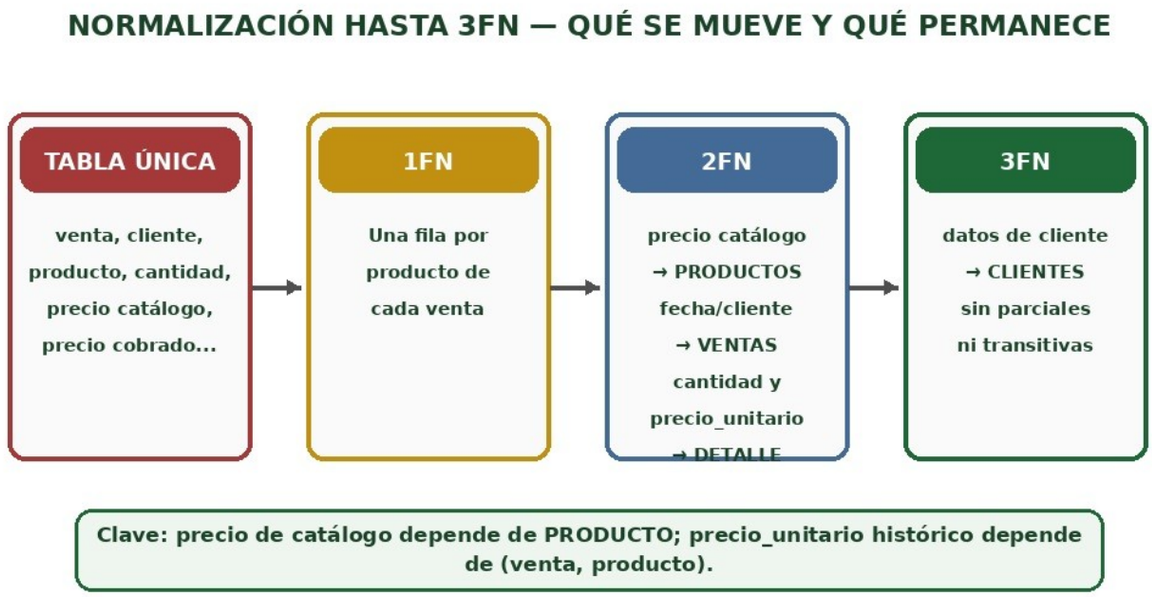


Figura 17.1 — Normalización hasta 3FN: el precio de catálogo va a Productos y los hechos de cada renglón permanecen en DetalleVenta.

Forma normal	Regla	Qué se hizo en el Copetín
1FN	Valores atómicos, sin grupos repetidos	Una fila por producto de la venta
2FN	Sin dependencias parciales de la clave	Precio y nombre → tabla Productos
3FN	Sin dependencias transitivas	Teléfono del cliente → tabla Clientes

Ejemplo — el Copetín, normalizado

Al aplicar las tres formas normales, la tabla gigante se separa en Productos, Clientes, Ventas y DetalleVenta. El precio de catálogo queda en Productos; cantidad y precio_unitario histórico quedan en DetalleVenta porque dependen del renglón venta-producto. Diseño y normalización coinciden.

Concepto clave — las tres formas normales

1FN: valores atómicos. 2FN: sin dependencias parciales. 3FN: sin dependencias transitivas. Cada una se apoya en la anterior.

Errores frecuentes — al normalizar

1. Buscar dependencias parciales donde no puede haberlas. Si la clave tiene un solo campo, no existe «una parte» de la clave: una tabla en 1FN con clave simple ya está en 2FN.
2. Sacar precio_unitario de DetalleVenta «porque el precio va en Productos». El precio de catálogo va en Productos; lo cobrado en cada venta es otro hecho y se queda en el renglón.
3. Creer que 1FN es «separar todo en partes». Atomicidad es un valor por celda para lo que el negocio usa como unidad: dividir el nombre en nombre y apellido es una decisión de diseño, no una exigencia de la 1FN.
4. Saltar pasos. Aplicar la 3FN sin haber resuelto la 2FN deja dependencias parciales escondidas.
5. Perder información al partir. Toda separación debe dejar una clave para volver a unir los datos: si desde las tablas no se reconstruyen los G. 134.000, la normalización está mal hecha.

En la práctica escolar y en muchos sistemas reales, llegar a la 3FN es suficiente: elimina la mayor parte de la redundancia sin complicar el diseño.

Dependencias transitivas: el corazón de la 3FN

Una dependencia transitiva es una cadena: un campo depende de la clave a través de otro campo intermedio. Imaginemos que la tabla Ventas guardara también el barrio del cliente: la venta determina al cliente y el cliente determina su barrio, así que el barrio depende de la venta «de rebote» (venta → cliente → barrio). La 3FN corta esas cadenas: el barrio pertenece a la tabla Clientes y a ninguna otra.

Ejemplo — la cadena que la 3FN corta

Si Ventas guardara el barrio, mudar a Rosa exigiría corregir el barrio en TODAS sus ventas (V1 y V3) además de su ficha — la anomalía de actualización otra vez, ahora por transitividad. Con 3FN, el barrio vive una sola vez en Clientes y la mudanza se registra una sola vez.

Ejemplo resuelto — normalizar la tabla única, forma por forma

Apliquemos el proceso completo a la tabla única del Copetín, verificando en cada forma qué problema se resuelve y comprobando al final que no se perdió ni un guaraní.

1. Punto de partida: la tabla única con renglones como «V1, 03/03/2026, C01, Rosa Duarte, chipa y gaseosa, precios, cantidades».
2. 1FN — valores atómicos: separar «chipa y gaseosa» en filas independientes, una por producto. La tabla crece a once filas (una por cada par venta-producto), todas con valores simples.
3. 2FN — dependencias completas de la clave: con clave venta + producto, nombre, categoría y precio de catálogo dependen sólo del producto, mientras fecha y cliente dependen sólo de la venta. Se separan Productos (producto, nombre, categoría, precio) y Ventas (venta, fecha, cliente, nombre del cliente); DetalleVenta conserva venta, producto, cantidad y precio_unitario, porque estos dos últimos describen el renglón de esa venta. Observá que el nombre del cliente viajó a Ventas junto con el cliente: depende de la venta, pero «de rebote».
4. 3FN — sin transitividades: en Ventas, el nombre del cliente depende de la venta a través del código de cliente (venta → cliente → nombre). Se separa: nace Clientes (C01 a C04) y en Ventas queda solo el código del cliente como clave foránea. El esquema queda en cuatro tablas.
5. Verificación de integridad del proceso: reconstruyendo cada venta desde las cuatro tablas (cantidad × precio_unitario, sumado por venta) se obtiene 14.000, 19.000, 34.000, 17.000 y 50.000 — total G. 134.000, idéntico al de la tabla original. Normalizar reorganizó los datos sin perder ni inventar nada.

Forma normal	Regla que impone	Qué nació en el Copetín
1FN	Valores atómicos, sin listas en una celda	Una fila por cada par venta-producto
2FN	Nada depende de una parte de la clave	Las tablas Productos y Ventas
3FN	Nada depende de la clave «de rebote»	La tabla Clientes

¿Hasta dónde normalizar? El equilibrio en la práctica

Existen formas normales más allá de la tercera, pero en la práctica profesional la 3FN es el punto de equilibrio habitual: elimina las anomalías importantes sin fragmentar los datos en tablas innecesarias. Fragmentar de más también tiene costo: cada consulta debe volver a unir las piezas. La regla práctica del diseñador: normalizar hasta 3FN y desviarse solo con una razón medida y documentada.

Desnormalizar: romper las reglas con permiso

En sistemas muy grandes, a veces se decide desnormalizar: reintroducir redundancia controlada para acelerar consultas críticas (guardar el total de la venta calculado, por ejemplo, para no recomputarlo en cada informe). Es una decisión de ingeniería avanzada, con reglas estrictas: se documenta qué se duplicó, por qué, y qué mecanismo

mantiene las copias sincronizadas. Sin ese permiso escrito y ese mecanismo, no es desnormalización: es simplemente una base mal hecha.

Decisión	Cuándo se justifica	Riesgo que se asume
Normalizar hasta 3FN	Siempre, como punto de partida	Consultas con más uniones de tablas
Desnormalizar un dato puntual	Consulta crítica y medida como lenta	Mantener las copias sincronizadas
No normalizar «para ahorrar trabajo»	Nunca	Todas las anomalías, sin control

Concepto clave — primero normalizar, después optimizar
El orden profesional es innegociable: se diseña normalizado, se mide el rendimiento real y — SOLO si una consulta concreta lo exige — se desnormaliza ese punto, documentado. Desnormalizar «por las dudas», antes de medir, es optimizar un problema que no existe al precio de crear varios que sí.

La normalización como red de seguridad del DER

Quizás notaste algo elegante: el esquema normalizado del Copetín (Productos, Clientes, Ventas, DetalleVenta) es EXACTAMENTE el que había salido del DER en la Clase 15. No es casualidad: un DER bien hecho produce tablas ya normalizadas, porque ubicar cada atributo en su entidad correcta ES respetar las dependencias funcionales. La normalización funciona entonces como red de seguridad: verifica formalmente lo que el buen diseño ya hizo — y atrapa lo que se escapó.

Ejemplo resuelto — decidir en tres escenarios

1. Escenario 1: el informe mensual del Copetín tarda un instante en calcular los totales desde DetalleVenta. ¿Desnormalizar guardando el total? No: no hay problema de rendimiento que justifique el riesgo. Se queda en 3FN.
2. Escenario 2: un supermercado con millones de renglones mide que su informe diario tarda horas por recalcular totales. ¿Desnormalizar? Sí: se guarda el total por venta, con un mecanismo automático que lo recalcula ante cualquier corrección de renglones, y la decisión se documenta.
3. Escenario 3: un diseñador propone «dejar el precio también en DetalleVenta para escribir menos uniones». ¿Se acepta? No: no midió nada, no documentó nada y reintroduce la anomalía de actualización que costó tres clases eliminar.
4. Regla extraída: la 3FN se abandona por evidencia medida y con red de sincronización — jamás por comodidad.

Aplicación profesional — histórico de precios: un hecho distinto
Productos.precio expresa el precio de catálogo actual. DetalleVenta.precio_unitario expresa lo que realmente se cobró en una operación concreta. Aunque ambos tengan valores iguales al inicio, representan hechos distintos: actualizar el catálogo no debe reescribir ventas pasadas. Guardar un total derivado adicional sí sería una desnormalización y debería justificarse y controlarse.

Actividades de aplicación

Ejercitá:

1. Enunciá la regla de la 1FN.
2. Enunciá la regla de la 2FN.
3. Enunciá la regla de la 3FN.
4. Indicá qué se movió a la tabla Productos y qué a la tabla Clientes al normalizar.

Resolvé (caso Copetín Karumbé):

5. La librería Arasa'i anota «3 cuadernos, 1 regla» en un solo campo del pedido N.º 12. Llevá ese renglón a 1FN.
6. Explicá por qué el precio de catálogo violaría la 2FN si se guardara en DetalleVenta, y por qué precio_unitario sí puede quedarse ahí.
7. Una tabla Clientes(código, nombre, barrio, ciudad_del_barrio) tiene clave simple. ¿Está en 2FN? ¿Y en 3FN? Justificá.

Pensá y decidí (justificá tu respuesta):

8. «La 3FN se puede aplicar sin haber pasado por la 1FN y la 2FN.»
9. «Al normalizar el Copetín se llega a las cuatro tablas del esquema final.»

Desafío:

10. Tomá la tabla mal diseñada que inventaste en la clase anterior y llevala hasta la 3FN, mostrando las tablas resultantes.

Práctica 17 — Normalizar los pedidos de la librería Arasa'i

Nombre y Apellido: _____ Curso: _____ Fecha: ____ / ____ / ____

Competencia de la práctica

Llevar una planilla nueva de 1FN a 3FN sin perder información, verificando la reconstrucción de los importes.

Entorno de trabajo: papel y lápiz.

Lo que necesitás saber

1FN: un valor por celda. 2FN: sin dependencias parciales. 3FN: sin dependencias transitivas. Normalizar bien no pierde ni inventa datos.

Precios de la librería: cuaderno (R1) G. 8.000 · lapicera (R2) G. 2.500 · mochila (R3) G. 95.000.

Actividad 1 — Primera forma normal

Objetivo: *Separar la lista de productos de cada pedido.*

Modelo — Planilla de pedidos

31 · 06/03/2026 · K01 · Ramona Giménez · 0984 111 222 · 2 cuadernos; 1 lapicera

32 · 06/03/2026 · K02 · Julio Sanabria · 0991 333 444 · 1 mochila; 2 lapiceras

33 · 07/03/2026 · K01 · Ramona Giménez · 0984 111 222 · 1 cuaderno; 1 mochila

Columnas: pedido · fecha · cod_cliente · cliente · teléfono · productos

Pasos:

1. Calculá primero el importe de cada pedido con la planilla original y anotalo.
2. Reescribí la planilla con una fila por producto de cada pedido (con cantidad y precio_unitario).
3. Indicá la clave de la tabla en 1FN.

Punto de control: Ninguna celda tiene más de un producto y la clave identifica cada fila.

Actividad 2 — Segunda y tercera forma normal

Objetivo: *Eliminar dependencias parciales y transitivas.*

Modelo — Preguntas guía

¿Qué datos dependen solo del pedido?

¿Cuáles solo del producto?

¿Hay algún dato que dependa del pedido a través de otro campo?

Pasos:

1. Aplicá la 2FN y escribí las tablas resultantes.
2. Aplicá la 3FN y escribí el esquema final con claves principales y foráneas.
3. Reconstruí desde tus tablas finales el importe de cada pedido.

Punto de control: La suma de los importes reconstruidos desde tus tablas finales es igual a la suma de los importes que calculaste con la planilla original.

Desafío final (para quienes terminan antes)

La librería quiere hacer descuentos: a veces sobre un producto de un pedido, a veces sobre el pedido entero. ¿En qué tabla va cada descuento?

Evaluación de la Unidad 3

Diseño: Entidad-Relación y normalización

Nombre y Apellido: _____ Curso: _____ Fecha: ____ / ____ / ____

Caso de la evaluación — la veterinaria Jaguarete

La veterinaria registra a los dueños (cédula, nombre y apellido, teléfonos), a sus mascotas (código, nombre, especie, fecha de nacimiento, edad) y las consultas (número, fecha, diagnóstico, monto). Cada mascota tiene un solo dueño y un dueño puede tener varias mascotas; cada consulta es de una sola mascota.

Parte A — Diagnóstico rápido (marcá la opción correcta):

1. La relación entre DUEÑO y MASCOTA del caso es:
   a) N:M
   b) 1:N
   c) 1:1

2. Una relación N:M se resuelve con:
   a) una tabla intermedia
   b) una vista
   c) un filtro

3. Que cada campo tenga un solo valor (atómico) corresponde a la:
   a) 3FN
   b) 2FN
   c) 1FN

Parte B — Resolución y aplicación:

4. Identificá las entidades del caso y clasificá estos atributos: teléfonos del dueño, edad de la mascota y nombre y apellido del dueño.

5. Determiná la cardinalidad entre MASCOTA y CONSULTA y escribí el par (mín, máx) de cada extremo.

6. Dibujá el DER de la veterinaria con sus entidades, atributos (identificador subrayado) y relaciones con cardinalidad.

7. En cada consulta se pueden recetar varios medicamentos y un medicamento se receta en muchas consultas, con una dosis. Traducí esa relación a tablas: tabla puente, clave y atributos.

8. La planilla Consultas(número, fecha, cod_mascota, nombre_mascota, especie, cédula_dueño, nombre_dueño, monto) tiene clave simple. ¿Está en 2FN? ¿Qué dependencias transitivas tiene? Llevala a 3FN.

UNIDAD 4 — La base en acción: Access y consultas

En esta unidad la base cobra vida: la vas a crear en Microsoft Access con sus relaciones y la vas a consultar con filtros, criterios, parámetros, totales y referencias cruzadas, hasta convertir los datos en decisiones.

Clase 18 — Crear la base en Access: tablas y relaciones

Capacidad	Ejecuta técnicas, procedimientos y normativas en el desarrollo de entidades normalizadas.
Tema	Crear tablas, claves y relaciones en Microsoft Access.
Indicadores de logro	• Crea una tabla con sus campos y clave principal en Access. • Define una relación entre tablas con clave foránea. • Activa la integridad referencial.

Del esquema a Access

Ya tenemos el esquema del Copetín: Productos, Clientes, Ventas y DetalleVenta. Ahora lo creamos en Microsoft Access, el gestor que trabajaremos. En Access, cada tabla se diseña indicando sus campos, el tipo de dato y la clave principal. Los procedimientos de este libro toman como referencia Microsoft Access para Windows en castellano. La navegación se contrastó históricamente en Microsoft Access de Office 2016, versión fuera de soporte desde el 14 de octubre de 2025. Para el trabajo escolar de 2026 se recomienda utilizar Access incluido en Microsoft 365 o Access 2024 con licencia válida y soporte vigente. Los nombres de comandos pueden variar según la versión; el docente verificará las rutas en su instalación.

Crear una tabla y su clave principal

Para crear la tabla Productos: en la pestaña Crear se elige Diseño de tabla; se escriben los campos (código, nombre, categoría, precio), se elige el tipo de cada uno y se marca el campo código como clave principal con el botón Clave principal. En DetalleVenta la clave es compuesta: se seleccionan las dos filas venta y producto con los selectores de fila de la izquierda (manteniendo presionada la tecla Ctrl) y se pulsa una sola vez Clave principal; aparecen dos iconos de clave.

Los tipos de datos en Access

Al diseñar cada campo se elige su tipo de dato, y elegir bien evita errores de carga. Los más usados: Texto corto (nombres y códigos), Número (cantidades), Moneda (precios e importes en guaraníes), Fecha y hora (fechas de venta) y Sí/No (casillas de verificación). El código conviene como Texto corto aunque tenga números, porque no se usa para calcular.

Campo del Copetín	Tipo de dato en Access	Por qué
código (P01, C01…)	Texto corto	Identifica; no se calcula con él
nombre, categoría	Texto corto	Palabras
precio / precio_unitario	Moneda	Catálogo actual / importe histórico cobrado
fecha	Fecha y hora	Permite filtrar por día
cantidad	Número	Se usa en cálculos (subtotal)

Definir relaciones e integridad referencial

Con las tablas creadas, en la pestaña Herramientas de base de datos se abre Relaciones, se arrastran las tablas y se une la clave principal de una con la clave foránea de otra. Al crear la relación se activa la casilla Exigir integridad referencial. Debajo aparecen otras dos casillas: Actualizar en cascada los campos relacionados (recomendable) y Eliminar en cascada los registros relacionados (que conviene dejar desmarcada, por lo visto en la Clase 9).

Así queda la ventana Relaciones del Copetín: cada tabla con su lista de campos (la clave principal identificada con el icono/indicador de clave) y las líneas 1–∞ que unen cada clave principal con su clave foránea.

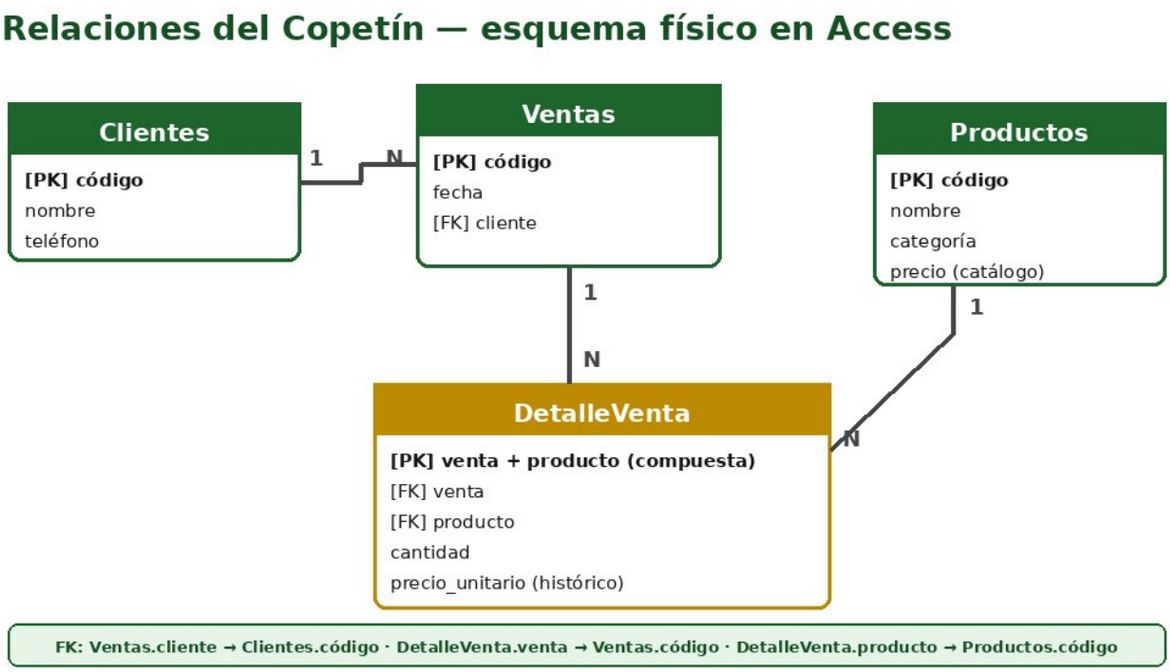


Figura 18.1 — El esquema del Copetín en la ventana Relaciones de Access.

Tabla	Clave principal	Clave foránea (hacia)
Productos	código	—
Clientes	código	—
Ventas	código	cliente (→ Clientes)
DetalleVenta	venta + producto (compuesta)	venta (→ Ventas), producto (→ Productos)

Ejemplo — crear la relación Ventas–Clientes

En la ventana Relaciones se arrastra el campo código de Clientes sobre el campo cliente de Ventas. Access muestra la línea 1:N y, al marcar «Exigir integridad referencial», impide registrar ventas de clientes inexistentes.

Concepto clave — diseñar es trabajo DDL

Crear tablas, definir claves y relaciones es trabajo de definición de datos (DDL). Access te lo facilita con menús, pero por debajo estás definiendo la estructura de la base.

En Access, los cambios de datos se guardan de manera distinta a los cambios de diseño. Acostumbrate a guardar explícitamente el diseño de cada tabla con un nombre claro (Productos, Clientes) y a atender cualquier aviso de guardado al cerrar un objeto modificado.

La vista Diseño y la vista Hoja de datos

Access ofrece dos ventanas sobre la misma tabla. La vista Diseño muestra el molde: los campos, sus tipos de datos y sus propiedades — es donde trabaja el diseñador. La vista Hoja de datos muestra el contenido: los registros en filas, listos para cargar y leer — es donde trabaja el usuario. Alternar entre ambas (botón Ver, en la pestaña Inicio) es el movimiento más repetido al construir una base.

Concepto clave — molde a la izquierda, datos a la derecha

La distinción DDL/DML de la Unidad 2 vive en estas dos vistas: la vista Diseño es el territorio de la definición; la Hoja de datos, el de la manipulación. Saber en cuál estás parado evita la mitad de los errores de principiante.

Propiedades de campo útiles

Debajo de la cuadrícula de la vista Diseño, cada campo tiene propiedades que afinan su comportamiento. Tres valen oro desde el primer día.

Propiedad	Qué hace	Uso en el Copetín
Requerido	Obliga a completar el campo	El precio de un producto no puede quedar vacío
Formato	Define cómo se muestra el dato	El precio como moneda en guaraníes
Valor predeterminado	Propone un valor al crear el registro	La fecha de hoy en cada venta nueva
Regla de validación	Rechaza valores fuera del dominio	Cantidad mayor que cero en DetalleVenta

Ejemplo resuelto — cargar los datos del Copetín, tabla por tabla

Con las cuatro tablas creadas y relacionadas, llega la carga de datos. El orden NO es libre: la integridad referencial exige cargar primero las tablas a las que otras apuntan. La figura muestra el orden correcto; los pasos lo ejecutan.



Figura 18.2 — El orden de carga que exige la integridad referencial: primero las tablas apuntadas, después las que apuntan.

Primero, Productos: los seis registros P01 a P06, con nombre, categoría y precio. No apunta a nadie: puede cargarse de entrada.

También primero, Clientes: C01 Rosa Duarte, C02 Luis Benítez, C03 Ana Gómez, C04 Miguel Ortiz. Tampoco apunta a nadie.

Segundo, Ventas: V1 a V5 con su fecha y su cliente. Ahora cada código de cliente que se tipea ya existe — Access lo acepta.

Tercero, DetalleVenta: las once filas (venta, producto, cantidad). Cada venta y cada producto referenciados ya existen. La base queda completa.

Prueba de fuego: intentá invertir el orden — cargar V1 antes que C01. Access rechaza el registro por integridad referencial: el orden de la figura no es una costumbre, es una consecuencia del diseño.

Aplicación profesional — Access como puerta de entrada

Access es una herramienta de base de datos de escritorio disponible en determinadas ediciones y suscripciones de Microsoft 365/Office para PC. Puede trabajar con archivos locales sin un servidor de base de datos dedicado; sus archivos tienen límites propios, por lo que la elección de herramienta debe basarse en volumen, concurrencia y necesidades del sistema. Los conceptos de tablas, claves, relaciones y consultas se transfieren a otros gestores.

Listas de valores y máscaras de entrada

Dos propiedades más ayudan a controlar la carga de datos. El Asistente para búsquedas puede mostrar una lista desplegable con valores como Panificados, Salados y Bebidas; para impedir valores fuera de esa lista debe configurarse la restricción correspondiente, como «Limitar a la lista», además de las validaciones que requiera el campo. La máscara de entrada, por su parte, controla el formato de escritura, por ejemplo un código de tres caracteres con letra inicial fija.

Ejemplo — la lista que elimina el error de tipeo

Sin lista, la categoría puede entrar como «salados», «Salado» o «SALADOS» — tres textos distintos para Access, y la consulta por categoría se fragmenta. Con la lista desplegable, entra siempre «Salados», idéntico, y la suma de G. 63.000 de la categoría sale entera de una sola consulta.

Importar datos: de la planilla a la base

Muchos negocios llegan a Access con su historia en planillas de Excel. La importación (pestaña Datos externos → grupo Importar y vincular → Excel) trae esos datos a una tabla, con un asistente que permite indicar la fila de encabezados, el tipo de cada columna y la clave principal. La regla de oro: importar hacia tablas que respeten el diseño normalizado — no arrastrar la «tabla única» de la planilla, sino repartir sus columnas entre Productos, Clientes y Ventas como aprendiste en la Unidad 3.

Ejemplo resuelto — del Excel del Copetín a la base normalizada

1. Punto de partida: una planilla con la tabla única — once filas con venta, fecha, cliente, producto, categoría, precio y cantidad, todo junto.
2. Preparación: en Excel se separan tres hojas — productos únicos (6 filas), clientes únicos (4 filas) y ventas únicas (5 filas) — más la hoja de renglones (11 filas) con solo códigos y cantidades.
3. Importación en orden de integridad: primero Productos y Clientes, después Ventas, por último DetalleVenta — el mismo orden de la Figura 18.2, ahora importando.
4. Verificación de tipos en el asistente: precio como Moneda, fecha como Fecha y hora, códigos como Texto corto.
5. Control final: la consulta de recaudación total sobre lo importado devuelve G. 134.000 — idéntico a la suma de la planilla original. Migración sin pérdidas.

Una migración frecuente consiste en pasar datos de planillas a una base estructurada. Normalizar y depurar ANTES de importar evita trasladar duplicaciones, inconsistencias y columnas mal definidas al nuevo sistema.

Actividades de aplicación

Ejercitá:

1. Escribí los pasos para crear la tabla Productos con su clave principal en Access.
2. Indicá en qué pestaña de Access se definen las relaciones.
3. Explicá qué hace la opción «Exigir integridad referencial».
4. Indicá la clave principal y la clave foránea de la tabla Ventas.

Resolvé (caso Copetín Karumbé):

5. Diseñá la tabla Clientes del Copetín (campos, tipos y clave principal) tal como la crearías en Access.
6. Describí cómo crear la relación entre DetalleVenta y Productos activando la integridad referencial.
7. Indicá cómo configurarías las tres casillas de la relación Clientes–Ventas (exigir integridad, actualizar en cascada, eliminar en cascada) y por qué.
8. Proponé una regla de validación y su texto de validación para el campo cantidad de DetalleVenta.

Pensá y decidí (justificá tu respuesta):

9. «En Access, la integridad referencial se activa al crear la relación.»
10. «Crear tablas y claves es trabajo DML.»

Desafío:

11. Escribí el plan completo para armar en Access las cuatro tablas del Copetín y sus relaciones, en el orden correcto.

Práctica 18 — La base de la segunda semana, creada en Access

Nombre y Apellido: _____ Curso: _____ Fecha: ____ / ____ / ____

Competencia de la práctica
Crear en Access una base relacional completa: tablas con tipos y propiedades, clave compuesta, relaciones con integridad referencial y carga de datos en el orden correcto.
Entorno de trabajo: Microsoft Access.

Lo que necesitás saber

Crear → Diseño de tabla. Clave compuesta: seleccionar las dos filas con Ctrl y pulsar Clave principal. Relaciones: Herramientas de base de datos → Relaciones → Exigir integridad referencial.

El catálogo cambió: desde el 10/03/2026 la gaseosa cuesta G. 9.000. Las ventas guardan en precio_unitario lo que realmente se cobró.

Creá una base en blanco llamada Copetin_Semana2.accdb en tu carpeta de trabajo. Las tablas de datos están al final de esta práctica.

Actividad 1 — Las cuatro tablas, con sus propiedades

Objetivo: *Crear Productos, Clientes, Ventas y DetalleVenta con tipos de datos y propiedades que protejan los datos.*

Modelo — Diseño pedido
Productos: código (Texto corto, PK) · nombre (Texto corto, Requerido) · categoría (Texto corto, lista Panificados/Salados/Bebidas con Limitar a la lista = Sí) · precio (Moneda, regla >0)
Clientes: código (Texto corto, PK) · nombre (Texto corto, Requerido) · teléfono (Texto corto, no requerido)
Ventas: código (Texto corto, PK) · fecha (Fecha y hora) · cliente (Texto corto)
DetalleVenta: venta + producto (PK compuesta) · cantidad (Número, regla >0) · precio_unitario (Moneda)

Pasos:

1. Creá cada tabla en la vista Diseño con sus campos y tipos.
2. Configurá las propiedades indicadas (Requerido, Regla de validación con su texto, lista de categorías).
3. En DetalleVenta, creá la clave compuesta seleccionando venta y producto con Ctrl.
4. Guardá cada tabla con su nombre exacto.

Punto de control: Al abrir DetalleVenta en vista Diseño se ven dos iconos de clave; al escribir «Lácteos» en categoría, Access lo rechaza.

Actividad 2 — Las relaciones

Objetivo: *Definir las tres relaciones con integridad referencial.*

Modelo — Relaciones
Clientes.código → Ventas.cliente
Ventas.código → DetalleVenta.venta
Productos.código → DetalleVenta.producto
En las tres: Exigir integridad referencial: Sí · Actualizar en cascada los campos relacionados: Sí · Eliminar en cascada los registros relacionados: No

Pasos:

1. Abrí Herramientas de base de datos → Relaciones y agregá las cuatro tablas.
2. Arrastrá cada clave principal sobre su clave foránea y configurá las casillas.
3. Guardá el diseño de relaciones.

Punto de control: Las tres líneas muestran 1 y ∞ en sus extremos.

Actividad 3 — La carga de datos

Objetivo: *Cargar los datos de la semana en el orden que exige la integridad referencial.*

Modelo — Orden
1.º Productos y Clientes → 2.º Ventas → 3.º DetalleVenta

Pasos:

1. Cargá Productos (8) y Clientes (6). Hugo Cáceres no dio teléfono: dejá el campo vacío.
2. Cargá Ventas (6).
3. Cargá DetalleVenta (13) con el precio_unitario de la tabla de datos, no el del catálogo.
4. Probá cargar un renglón con el producto P09 y cancelalo.

Punto de control: El total de registros coincide con la suma de los registros indicados en las cuatro tablas de datos. Verificaste además que los valores históricos de DetalleVenta no fueron sustituidos automáticamente por los precios actuales del catálogo.

Desafío final (para quienes terminan antes)

Intentá cargar un renglón con cantidad 0 y otro con una venta inexistente (V20). Anotá el mensaje de cada rechazo y qué regla lo provocó.

Datos para cargar

Productos (catálogo actual)

código	nombre	categoría	precio
P01	Chipa	Panificados	G. 3.000
P02	Mbeju	Panificados	G. 5.000
P03	Empanada	Salados	G. 6.000
P04	Milanesa	Salados	G. 15.000
P05	Cocido	Bebidas	G. 4.000
P06	Gaseosa	Bebidas	G. 9.000
P07	Jugo natural	Bebidas	G. 7.000
P08	Sopa paraguaya	Panificados	G. 6.000

Clientes

código	nombre	teléfono
C01	Rosa Duarte	0981 401 220
C02	Luis Benítez	0982 315 774
C03	Ana Gómez	0971 662 109
C04	Miguel Ortiz	0983 590 418
C05	Teresa Villalba	0985 207 336
C06	Hugo Cáceres	(sin teléfono)

Ventas

código	fecha	cliente
V6	09/03/2026	C02
V7	09/03/2026	C05
V8	10/03/2026	C01
V9	10/03/2026	C06
V10	11/03/2026	C05
V11	11/03/2026	C03

DetalleVenta

venta	producto	cantidad	precio_unitario
V6	P03	2	G. 6.000
V6	P07	1	G. 7.000
V7	P01	3	G. 3.000
V7	P05	2	G. 4.000
V8	P04	1	G. 15.000
V8	P06	1	G. 8.000
V8	P01	2	G. 3.000
V9	P08	2	G. 6.000
V9	P07	1	G. 7.000
V10	P02	1	G. 5.000
V10	P05	1	G. 4.000
V11	P03	4	G. 6.000
V11	P06	2	G. 9.000

Clase 19 — Filtros y consultas de selección

Capacidad	Reconoce los conceptos requeridos en la programación de base de datos utilizando un gerenciador de base de datos.
Tema	Filtros. La consulta de selección con criterios.
Indicadores de logro	• Aplica un filtro para ver un grupo de registros. • Crea una consulta de selección con criterio. • Interpreta el resultado de una consulta.

Filtrar: ver un grupo rápido

Un filtro muestra solo los registros que cumplen una condición, sin modificar la tabla. En la tabla Ventas, filtrar por la fecha completa 03/03/2026 muestra solo V1 y V2, las ventas de ese día.

La consulta de selección

Una consulta de selección es una pregunta guardada a la base: elige qué campos mostrar, de qué tablas, y con qué criterio. A diferencia del filtro, se guarda y se reutiliza, y puede combinar varias tablas.

Criterios

El criterio es la condición que deben cumplir los registros. Se escribe en la fila Criterios de la consulta: por ejemplo, #03/03/2026# en el campo fecha muestra solo las ventas registradas ese día.

La cuadrícula de diseño, paso a paso

En la pestaña Crear se elige Diseño de consulta, se agrega la tabla y aparece la cuadrícula de diseño: cada columna es un campo de la consulta. En la fila Campo se eligen los campos a mostrar; en Orden se puede ordenar ascendente o descendente; la casilla Mostrar decide si el campo se ve; y en Criterios se escribe la condición. El botón Ejecutar (signo de admiración rojo) corre la consulta y muestra el resultado como una hoja de datos.

Los criterios siguen reglas simples: los números se escriben directos (por ejemplo, > 5000 en el precio de Productos), los textos entre comillas ("Bebidas") y las fechas entre numerales (#04/03/2026#). Se pueden combinar criterios en varias columnas: todos en la misma fila se exigen a la vez (Y); en filas distintas, alcanza con cumplir uno (O).

La figura muestra la cuadrícula ya completa para la consulta «ventas del 03/03/2026»: el criterio #03/03/2026# se escribe en la fila Criterios del campo fecha y, al ejecutar, aparecen V1 y V2.



Figura 19.1 — La cuadrícula de diseño y su resultado: solo los registros que cumplen el criterio.

Consulta: ventas del 03/03/2026	Cliente
V1	C01
V2	C02

Ejemplo — una consulta con criterio

Para ver las ventas del 03/03/2026 se crea una consulta sobre Ventas y se escribe #03/03/2026# en la fila Criterios del campo fecha. El resultado son V1 (C01) y V2 (C02). El filtro mostraría momentáneamente el mismo subconjunto; la consulta queda guardada para reutilizarla.

Concepto clave — filtro y consulta

El filtro es temporal y se aplica sobre una tabla; la consulta se guarda, se reutiliza y puede combinar varias tablas.

Operadores de comparación y comodines

El criterio es una condición escrita en el idioma de Access. Sus piezas básicas son los operadores de comparación (mayor, menor, igual, distinto, entre) y los comodines para texto, que permiten buscar por patrones: el asterisco reemplaza cualquier cantidad de caracteres. La figura reúne tres criterios típicos ya escritos, con su resultado sobre los datos reales.



Figura 19.2 — Tres criterios escritos en la cuadrícula y lo que devuelve cada uno sobre los datos del Copetín.

Operador	Significa	Ejemplo de criterio
> < >= <=	Mayor, menor, mayor o igual, menor o igual	> 5000
= <>	Igual, distinto	<> "Bebidas"
Entre … Y …	Dentro de un rango, incluidos los bordes	Entre 15000 Y 40000
Como "…*"	Texto que responde a un patrón	Como "P*" (empieza con P)
Es Nulo	El campo está vacío	Es Nulo (detecta datos faltantes)

Ordenar y ocultar columnas en la cuadrícula

La cuadrícula de diseño tiene dos filas discretas pero poderosas. La fila Orden permite ordenar el resultado (ascendente o descendente) por la columna elegida — el precio de los productos de mayor a menor, por ejemplo. Y la casilla Mostrar permite usar un campo para filtrar sin exhibirlo: la consulta puede filtrar por categoría y mostrar solo nombre y precio, con la categoría trabajando desde bambalinas.

Ejemplo resuelto — tres consultas sobre las ventas reales

Diseñemos tres consultas de selección sobre la base del Copetín y anticipemos su resultado exacto — anticipar el resultado es el mejor control de calidad de una consulta.

Consulta 1 — ventas del primer día: tabla Ventas, criterio = #03/03/2026# en fecha. Resultado esperado: V1 (C01) y V2 (C02). Dos registros.

Consulta 2 — ventas del 04/03/2026: tabla Ventas, criterio = #04/03/2026# en fecha. Resultado esperado: V3 (C01) y V4 (C03). Dos registros.

Consulta 3 — productos de la categoría Bebidas: tabla Productos, criterio = "Bebidas", casilla Mostrar de la categoría desactivada. Resultado esperado: Cocido (G. 4.000) y Gaseosa (G. 8.000), mostrando solo nombre y precio.

1. Ejecución y verificación: si algún resultado difiere de lo anticipado, el error está en el criterio o en la anticipación — y encontrarlo es exactamente el aprendizaje.

Concepto clave — anticipá antes de ejecutar

El profesional no ejecuta a ver qué sale: calcula primero qué DEBERÍA salir y usa la ejecución para confirmar. La consulta valida el cálculo y el cálculo valida la consulta — el mismo control cruzado que usa la contabilidad.

Errores frecuentes — fechas en las consultas

1. Día y mes cruzados. En la cuadrícula de diseño, Access interpreta la fecha según la configuración regional del equipo (en Paraguay, día/mes/año). En la vista SQL, en cambio, los literales entre numerales se escriben siempre mes/día/año: #04/03/2026# en la cuadrícula es el 4 de marzo y en SQL aparece como #3/4/2026#. En este libro, todos los criterios de fecha están escritos en la cuadrícula de Diseño de un equipo configurado con día/mes/año.

2. Probar con fechas engañosas. El 03/03 se lee igual en los dos órdenes y no delata el error. Para verificar una consulta de fechas, probá con un día mayor que 12 (por ejemplo, el 15/03), que no admite dos lecturas.

3. Escribir la fecha como texto. "04/03/2026" entre comillas no es una fecha: la consulta no devuelve nada o da un error de tipos. Las fechas van entre numerales.

Consultas sobre fechas: rangos y precisión

Las fechas merecen capítulo propio porque son el filtro más usado de cualquier negocio: el día, la semana, el mes. En los criterios, las fechas van entre numerales (#04/03/2026#) y se combinan con los operadores ya conocidos: un día exacto con el igual, un rango con Entre … Y …, todo lo posterior a una fecha con el mayor.

Criterio de fecha	Qué devuelve en el Copetín
= #03/03/2026#	V1 y V2 (las ventas del primer día)
Entre #03/03/2026# Y #04/03/2026#	V1, V2, V3 y V4 (dos días completos)
> #03/03/2026#	V3, V4 y V5 (todo lo posterior al primer día)
= Fecha()	Las ventas de hoy (la función Fecha() devuelve el día actual)

Guardar, nombrar y documentar consultas

Una consulta guardada es un activo del sistema: se ejecuta muchas veces, la heredan otros usuarios y puede servir como origen de informes. Merece entonces un nombre que diga qué hace (ConsVentasPorFecha, ConsProductosPorCategoria) y no un «Consulta1» indescifrable. En bases con muchas consultas, un prefijo por tipo (Cons, Par, Tot) mantiene el panel de navegación legible.

Ejemplo resuelto — las ventas del primer día, con control

1. Objetivo: localizar las ventas registradas el 03/03/2026.
2. Diseño: consulta sobre Ventas, criterio = #03/03/2026# en fecha, mostrando código, fecha y cliente.
3. Anticipación del resultado: deben aparecer V1 (cliente C01) y V2 (cliente C02).
4. Ejecución: aparecen dos registros, tal como se anticipó.
5. Control por complemento: la base tiene cinco ventas; si V1 y V2 son las del 03/03/2026, las otras tres deben ser V3, V4 y V5. Si aparece una venta de más o falta una, hay que revisar el criterio o las fechas cargadas.

Cierre profesional: la consulta se guarda como ConsVentasDelDia — mañana, cambiando la fecha del criterio (o volviéndola paramétrica), servirá para cualquier día.

Concepto clave — el complemento como control

Verificar «el resto» (lo que NO devolvió la consulta) es un control tan potente como verificar el resultado: si la parte y el complemento no suman el total, algo quedó afuera o entró de más. Es el mismo principio del arqueo de caja: lo contado más lo faltante debe dar lo esperado.

Actividades de aplicación

Ejercitá:

1. Explicá la diferencia entre un filtro y una consulta.
2. Indicá dónde se escribe el criterio en una consulta.
3. Escribí el criterio para mostrar los productos cuyo precio esté entre G. 4.000 y G. 6.000, incluidos los bordes, e indicá cuáles aparecen.
4. Interpretá: ¿qué devuelve una consulta de Ventas con criterio fecha >= #04/03/2026#?

Resolvé (caso Copetín Karumbé):

5. Diseñá (en papel) una consulta sobre Productos que muestre nombre y precio de los productos que NO son Bebidas, ordenados del más caro al más barato. Indicá el resultado.

6. Escribí los criterios para listar las ventas del cliente C01 realizadas después del 03/03/2026 e indicá el resultado.

7. Una compañera escribió "04/03/2026" (entre comillas) como criterio de fecha y no obtuvo resultados. Explicá el error y corregilo.

Pensá y decidí (justificá tu respuesta):

8. «Un filtro modifica los datos de la tabla.»

9. «Una consulta de selección se guarda y se puede reutilizar.»

Desafío:

10. Diseñá una consulta que detecte los productos sin categoría cargada y explicá por qué el criterio = "" no sirve.

Autoevaluación antes de la Práctica 19

Marcá con una X. Si algo queda en «Todavía no», repasá esa parte de la clase antes de empezar la Práctica 19.

Puedo…	Sí	Todavía no
Aplica un filtro para ver un grupo de registros.		
Crea una consulta de selección con criterio.		
Interpreta el resultado de una consulta.		

Práctica 19 — Filtros y consultas sobre la segunda semana

Nombre y Apellido: _____ Curso: _____ Fecha: ____ / ____ / ____

Competencia de la práctica
Aplicar filtros y diseñar consultas de selección con criterios de comparación, rango, texto, nulos y fechas, anticipando el resultado y verificándolo.
Entorno de trabajo: Microsoft Access.

Lo que necesitás saber

Números directos (> 6000), textos entre comillas ("Bebidas"), fechas entre numerales (#10/03/2026#, escrita en la cuadrícula de Diseño con configuración día/mes/año). Misma fila = Y; filas distintas = O. Es Nulo encuentra campos vacíos.

Abrí Copetin_Semana2.accdb (Práctica 18). Antes de ejecutar cada consulta, escribí en tu hoja qué esperás que devuelva.

Actividad 1 — Filtrar sin consultar

Objetivo: *Usar el filtro por selección en la hoja de datos de Ventas.*

Modelo — Pedido
Ver solo las ventas del 10/03/2026; luego quitar el filtro.

Pasos:

1. Abrí Ventas en vista Hoja de datos.
2. Hacé clic derecho sobre una fecha 10/03/2026 → «Es igual a 10/03/2026».
3. Anotá cuántas ventas se ven y quitá el filtro con Alternar filtro.

Punto de control: El filtro muestra solamente las ventas de la fecha elegida y, al quitarlo, reaparece el conjunto completo sin que ningún registro haya sido modificado.

Actividad 2 — Cuatro consultas con criterio

Objetivo: *Diseñar, anticipar, ejecutar y guardar cuatro consultas.*

Modelo — Consultas pedidas
ConsProductosCaros: nombre y precio de los productos de más de G. 6.000, del más caro al más barato.
ConsBebidasEconomicas: bebidas de menos de G. 8.000 (dos criterios en la misma fila).
ConsClientesSinTelefono: clientes con el teléfono vacío.
ConsVentas9y10: ventas entre el 09/03/2026 y el 10/03/2026, incluidos los bordes.

Pasos:

1. Crear → Diseño de consulta; agregá la tabla y bajá los campos.
2. Escribí el criterio de cada consulta y, si corresponde, el orden.
3. Ejecutá y compará con lo que anticipaste.
4. Guardá cada consulta con su nombre.

Punto de control: Cada resultado coincide con lo anticipado. Los registros devueltos y los no devueltos, juntos y sin superposición, reconstruyen el conjunto completo de Ventas.

Actividad 3 — La fecha por dentro

Objetivo: *Comprobar en la vista SQL cómo guarda Access las fechas del criterio.*

Modelo — Qué mirar
Inicio → Ver → Vista SQL de ConsVentas9y10

Pasos:

1. Abrí la vista SQL y copiá la línea WHERE.
2. Explicá por qué el 10/03 aparece escrito como 3/10.
3. Escribí qué fecha entendería Access si en la vista SQL alguien escribiera #10/03/2026#.

Punto de control: Tu explicación menciona el orden mes/día/año de la vista SQL.

Desafío final (para quienes terminan antes)

Diseñá una consulta con los productos cuyo nombre contiene la letra «a» y cuestan menos de G. 6.000. Anticipá el resultado antes de ejecutarla.

Clase 20 — Consultas paramétricas y con criterios

Capacidad	Reconoce los conceptos requeridos en la programación de base de datos utilizando un gerenciador de base de datos.
Tema	La consulta paramétrica. Criterios que se piden al usuario.
Indicadores de logro	• Explica qué es una consulta paramétrica. • Escribe un parámetro entre corchetes en el criterio. • Interpreta el resultado según el valor ingresado.

Una consulta que pregunta

Una consulta paramétrica pide un dato al usuario cada vez que se ejecuta, en lugar de tener el criterio fijo. Sirve para responder preguntas del tipo «las ventas de tal cliente» sin crear una consulta distinta para cada cliente.

El parámetro entre corchetes

El parámetro se escribe en la fila Criterios entre corchetes, con un texto que se muestra al usuario. Por ejemplo, en el campo cliente se escribe [Ingresá el código de cliente]. Al ejecutar, Access pregunta y filtra con la respuesta.

Consultas con más de una tabla

Una consulta puede combinar varias tablas relacionadas: si se agregan Ventas y Clientes, Access une los registros por la relación ya definida (cliente ↔ código) y permite mostrar la fecha de la venta junto al nombre del cliente. Por eso importa haber creado bien las relaciones: las consultas las aprovechan sin trabajo extra.

El parámetro también admite comodines: con el criterio Como [Ingresá parte del nombre] & "*", escribir «Ro» encuentra a Rosa. El asterisco reemplaza cualquier final de texto.

Cliente ingresado	Ventas	Fechas
C01 (Rosa Duarte)	V1, V3	03/03/2026, 04/03/2026
C04 (Miguel Ortiz)	V5	05/03/2026

Ejemplo — las ventas de un cliente

Con el parámetro [Ingresá el código de cliente], si se escribe C01 la consulta muestra V1 y V3; si se escribe C04, muestra V5. Una sola consulta sirve para todos los clientes sin calcular todavía totales.

Concepto clave — consulta paramétrica

Consulta cuyo criterio se pide al usuario al ejecutarla, escrito entre corchetes en la fila Criterios.

Combinar criterios: Y y O

Los criterios se combinan con dos lógicas. Escritos en la MISMA fila de la cuadrícula, se exigen todos a la vez (Y): cliente = "C01" Y fecha = #04/03/2026#, cuyo resultado es V3. Escritos en FILAS distintas (la fila «o»), alcanza con cumplir uno (O): cliente = "C01" O fecha = #03/03/2026#, que devuelve V1, V2 y V3. La figura contrasta ambos resultados.

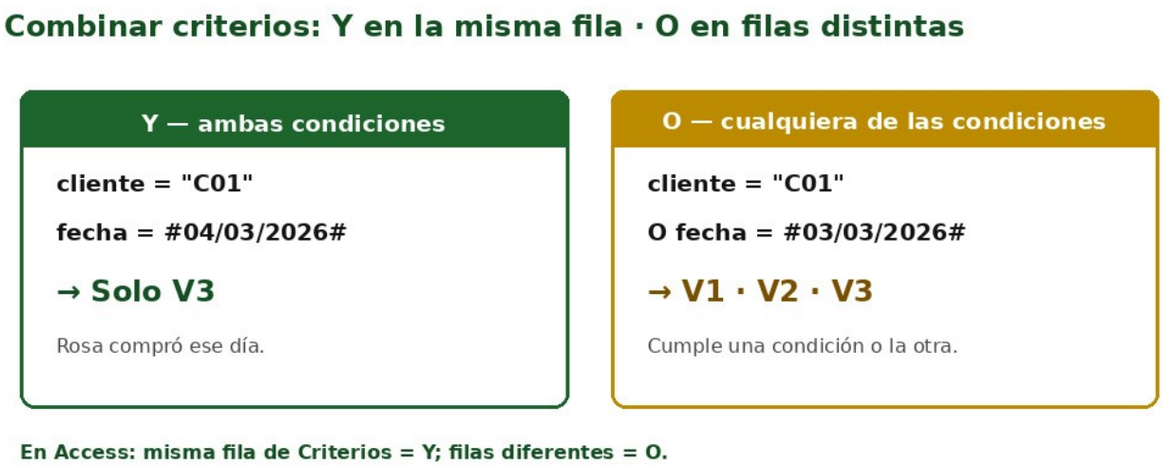


Figura 20.1 — La misma pareja de criterios, combinada con Y (misma fila) y con O (filas distintas): resultados muy distintos.

Combinación	Cómo se escribe	Resultado con los datos del Copetín
cliente = "C01" Y fecha = #04/03/2026#	Ambos criterios en la misma fila	Solo V3
cliente = "C01" O fecha = #03/03/2026#	Un criterio en cada fila	V1, V2 y V3

Ejemplo resuelto — la consulta multi-tabla del Copetín, paso a paso

Una consulta multi-tabla útil del Copetín puede usar varias tablas a la vez: «ingresá una categoría y te muestro los renglones de venta de esa categoría». La construimos sin totales; los agregados se enseñan recién en la Clase 21.

1. Agregar las cuatro tablas a la consulta: Ventas, DetalleVenta, Productos y Clientes. Access dibuja automáticamente las relaciones ya definidas.
2. Bajar a la cuadrícula los campos necesarios: código de venta (Ventas), categoría y nombre del producto (Productos), cantidad y precio_unitario (DetalleVenta), y nombre del cliente (Clientes).
3. Escribir el criterio paramétrico en la categoría: [Ingresá la categoría]. Al ejecutar, Access preguntará el valor.
4. Probar con «Salados»: aparecen tres renglones de detalle, correspondientes a la milanesa de V2, las empanadas de V3 y las milanesas de V5. La consulta selecciona; todavía no suma.
5. Probar con «Bebidas»: aparecen cinco renglones de detalle: gaseosa de V1, cocido de V2, gaseosa de V3, gaseosa de V5 y cocido de V5.
6. Control de alcance: al cambiar el parámetro cambia el subconjunto de renglones, pero la consulta sigue siendo la misma. El cálculo de subtotales y la recaudación por categoría se incorporan en la Clase 21, cuando se enseñan consultas de totales.

Concepto clave — el parámetro multiplica el valor

Sin parámetro, harían falta varias consultas casi idénticas (una por valor) y habría que crear otra si aparece uno nuevo. Con parámetro, una sola consulta responde distintos valores. Menos objetos y menos mantenimiento; los agregados se estudian en la Clase 21.

Detrás de la cuadrícula: la vista SQL

La cuadrícula de diseño es un traductor amable: cada consulta que armás se representa mediante SQL, el lenguaje estándar ampliamente utilizado en los gestores relacionales. La vista SQL (botón Ver → Vista SQL) muestra la instrucción con palabras clave como SELECT, FROM y WHERE. No hace falta escribirla todavía; sí vale la pena leerla. Cada gestor puede incorporar extensiones o particularidades de su propio dialecto SQL.

Concepto clave — la cuadrícula enseña SQL sin decirlo

Cada columna de la cuadrícula es un campo del SELECT; cada tabla agregada, parte del FROM; cada criterio, una condición del WHERE. Quien domina la cuadrícula ya piensa en SQL — solo le falta la sintaxis, que es lo más fácil de aprender.

Parámetros con comodines: buscar por parte del texto

El parámetro simple exige el valor exacto: C01, Salados. Combinado con el comodín, se vuelve un buscador flexible: el criterio Como "*" & [Escribí parte del nombre] & "*" encuentra los registros cuyo texto CONTIENE lo tipeado, en cualquier posición. Escribir «duar» encuentra a Rosa Duarte; escribir «gas», a la gaseosa.

Criterio paramétrico	Qué exige	Ejemplo de uso
[Ingresá el código]	Coincidencia exacta	C01 devuelve solo a Rosa
Como [Letra inicial] & "*"	Que empiece con lo tipeado	«M» devuelve Mbeju y Milanesa
Como "*" & [Parte del nombre] & "*"	Que contenga lo tipeado	«an» devuelve Empanada (en Productos) o Ana Gómez (en Clientes)

Ejemplo resuelto — el buscador de productos del Copetín

1. Necesidad: el cajero quiere encontrar productos tipeando cualquier parte del nombre, sin saber códigos.

2. Diseño: consulta sobre Productos, mostrando código, nombre y precio; criterio en el nombre: Como "*" & [Buscar producto:] & "*".

Prueba 1 — se tipea «chi»: devuelve Chipa (G. 3.000). Un registro.

Prueba 2 — se tipea «a»: devuelve Chipa, Empanada, Milanesa y Gaseosa — todos los nombres que contienen la letra. Cuatro registros.

Prueba 3 — se tipea «zz»: devuelve vacío, sin error. El buscador degrada con elegancia.

3. Entrega: guardada como ParBuscadorProductos, queda lista para conectarse a un formulario — el botón «Buscar» de cualquier sistema comercial es, por dentro, exactamente esto.

Aplicación profesional — búsquedas flexibles

Buscar por nombre, código o parte del texto es un patrón común en sistemas de gestión. Una consulta con comodines permite reutilizar el mismo diseño para localizar coincidencias parciales sin crear una consulta diferente para cada búsqueda.

Guardar y documentar consultas paramétricas

Una consulta paramétrica conviene guardarla con un nombre que describa la pregunta que responde, por ejemplo ParVentasPorCliente o ParVentasPorFecha. Documentá qué parámetro solicita, sobre qué campo trabaja y qué columnas devuelve. Así la misma consulta puede reutilizarse en otros objetos (un formulario de búsqueda, un informe) sin que nadie tenga que adivinar qué pide ni qué devuelve.

Actividades de aplicación

Ejercitá:

1. Definí consulta paramétrica.
2. Escribí cómo se anota un parámetro en el criterio.
3. Indicá una ventaja de una consulta paramétrica frente a una fija.
4. Interpretá: si en el parámetro de cliente se ingresa C02, ¿qué muestra?

Resolvé (caso Copetín Karumbé):

5. Escribí un criterio paramétrico que pida una fecha inicial y una final, e indicá el resultado si se ingresan 04/03/2026 y 05/03/2026.
6. Con el buscador Como "*" & [Buscar producto:] & "*", indicá qué devuelve si se tipea «e».
7. Escribí cómo pedirías las ventas del cliente que ingrese el usuario y del 04/03/2026 a la vez, e indicá el resultado para C03.

Pensá y decidí (justificá tu respuesta):

8. «Una consulta paramétrica tiene el criterio fijo.»
9. «El parámetro se escribe entre corchetes en la fila Criterios.»

Desafío:

10. Diseñá una consulta paramétrica que pida una fecha y muestre las ventas de ese día; indicá el resultado si se ingresa 05/03/2026.

Práctica 20 — Consultas que preguntan, sobre la segunda semana

Nombre y Apellido: _____ Curso: _____ Fecha: ____ / ____ / ____

Competencia de la práctica

Crear consultas paramétricas simples, con comodines y multi-tabla, y probarlas con valores que dan resultados, ninguno o varios.

Entorno de trabajo: Microsoft Access.

Lo que necesitás saber

El parámetro va entre corchetes en la fila Criterios. Con Como y el asterisco se busca por parte del texto. Una consulta con varias tablas usa las relaciones ya definidas.

Abrí Copetin_Semana2.accdb. Probá cada consulta con todos los valores pedidos, incluso los que parecen no tener sentido.

Actividad 1 — Las ventas de un cliente

Objetivo: *Crear ParVentasPorCliente y probarla con tres clientes.*

Modelo — Diseño

Tabla Ventas · campos código, fecha y cliente · en cliente: [Ingresá el código de cliente]

Pasos:

1. Diseñá y guardá la consulta.
2. Ejecutala con C05, con C06 y con C04.
3. Anotá cuántas ventas devuelve cada prueba.

Punto de control: La consulta produce una respuesta coherente tanto para clientes con ventas como para un cliente que no tenga registros coincidentes.

Actividad 2 — Renglones por categoría y un buscador

Objetivo: *Crear una consulta paramétrica multi-tabla y un buscador con comodines.*

Modelo — Consultas

ParRenglonesPorCategoria: Ventas + DetalleVenta + Productos + Clientes; muestra venta, nombre del cliente, producto, cantidad y precio_unitario; parámetro en categoría.

ParBuscador: Productos; en nombre: Como "*" & [Buscar producto:] & "*".

Pasos:

1. Agregá las cuatro tablas y verificá que Access dibuje las relaciones.
2. Probá ParRenglonesPorCategoria con «Panificados» y con «Bebidas».
3. Probá ParBuscador con «jug» y con «pa».

Punto de control: La cantidad de renglones de «Panificados» más los de «Bebidas» más los de «Salados» da la cantidad total de renglones de DetalleVenta.

Desafío final (para quienes terminan antes)

Creá ParVentasEntreFechas, que pida una fecha inicial y una final, y probala con 10/03/2026 y 11/03/2026.

Clase 21 — Totales, campos calculados y referencias cruzadas

Capacidad	Reconoce los conceptos requeridos en la programación de base de datos utilizando un gerenciador de base de datos.
Tema	Consultas de totales: Suma, Promedio y Cuenta en Access (Sum/Avg/Count en SQL), campos calculados y referencias cruzadas.
Indicadores de logro	• Calcula totales agrupando registros. • Crea un campo calculado en una consulta. • Interpreta una tabla de referencias cruzadas.

Consultas de totales

Una consulta de totales agrupa registros y calcula. En la interfaz de Access aparecen opciones como Suma, Promedio, Cuenta, Máx y Mín; en SQL las funciones habituales son Sum(), Avg(), Count(), Max() y Min(). Con la fila Total activada se decide qué campo agrupa y qué operación se aplica.

Total recaudado por cliente	Total (G.)
Rosa Duarte	48.000
Luis Benítez	19.000
Ana Gómez	17.000
Miguel Ortiz	50.000
TOTAL	134.000

La recaudación total del Copetín es G. 134.000 en 5 ventas, con una venta promedio de G. 26.800.

Función de la fila Total	Qué calcula	Sobre el Copetín devuelve
Agrupar por (categoría)	Un renglón por valor distinto	Panificados, Salados, Bebidas
Suma (Subtotal)	La suma dentro de cada grupo	23.000, 63.000 y 48.000
Cuenta (renglón de detalle)	Cuántos renglones hay en cada grupo	Cantidad de líneas de detalle; no equivale necesariamente a ventas distintas
Promedio (TotalVenta)	Promedio de una consulta previa TotalPorVenta	G. 26.800 por venta, calculado en una segunda consulta
Máx / Mín (TotalVenta)	Extremos de la consulta previa TotalPorVenta	Mayor G. 50.000; menor G. 14.000

Concepto clave — del registro al resumen

Una consulta de selección devuelve registros. Una consulta de totales agrupa y resume. Para calcular el promedio, máximo o mínimo POR VENTA no se deben promediar directamente los renglones de DetalleVenta: primero se crea TotalPorVenta con Suma(Subtotal) agrupada por venta y luego una segunda consulta aplica Promedio, Máx o Mín sobre TotalVenta.

Las dos consultas de totales de la figura agrupan las mismas 5 ventas por caminos distintos — por cliente y por categoría — y ambas suman la misma recaudación: G. 134.000.



Figura 21.1 — Dos consultas de totales sobre las mismas ventas: el control cruzado del Copetín.

Campos calculados

Un campo calculado genera un dato nuevo a partir de otros, con una fórmula. En DetalleVenta, el subtotal se calcula como cantidad × precio_unitario; no se guarda, se calcula al mostrar.

Referencias cruzadas

Una consulta de referencias cruzadas (tabla de doble entrada) resume los datos cruzando dos dimensiones. En el Copetín, las filas representan clientes, las columnas categorías y cada celda muestra Suma(Subtotal).

Importe por categoría	Total (G.)
Panificados	23.000
Bebidas	48.000
Salados	63.000
TOTAL	134.000

Un paso más: formularios e informes

Access completa el sistema con dos objetos más. El formulario es una pantalla amigable para cargar datos (en vez de escribir en la tabla directamente): con el asistente de la pestaña Crear se genera en minutos un formulario de carga de ventas. El informe presenta los resultados listos para imprimir: un informe basado en la consulta de totales por categoría se convierte en el reporte mensual del negocio. Formularios e informes se apoyan en las tablas y consultas que ya construiste: son la cara visible del sistema.

Ejemplo — el reporte final del Copetín

Agrupando por cliente se obtiene la recaudación total: G. 134.000. El mismo total sale agrupando por producto o por categoría (Panificados 23.000, Salados 63.000, Bebidas 48.000). Que los tres caminos den lo mismo confirma que la base está bien diseñada.

Concepto clave — totales, cálculos y cruces

Suma/Promedio/Cuenta en Access (Sum/Avg/Count en SQL) agregan datos; el campo calculado deriva valores con una fórmula y las referencias cruzadas los presentan en doble entrada.

Ejemplo cotidiano — de los datos a las decisiones

Con estas consultas, el dueño del Copetín sabe cuánto recaudó, qué producto vende más y qué categoría le rinde. Los datos, bien consultados, guían las decisiones del negocio.

La consulta de referencias cruzadas, paso a paso

La referencia cruzada responde dos preguntas a la vez: agrupa por un campo en las filas y por otro en las columnas, y calcula en cada cruce. Para el Copetín, la gran cruzada es clientes por categorías: quién compró

cuánto de cada cosa. La figura muestra el resultado completo — y su doble control: cada fila suma el total del cliente y cada columna, el total de la categoría.



Figura 21.2 — La tabla cruzada del Copetín: clientes en filas, categorías en columnas y la recaudación total en la esquina.

1. Crear una consulta con las cuatro tablas y el subtotal (cantidad × precio_unitario) como campo calculado.

2. Convertirla en referencia cruzada (con Crear → Asistente para consultas → Asistente para consultas de referencias cruzadas o, en la vista Diseño, con Tipo de consulta → Tabla de referencias cruzadas): el nombre del cliente como encabezado de fila; la categoría como encabezado de columna; la suma del subtotal como valor.

3. Leer el resultado por filas: Rosa 6.000 + 18.000 + 24.000 = G. 48.000; Luis 15.000 + 4.000 = G. 19.000; Ana G. 17.000; Miguel 30.000 + 20.000 = G. 50.000.

4. Leer el resultado por columnas: Panificados G. 23.000; Salados G. 63.000; Bebidas G. 48.000.

5. Doble control: filas y columnas convergen en la misma esquina — G. 134.000. Si un solo cruce estuviera mal, las dos sumas no coincidirían: la cruzada trae su propio auditor incorporado.

Concepto clave — la esquina que audita

Que el total por clientes y el total por categorías coincidan al guaraní no es casualidad: son dos particiones de las MISMAS once filas de DetalleVenta. Toda cifra de gestión seria se verifica así, por caminos independientes que deben encontrarse.

Informes: presentar los resultados

El último eslabón es mostrar. Un informe de Access toma una tabla o consulta y la presenta lista para imprimir o exportar: con títulos, agrupamientos, subtotales y formato. El informe de recaudación por categoría del Copetín se crea en minutos desde la consulta ya hecha (pestaña Crear → grupo Informes) — porque el trabajo duro, el de los datos correctos, ya está hecho.

Consultas de acción: modificar en masa

Hasta acá, todas las consultas leían. Las consultas de acción escriben: actualizan, agregan o eliminan MUCHOS registros de una sola vez. Son poderosas y por eso mismo delicadas: una consulta de actualización mal filtrada modifica todo lo que alcanza, sin preguntar registro por registro. El protocolo profesional es fijo: respaldar, probar el filtro primero como consulta de selección (para VER qué alcanzaría) y recién entonces ejecutar la acción.

Consulta de acción	Qué hace	Precaución central
De actualización	Modifica un campo en los registros que cumplen el criterio	Probar el criterio como selección antes
De datos anexados	Agrega registros desde otra tabla o consulta	Verificar tipos y claves antes de anexar
De eliminación	Borra los registros que cumplen el criterio	Respaldo previo, siempre
De creación de tabla	Guarda un resultado como tabla nueva	Útil para instantáneas históricas

Ejemplo resuelto — un aumento simulado, con protocolo completo

Escenario hipotético de práctica: el Copetín evalúa aumentar 10% los Panificados. Simulemos el protocolo completo (es un ensayo — los precios vigentes del caso siguen siendo los de la tabla de Productos).

1. Respaldo: copia de la base antes de tocar nada.

2. Ensayo del filtro como consulta de selección: criterio = "Panificados" en la categoría. Devuelve Chipa (G. 3.000) y Mbeju (G. 5.000): exactamente los dos registros que deben cambiar, ninguno de más.

3. Conversión a consulta de actualización: el campo precio se actualizaría a precio × 1,1. Anticipación: la chipa pasaría a G. 3.300 y el mbeju a G. 5.500.

4. Ejecución simulada y verificación: dos registros afectados — coincide con el ensayo del filtro. Los Salados y las Bebidas, intactos.

Cierre del ensayo: se restaura el respaldo y los precios vuelven a los vigentes del caso. El protocolo — respaldar, ensayar, anticipar, ejecutar, verificar — queda incorporado; es el mismo con 6 productos que con 6.000.

Exportar: los resultados salen a trabajar

El destino final de muchas consultas está fuera de Access: la pestaña Datos externos exporta cualquier tabla, consulta o informe a Excel (para seguir analizando), a PDF (botón PDF o XPS, para compartir sin que se modifique) o a texto (para otros sistemas). La tabla cruzada del Copetín, exportada a PDF, es un reporte de gestión terminado, listo para leer en cualquier pantalla.

> **Concepto clave — leer es libre, escribir es protocolo**
>
> La distinción final del libro: las consultas de lectura se pueden ensayar sin miedo — lo peor que devuelven es un resultado equivocado. Las de escritura modifican la realidad de la base: exigen respaldo, ensayo y verificación, siempre. Esa prudencia con los datos ajenos es, quizás, el aprendizaje más profesional de todos.

Actividades de aplicación

Ejercitá:

1. Nombrá tres operaciones de una consulta de totales.
2. Escribí la fórmula del subtotal como campo calculado.
3. Explicá qué muestra una consulta de referencias cruzadas.
4. Usá Cuenta: ¿cuántos renglones de detalle tiene cada categoría?

Resolvé (caso Copetín Karumbé):

5. Agrupando por producto, calculá cuántas unidades se vendieron de cada uno. ¿Coincide el producto más vendido en unidades con el que más ingresó?
6. Agrupando por fecha, calculá la recaudación de cada día y verificá que sumen la recaudación total.
7. Explicá por qué aplicar Promedio al Subtotal de los 11 renglones no da la venta promedio.

Pensá y decidí (justificá tu respuesta):

8. «Un campo calculado se guarda como dato en la tabla.»
9. «La suma por cliente, por producto y por categoría debe dar la misma recaudación total.»

Desafío:

10. Siguiendo el protocolo de la clase, describí una consulta de actualización que cambie a G. 8.500 el precio de catálogo de la gaseosa, y explicá por qué las ventas pasadas no cambian.

Práctica 21 — Totales, cálculos y cruces de la segunda semana

Nombre y Apellido: _____ Curso: _____ Fecha: ____ / ____ / ____

Competencia de la práctica
Construir consultas de totales, campos calculados y referencias cruzadas con controles cruzados, y detectar el error de usar el precio de catálogo en lugar del precio cobrado.
Entorno de trabajo: Microsoft Access.

Lo que necesitás saber

Subtotal: [cantidad]*[precio_unitario]. La fila Total (botón Totales) agrupa y calcula. La referencia cruzada pone un campo en filas, otro en columnas y un valor en el cruce.

Promedio por venta: primero se suma por venta (TotalPorVenta) y después se promedia.

Abrí Copetin_Semana2.accdb. En esta práctica vas a comprobar los mismos totales por varios caminos.

Actividad 1 — El detalle calculado

Objetivo: *Crear ConsDetalle con el campo calculado Subtotal.*

Modelo — Diseño
Tablas: Ventas, DetalleVenta, Productos, Clientes · campos: venta, fecha, cliente, nombre del cliente, categoría, producto, cantidad, precio_unitario · Subtotal: [cantidad]*[precio_unitario]

Pasos:

1. Diseñá la consulta con el campo calculado.
2. Ejecutala y revisá el subtotal de cada renglón de la venta V11.
3. Guardala: será la base de las demás.

Punto de control: ConsDetalle contiene un renglón por cada registro de DetalleVenta y ningún Subtotal queda vacío.

Actividad 2 — Totales por cuatro caminos

Objetivo: *Crear consultas de totales y verificar que coincidan.*

Modelo — Consultas sobre ConsDetalle
TotPorCliente · TotPorCategoria · TotPorProducto (con Suma de cantidad y de Subtotal) · TotalPorVenta y, sobre ella, PromedioVenta (Promedio, Máx y Mín de TotalVenta)

Pasos:

1. Creá cada consulta con el botón Totales: Agrupar por en el campo de grupo y Suma en Subtotal.
2. Anotá el total general de cada una.
3. Creá TotalPorVenta y luego PromedioVenta.

Punto de control: Los totales generales por cliente, por categoría y por producto son iguales entre sí. Además, el promedio multiplicado por la cantidad de ventas da ese mismo total.

Actividad 3 — La cruzada y la trampa del precio

Objetivo: *Armar la referencia cruzada y comprobar qué pasa si se usa el precio de catálogo.*

Modelo — Pasos
Referencia cruzada sobre ConsDetalle: nombre del cliente (fila) · categoría (columna) · Suma de Subtotal (valor).
Campo de prueba en una copia de ConsDetalle: SubtotalCatalogo: [cantidad]*[Productos].[precio]

Pasos:

1. Creá la cruzada con el asistente o desde la vista Diseño.
2. Verificá la suma de una fila y la de una columna.
3. Creá la copia con SubtotalCatalogo y sumalo.
4. Encontrá el renglón que explica la diferencia.

Punto de control: La cruzada coincide con TotPorCliente y TotPorCategoria. Comparaste la suma con precio de catálogo con la suma real y, si difieren, señalaste el renglón o los renglones que explican la diferencia.

Desafío final (para quienes terminan antes)

Creá un informe basado en TotPorCategoria (Crear → Informe) y exportalo a PDF (Datos externos → PDF o XPS) con el nombre Recaudacion_Semana2.pdf.

Evaluación de la Unidad 4

La base en acción: Access y consultas

Nombre y Apellido: _____ Curso: _____ Fecha: ____ / ____ / ____

Parte A — Diagnóstico rápido (marcá la opción correcta):

1. Crear tablas, claves y relaciones en Access es trabajo:
   a) DML
   b) de consulta
   c) DDL

2. Una consulta cuyo criterio se pide al usuario al ejecutarla es:
   a) de selección fija
   b) paramétrica
   c) de definición

3. Para sumar, promediar o contar agrupando registros se usa una consulta de:
   a) totales
   b) parámetros
   c) estructura

Parte B — Resolución y aplicación:

4. Escribí los pasos para crear la tabla DetalleVenta con su clave compuesta y relacionarla con Ventas y Productos exigiendo integridad referencial.

5. Escribí el criterio para una consulta que muestre las ventas del 05/03/2026 e indicá cuántos registros esperás.

6. Escribí un criterio paramétrico para consultar las ventas de un cliente e indicá qué devuelve si se ingresa C03.

7. En una jornada distinta, se vendieron: 3 chipas a G. 4.000 cada una; 2 mbeju a G. 7.000; 4 empanadas a G. 8.000; 2 milanesas a G. 16.000; 3 cocidos a G. 4.500; y 2 jugos a G. 8.500. Agrupá los importes en Panificados, Salados y Bebidas, y calculá la recaudación total.

8. Escribí la fórmula del subtotal como campo calculado y calculalo para 4 chipas a G. 3.000.

Evaluación integradora de la 2.ª etapa

Unidades 3 y 4 · Cierre de la 2.ª etapa (noviembre)

Nombre y Apellido: _____ Curso: _____ Fecha: ____ / ____ / ____

Parte A — Diagnóstico rápido (marcá la opción correcta):

1. El dibujo del modelo con entidades, atributos y relaciones es el:
   a) SGBD
   b) diagrama entidad-relación
   c) filtro

2. Llegar a la 3FN sirve para:
   a) aumentar la redundancia
   b) eliminar redundancia y anomalías
   c) crear vistas

3. Una consulta de totales permite:
   a) sumar y promediar agrupando
   b) definir tablas
   c) exigir integridad

Parte B — Resolución y aplicación:

4. Dibujá el DER del Copetín (Cliente, Venta, Producto) con la cardinalidad de cada relación y los atributos de la relación «contiene».

5. Tradució el DER a tablas: nombrá las cuatro tablas e indicá sus claves principales y foráneas.

6. Explicá, con un ejemplo del Copetín, una anomalía de actualización y cómo la evita la normalización.

7. Escribí el criterio paramétrico para consultar las ventas de un cliente e indicá el resultado para C02.

8. En un nuevo local se registraron cinco ventas: F1 G. 17.000, F2 G. 22.000, F3 G. 39.000, F4 G. 18.000 y F5 G. 57.000. F1 y F3 corresponden a Rosa; F2 a Luis; F4 a Ana; F5 a Miguel. Calculá la recaudación por cliente, el total y el promedio por venta.

Proyecto Final Integrador — La solución digital completa

Durante el año construiste, con las distintas materias, las piezas de un sistema para un emprendimiento paraguayo. En este proyecto final, que se presenta en la Feria de Informática, las juntás en una solución digital completa: la base de datos del emprendimiento, diseñada, normalizada y consultada, como corazón del sistema de gestión.

El emprendimiento de referencia es el Copetín Karumbé, pero tu equipo puede elegir otro (una librería, una cantina, un club, una ferretería del barrio). Lo importante es que la solución resuelva una necesidad real: registrar lo que el negocio hace y obtener información útil para decidir.

Competencia integradora

Integrar el diseño, la creación y la consulta de una base de datos en una solución digital que responda a las necesidades de gestión de un emprendimiento, y presentarla con claridad ante un público.

Lo que aportás desde Algorítmica

- El diagrama entidad-relación del emprendimiento y su esquema de tablas normalizado hasta 3FN.
- La base de datos creada en Access, con claves y relaciones con integridad referencial.
- Las consultas que responden las preguntas del negocio: recaudación total, ventas por cliente, producto más vendido, importe por categoría.

El mapa de la figura muestra cómo se arma la solución: el diseño alimenta la base de datos, la base responde con consultas (y, como extensión recomendada, con formularios e informes), y todo desemboca en la presentación de la feria junto con los aportes de las demás materias.

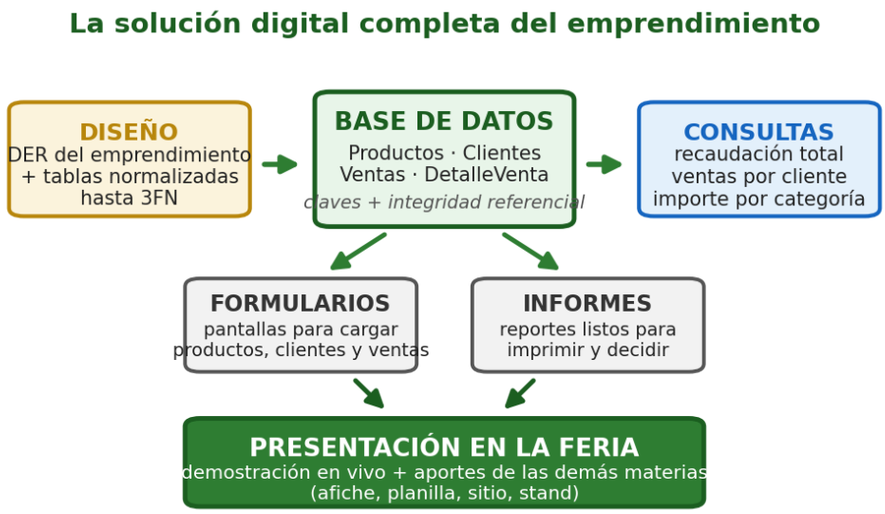


Figura PFI.1 — Mapa de la solución digital completa: del diseño a la Feria de Informática.

Requisitos mínimos de la solución

Componente	Qué debe tener
Diseño	Mini-mundo delimitado y lista de al menos cuatro preguntas del negocio · DER completo · al menos una relación N:M resuelta con tabla puente.
Normalización	Esquema en 3FN, con las dependencias funcionales principales escritas y justificadas.
Base en Access	Al menos cuatro tablas con tipos y propiedades · claves principales (una compuesta) · relaciones con integridad referencial · datos de prueba propios (no los del libro).
Consultas	Al menos cinco: una de selección con criterio, una paramétrica, una de totales, una con campo calculado y una de referencias cruzadas.
Presentación	Guion de demostración de cinco minutos.

Extensión recomendada: agregá al menos un formulario de carga y un informe. No son requisitos mínimos para aprobar el proyecto, pero mejoran la usabilidad y la presentación profesional de la solución.

Etapas del proyecto

Taller	Qué hacés
Taller 1 — Definición y diseño	Elegís el emprendimiento, relevás sus preguntas, dibujás el DER y lo llevás a tablas normalizadas.
Taller 2 — Construcción	Creás la base en Access, definís las relaciones, cargás datos de prueba y formulás las consultas.
Taller 3 — Integración y demostración	Sumás las piezas de las otras materias, verificás cada consulta con un control cruzado y ensayás la presentación.

Trabajo interdisciplinario: qué aporta cada materia

Materia	Aporte a la solución digital
Algorítmica	Diseño, creación y consulta de la base de datos del sistema.
Matemática Aplicada	Cálculos del negocio: precios, porcentajes, totales y promedios.
Gabinete de Software	Documentos, planillas y presentación del sistema para la feria.
Gabinete de Laboratorio	Puesta a punto del equipo y resguardo de los archivos del proyecto.
Dibujo Técnico	Identidad visual: logo, afiche y plano del stand a escala.

Qué entrega el equipo

- Carpeta del proyecto con el documento de diseño (mini-mundo, preguntas, DER, esquema y normalización).
- Archivo de la base (.accdb) con tablas, relaciones y consultas; el formulario y el informe, si se hicieron como extensión recomendada.
- Guion de la demostración y reparto de roles del equipo.

Pautas para el equipo

- Trabajen en equipos de 3 o 4 integrantes, con roles claros (diseño, base, consultas, presentación).
- Usen el mismo emprendimiento en todas las materias.
- Cuiden que cada consulta responda una pregunta real del negocio y que sus resultados se puedan verificar por otro camino.
- Preparen una explicación breve y clara: qué problema resuelve el sistema y qué información aporta al negocio.

Capacidad transversal del programa

Capacidad transversal MEC: Reconoce la importancia de la práctica Conductual en el desarrollo de las actividades. En el proyecto se evidencia al cumplir los procedimientos acordados, sostener el rol asignado en el equipo, resguardar los datos del emprendimiento, verificar cada consulta con un control cruzado y registrar los cambios en la carpeta del proyecto.

Algorítmica · Lenguajes y bases de datos

3.er Curso · Bachillerato Técnico en Servicios · Especialidad Informática

En este libro

- 21 clases desarrolladas con ejemplos del caso integrador
- 21 prácticas paso a paso en PSeInt, en papel y en Microsoft Access
- Diseño Entidad-Relación, normalización hasta 3FN y consultas
- Prueba diagnóstica, 4 evaluaciones de unidad y 2 de etapa
- Proyecto Final Integrador para la Feria de Informática

Para el docente

- Solucionario docente con todas las respuestas
- Plan anual de 36 encuentros (4 horas cátedra semanales)
- 36 planes de clase listos para usar

Enfoque «Aprender Haciendo»

Caso integrador: Copetín Karumbé

República del Paraguay · 2026